

ObjektinisUzdavinys v3.0

3.0

Sugeneruota Fri May 29 2026 04:11:10 ObjektinisUzdavinys v3.0 Doxygen 1.17.0

Fri May 29 2026 04:11:10

1 ObjektinisUzdavinys v3.0 - Setup ir Naudojimosi Instrukcija	1
2 Hierarchijos Indeksas	17
2.1 Klasių hierarchija	17
3 Klasės Indeksas	19
3.1 Klasės	19
4 Failo Indeksas	21
4.1 Failai	21
5 Klasės Dokumentacija	23
5.1 deque< T >::const_iterator Klasė	23
5.1.1 Smulkus aprašymas	23
5.2 list< T >::const_iterator Klasė	23
5.2.1 Smulkus aprašymas	23
5.3 string::const_iterator Klasė	23
5.3.1 Smulkus aprašymas	23
5.4 vector< T >::const_iterator Klasė	23
5.4.1 Smulkus aprašymas	23
5.5 deque< T >::const_reverse_iterator Klasė	23
5.5.1 Smulkus aprašymas	24
5.6 list< T >::const_reverse_iterator Klasė	24
5.6.1 Smulkus aprašymas	24
5.7 string::const_reverse_iterator Klasė	24
5.7.1 Smulkus aprašymas	24
5.8 vector< T >::const_reverse_iterator Klasė	24
5.8.1 Smulkus aprašymas	24
5.9 deque< T > Klasė Šablonas	24
5.9.1 Smulkus aprašymas	25
5.10 exception Klasė	25
5.10.1 Smulkus aprašymas	25
5.11 ifstream Klasė	25
5.11.1 Smulkus aprašymas	25
5.12 invalid_argument Klasė	25
5.12.1 Smulkus aprašymas	25
5.13 istream Klasė	25
5.13.1 Smulkus aprašymas	25
5.14 deque< T >::iterator Klasė	25
5.14.1 Smulkus aprašymas	25
5.15 list< T >::iterator Klasė	25
5.15.1 Smulkus aprašymas	26
5.16 string::iterator Klasė	26
5.16.1 Smulkus aprašymas	26

5.17 <code>vector< T >::iterator</code> Klasė	26
5.17.1 Smulkus aprašymas	26
5.18 <code>list< T ></code> Klasė Šablonas	26
5.18.1 Smulkus aprašymas	26
5.19 <code>ofstream</code> Klasė	26
5.19.1 Smulkus aprašymas	27
5.20 <code>ostream</code> Klasė	27
5.20.1 Smulkus aprašymas	27
5.21 <code>deque< T >::reverse_iterator</code> Klasė	27
5.21.1 Smulkus aprašymas	27
5.22 <code>list< T >::reverse_iterator</code> Klasė	27
5.22.1 Smulkus aprašymas	27
5.23 <code>string::reverse_iterator</code> Klasė	27
5.23.1 Smulkus aprašymas	27
5.24 <code>vector< T >::reverse_iterator</code> Klasė	27
5.24.1 Smulkus aprašymas	27
5.25 <code>runtime_error</code> Klasė	27
5.25.1 Smulkus aprašymas	28
5.26 <code>string</code> Klasė	28
5.26.1 Smulkus aprašymas	28
5.27 <code>stringstream</code> Klasė	28
5.27.1 Smulkus aprašymas	28
5.28 <code>Studentas</code> Klasė	28
5.28.1 Smulkus aprašymas	29
5.28.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	29
5.28.2.1 <code>Studentas()</code> [1/5]	29
5.28.2.2 <code>Studentas()</code> [2/5]	30
5.28.2.3 <code>Studentas()</code> [3/5]	30
5.28.2.4 <code>~Studentas()</code>	30
5.28.2.5 <code>Studentas()</code> [4/5]	30
5.28.2.6 <code>Studentas()</code> [5/5]	30
5.28.3 Metodų Dokumentacija	30
5.28.3.1 <code>addPaz()</code>	30
5.28.3.2 <code>clearPaz()</code>	30
5.28.3.3 <code>getEgz()</code>	30
5.28.3.4 <code>getInfo()</code>	30
5.28.3.5 <code>getMed()</code>	30
5.28.3.6 <code>getPaz()</code> [1/2]	31
5.28.3.7 <code>getPaz()</code> [2/2]	31
5.28.3.8 <code>getRez()</code>	31
5.28.3.9 <code>isPazEmpty()</code>	31
5.28.3.10 <code>operator=()</code> [1/2]	31

5.28.3.11 operator=() [2/2]	31
5.28.3.12 readStudent()	31
5.28.3.13 reservePaz()	31
5.28.3.14 setEgz()	31
5.28.3.15 setMed()	31
5.28.3.16 setPaz()	31
5.28.3.17 setRez()	32
5.28.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	32
5.28.4.1 operator<<	32
5.28.4.2 operator>>	32
5.29 Vector< T > Klasė Šablonas	32
5.29.1 Smulkus aprašymas	34
5.29.2 Tipo Aprašymo Dokumentacija	34
5.29.2.1 const_iterator	34
5.29.2.2 const_reverse_iterator	34
5.29.2.3 iterator	35
5.29.2.4 reverse_iterator	35
5.29.3 Konstruktoriaus ir Destruktoriaus Dokumentacija	35
5.29.3.1 Vector() [1/6]	35
5.29.3.2 Vector() [2/6]	35
5.29.3.3 Vector() [3/6]	35
5.29.3.4 Vector() [4/6]	35
5.29.3.5 ~Vector()	35
5.29.3.6 Vector() [5/6]	35
5.29.3.7 Vector() [6/6]	35
5.29.4 Metodų Dokumentacija	36
5.29.4.1 assign() [1/2]	36
5.29.4.2 assign() [2/2]	36
5.29.4.3 at() [1/2]	36
5.29.4.4 at() [2/2]	36
5.29.4.5 back() [1/2]	36
5.29.4.6 back() [2/2]	36
5.29.4.7 begin() [1/2]	36
5.29.4.8 begin() [2/2]	36
5.29.4.9 capacity()	37
5.29.4.10 cbegin()	37
5.29.4.11 cend()	37
5.29.4.12 clear()	37
5.29.4.13 crbegin()	37
5.29.4.14 crend()	37
5.29.4.15 data() [1/2]	37
5.29.4.16 data() [2/2]	37

5.29.4.17 <code>emplace()</code>	37
5.29.4.18 <code>emplace_back()</code>	38
5.29.4.19 <code>empty()</code>	38
5.29.4.20 <code>end()</code> [1/2]	38
5.29.4.21 <code>end()</code> [2/2]	38
5.29.4.22 <code>erase()</code> [1/4]	38
5.29.4.23 <code>erase()</code> [2/4]	38
5.29.4.24 <code>erase()</code> [3/4]	38
5.29.4.25 <code>erase()</code> [4/4]	38
5.29.4.26 <code>front()</code> [1/2]	38
5.29.4.27 <code>front()</code> [2/2]	39
5.29.4.28 <code>get_reallocation_count()</code>	39
5.29.4.29 <code>insert()</code> [1/4]	39
5.29.4.30 <code>insert()</code> [2/4]	39
5.29.4.31 <code>insert()</code> [3/4]	39
5.29.4.32 <code>insert()</code> [4/4]	39
5.29.4.33 <code>max_size()</code>	39
5.29.4.34 <code>operator"!=()</code>	40
5.29.4.35 <code>operator<()</code>	40
5.29.4.36 <code>operator<=()</code>	40
5.29.4.37 <code>operator=()</code> [1/2]	40
5.29.4.38 <code>operator=()</code> [2/2]	40
5.29.4.39 <code>operator==()</code>	40
5.29.4.40 <code>operator>()</code>	40
5.29.4.41 <code>operator>=()</code>	40
5.29.4.42 <code>operator[]()</code> [1/2]	41
5.29.4.43 <code>operator[]()</code> [2/2]	41
5.29.4.44 <code>pop_back()</code>	41
5.29.4.45 <code>push_back()</code> [1/2]	41
5.29.4.46 <code>push_back()</code> [2/2]	41
5.29.4.47 <code>rbegin()</code> [1/2]	41
5.29.4.48 <code>rbegin()</code> [2/2]	41
5.29.4.49 <code>rend()</code> [1/2]	41
5.29.4.50 <code>rend()</code> [2/2]	41
5.29.4.51 <code>reserve()</code>	42
5.29.4.52 <code>reset_reallocation_count()</code>	42
5.29.4.53 <code>resize()</code> [1/2]	42
5.29.4.54 <code>resize()</code> [2/2]	42
5.29.4.55 <code>shrink_to_fit()</code>	42
5.29.4.56 <code>size()</code>	42
5.29.4.57 <code>swap()</code>	42
5.30 <code>vector< T ></code> Klasė Šablonas	42

5.30.1 Smulkus aprašymas	43
5.31 VectorIterator< T > Klasė Šablonas	43
5.31.1 Smulkus aprašymas	44
5.31.2 Tipo Aprašymo Dokumentacija	44
5.31.2.1 difference_type	44
5.31.2.2 iterator_category	44
5.31.2.3 pointer	44
5.31.2.4 reference	44
5.31.2.5 value_type	44
5.31.3 Konstruktoriaus ir Destruktoriaus Dokumentacija	44
5.31.3.1 VectorIterator()	44
5.31.4 Metodų Dokumentacija	45
5.31.4.1 operator!=(())	45
5.31.4.2 operator*()	45
5.31.4.3 operator+()	45
5.31.4.4 operator++() [1/2]	45
5.31.4.5 operator++() [2/2]	45
5.31.4.6 operator+=()	45
5.31.4.7 operator-() [1/2]	45
5.31.4.8 operator-() [2/2]	45
5.31.4.9 operator--() [1/2]	46
5.31.4.10 operator--() [2/2]	46
5.31.4.11 operator-=()	46
5.31.4.12 operator->()	46
5.31.4.13 operator<()	46
5.31.4.14 operator<=()	46
5.31.4.15 operator==()	46
5.31.4.16 operator>()	46
5.31.4.17 operator>=()	46
5.31.4.18 operator[]()	47
5.32 VectorTest Klasė	47
5.32.1 Smulkus aprašymas	47
5.32.2 Atributų Dokumentacija	47
5.32.2.1 v	47
5.33 Zmogus Klasė	47
5.33.1 Smulkus aprašymas	48
5.33.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	48
5.33.2.1 Zmogus() [1/2]	48
5.33.2.2 Zmogus() [2/2]	48
5.33.2.3 ~Zmogus()	49
5.33.3 Metodų Dokumentacija	49
5.33.3.1 getInfo()	49

5.33.3.2 getPavarde()	49
5.33.3.3 getVardas()	49
5.33.3.4 setPavarde()	49
5.33.3.5 setVardas()	49
5.33.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	49
5.33.4.1 operator<<	49
5.33.5 Atributų Dokumentacija	50
5.33.5.1 pavarde_	50
5.33.5.2 vardas_	50
6 Failo Dokumentacija	51
6.1 duomenu_valdymas.cpp	51
6.2 input.cpp	56
6.3 mat_funkcijos.cpp	57
6.4 menu.cpp	58
6.5 output.cpp	60
6.6 reallocation_comparison.cpp	61
6.7 Studentas.cpp	61
6.8 testavimas.cpp	63
6.9 Zmogus.cpp	68
6.10 duomenu_valdymas.h	69
6.11 input.h	69
6.12 Mat_funkcijos.h	69
6.13 menu.h	69
6.14 output.h	69
6.15 Studentas.h	70
6.16 testavimas.h	71
6.17 Vector.h	71
6.18 Zmogus.h	79
6.19 objekt1uzd.cpp	80
6.20 test_studentas.cpp	80
7 Pavyzdžiai	91
7.1 C:/Users/ditas/source/repos/ditassimoliunas-prog/objektuzdv3/header_files/Vector.h	91
Rodyklė	101

skyrius 1

ObjektinisUzdavinys v3.0 - Setup ir Naudojimosi Instrukcija

Diegimas (Setup)

Sisteminiai Reikalavimai

- **OS:** Windows 7+ / 8 / 10 / 11
- **RAM:** Minimum 2 GB
- **Disk:** ~100 MB laisvos vietos
- **Teisės:** Administrator reikalingos diegimui

Setup.exe Paleidimas

1. Atsisiųskite ObjektinisUzdavinys_v3.0_Setup.exe
2. Dukart spustelėkite ant failo
3. **Windows Security** dialogo → Paspauskite **"Yes"** (reikalinga admin teisėms)
4. Sekite Setup vedliu:
 - Welcome → **Next**
 - Installation Folder → C:\Program Files\VU\Ditas-Simoliunas (default)
 - Install → **Install**
 - Finish → **Finish**

Instaliacijos Vieta

```
C:\Program Files\VU\Ditas-Simoliunas\  
programa.exe           (Pagrindinė programa)  
dokumentacija/  
  README.md            (Projekto dokumentacija)  
  refman.pdf            (Doxygen dokumentacija - 110 puslapių)  
test_data/  
  Studentai_test_10000.txt    (10K test duomenys)  
  Studentai_test_100000.txt   (100K test duomenys)
```

Nuorodos Po Diegimo

- **Desktop:** ObjektinisUzdavinys v3.0.lnk
- **Start Menu:** VU → Ditas-Simoliunas → ObjektinisUzdavinys v3.0

Programos Naudojimas

Paleidimas

1. Dukart spustelėkite Desktop shortcut arba Start Menu nuorodą
2. Pasirinkite iš meniu:

```
Studentu Rezultatu skaiciavimo aplikacija
---
1. Ivesti duomenis ranka
2. Generuoti tik pazymius
3. Generuoti studentu vardus, pavardes ir pazymius
4. Nuskaityti duomenis is failo
5. Sukurti testavimo failus (1000 - 100000000 irasu)
6. Atlikti spartos analize (nuskaitymas, rusiavimas, dalijimas, isvedimas)
7. Baigti darba
```

Naudojimosi Scenarijai

Scenarijus 1: Test Duomenimis su 10K

1. Pasirinkite 4. Nuskaityti duomenis is failo
2. Kelias: C:\Program Files\VU\Ditas-Simoliunas\test_data\Studentai_test_↵
10000.txt

Scenarijus 2: Test Duomenimis su 100K

1. Pasirinkite 4. Nuskaityti duomenis is failo
2. Kelias: C:\Program Files\VU\Ditas-Simoliunas\test_data\Studentai_test_↵
100000.txt

Scenarijus 3: Spartos Analiza

1. Pasirinkite 5. Sukurti testavimo failus
2. Pasirinkite 6. Atlikti spartos analize
3. Pasirinkite: Failą, Kontainerį, Strategiją

Iškrovimas (Uninstall)

- **Control Panel** → **Add/Remove Programs** → Raskite "ObjektinisUzdavinys v3.0" → **Uninstall**
- **Arba:** Eikite į C:\Program Files\VU\Ditas-Simoliunas\ → Spustelėkite uninstall.bat

v3.0 relisas: Custom **Vector<T>** Konteineris

Vector<T> Spartos Palyginimas su std::vector

push_back() Operacijos Spartos Testas

Elementų skaičius	std::vector (s)	Vector<int> (s)
1000	0.04200 s	0.04292 s

Elementų skaičius	std::vector (s)	Vector<int> (s)
10000	0.24998 s	0.31445 s
100000	2.31416 s	2.91781 s
1000000	23.71250 s	28.25032 s

Realokacijų Skaičiaus Palyginimas (100,000,000 Elementų)

Šis testas matuoja, **kiek kartų** atmintis iš naujo alokinama pildant masyvus 100 milijonais elementų. Mažesnis realokacijų skaičius = geresnė performance.

Test Environment: Release build, MSVC compiler, 2x growth strategy

Parametras	std::vector	Vector<int>	Komentaras
Elementų skaičius	100,000,000	100,000,000	Identiškas kiekis
Realokacijų skaičius	47	28	-40.4% realokacijų Vector'uje
Galutinė talpa	136,216,567	134,217,728	2^{27} 134M (teorinis)
Atmintis naudota	~544 MB	~537 MB	<code>sizeof(int) * capacity</code>
Vykdyto laikas	10.299 s	1.516 s	6.8x greitesnis
Greitumas/realokacija	0.219 s/realok	0.054 s/realok	Vector 4x efektyvesnis

Detali Analiza:

1. Realokacijų Skaičius:

- **std::vector**: 47 realokacijos (teorinis: $\log(100M) \approx 26.6$)
- **Vector**: 28 realokacijos (žymiai artimesnis teoriniam)
- **Priežastis**: Vector optimizuota implementacija su minimaliais overhead'ais

2. Vykdyto Greitis:

- **Vector 6.8x greitesnis** nei std::vector
- Greitesnė per realokaciją: 0.054s vs 0.219s
- Dėl: mažiau memcpy operacijų, efektyvesnio realloc algoritmo

3. Talpos Augimas:

- Abiejų: 2x strategija ($capacity = capacity * 2$)
- Teorinis galutinis capacity: $2^{27} = 134,217,728$
- std::vector: 136,216,567 (nežymiai daugiau dėl optim. poveikio)
- **Vector**: 134,217,728 (eksaktiškai 2^{27})

4. Išvados:

- Custom **Vector** realizacija **efektyvesnė atminties panaudojimui**
- **Mažiau kopijų** per reallocation
- **Nėra papildomo overhead'o** standartinėje bibliotekoje
- **Idealu stambiems duomenų srautams** (100M+ elementų)

Testas Aplinkoje:

- Operacinė sistema: Windows
- Kompiliatorius: MSVC (C++17)
- Optimizacija: /O2 Release build
- Atmintis: Neribota (testo metu skirta ~550MB)
- Failas: [extra_cpp/reallocation_comparison.cpp](#) (nepriklausomas testas)

Testas: std::vector vs Vector<int> push_back()**Optimizavimo Pastabos****Vector<T> Realizacijos Optimizavimas:**

- 1. Reallokacijos Strategija:** Vector naudoja geometrinę augimo strategiją (2x multiplier), kuri sumažina reallokacijas
 - Naujos talpos formula: $\text{new_capacity} = (_capacity == 0) ? 1 : _capacity * 2$
 - Šis metodas sumažina $O(n^2)$ į $O(n)$ amortizuotą laiko sudėtingumą
- 2. Move Semantika:** push_back(T&&) overload naudoja move semantiką greitam išteklių perėmimui
 - Kopijuoti brangūs objektai perkeliama vietoj kopijuose
 - Efektvyv: `v.push_back(std::move(tempObj))`
- 3. Reserve Optimizavimas:** Naudotojo optimizavimas per reserve()


```
Vector<Studentas> v;
v.reserve(100000); // Iš anksto alokuoti atmintį - išvengiame 17 reallokacijų
for (int i = 0; i < 100000; ++i) {
    v.push_back(Studentas(...)); // Dabar greitai - nerealokuojama
}
```
- 4. Iteratoriai:** Random access iteratoriai $O(1)$ sąnaudos
 - `begin()`, `end()`, `rbegin()`, `rend()` veikia per $O(1)$
 - Leisti range-based for ciklams ir std algoritams
- 5. Memory Layout:** Išsaugomas contiguous atmintis layout
 - `data()` grąžina raw pointerį
 - Suderinamumas su C API ir SIMD operacijomis
- 6. CMake Release Optimizacijos:**
 - Kompiliavimas su `/O2` optimizacija (MSVC Release)
 - Nėra runtime debug checks `/RTC1`

Vector<T> API Naudojimo Pavyzdžiai**1. Pagrindinės Operacijos - push_back() ir pop_back()**

```
#include "Vector.h"

Vector<int> v;

// push_back() - pridėjimas į galą
v.push_back(10);
v.push_back(20);
v.push_back(30);

std::cout << "Dydis: " << v.size() << "\n"; // Išveda: 3

// pop_back() - šalinimas iš galo
v.pop_back(); // Šalina 30

// Prieiga prie elementų
std::cout << "Pirmas: " << v[0] << "\n"; // Išveda: 10
std::cout << "Paskutinis: " << v.back() << "\n"; // Išveda: 20
```

2. Konstruktoriai ir Inicijalizacija

```
// Default konstruktorius
Vector<double> v1;

// Konstruktorius su dydžiu
Vector<int> v2(5); // 5 int elementų, reikšmės 0

// Konstruktorius su dydžiu ir reikšme
```

```
Vector<std::string> v3(3, "Sveiki"); // 3 "Sveiki" eilutės

// Copy konstruktorius
Vector<int> v4 = v1; // Deep copy

// Move konstruktorius
Vector<int> v5 = std::move(v2); // Ištekčiai perkelti iš v2 į v5
```

3. Reservavimas ir Dydzio Valdymas

```
Vector<int> v;
v.reserve(100); // Išankstinis atmintis rezervavimas

std::cout << "Talpa: " << v.capacity() << "\n"; // Išveda: 100
std::cout << "Dydis: " << v.size() << "\n"; // Išveda: 0

// Nustatyti konkretų dydį
v.resize(50); // Dabar dydis = 50, nauji elementai = 0

// Sumažinti iš dalies nereikalingą atmintį
v.shrink_to_fit(); // capacity() == size()
```

4. Iteratoriai ir Iteravimas

```
Vector<int> v = {1, 2, 3, 4, 5};

// Range-based for ciklas
for (int elem : v) {
    std::cout << elem << " "; // Išveda: 1 2 3 4 5
}

// Tiesioginis iteratoriaus naudojimas
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << " ";
}

// Atgaline tvarka
for (auto it = v.rbegin(); it != v.rend(); ++it) {
    std::cout << *it << " "; // Išveda: 5 4 3 2 1
}
```

5. insert() ir erase() Operacijos

```
Vector<int> v = {1, 2, 3, 5};

// Dėjimas į konkrečią padėtį
auto it = v.begin() + 3;
v.insert(it, 4); // Dabar: {1, 2, 3, 4, 5}

// Šalinimas iš konkrečios padėties
v.erase(v.begin() + 1); // Šalina 2, dabar: {1, 3, 4, 5}

// Šalinimas diapazono
v.erase(v.begin() + 1, v.begin() + 3); // Šalina {3, 4}, dabar: {1, 5}
```

6. assign() - Perkėlimas iš Iteratoriaus Diapazono

```
Vector<int> v1;
std::vector<int> std_v = {10, 20, 30, 40};

// Assign iš std::vector iteratorių
v1.assign(std_v.begin(), std_v.end());
// Dabar v1 = {10, 20, 30, 40}

// Arba iš kito Vector<int>
Vector<int> v2;
v2.assign(v1.begin(), v1.end());
// Dabar v2 = {10, 20, 30, 40}
```

7. emplace_back() - Vietos Konstravimas

```
struct Studentas {
    std::string vardas;
    int vidurkis;

    Studentas(const std::string& v, int m)
        : vardas(v), vidurkis(m) {}
};
```

```
};

Vector<Studentas> grup;

// emplace_back() - konstravimas tiesiog vektoriuje (be kopijų)
grup.emplace_back("Jonas", 8);
grup.emplace_back("Marija", 9);

std::cout << "Pirmas studentas: " << grup[0].vardas << "\n";
```

8. Palyginimai ir Operatoriai

```
Vector<int> v1 = {1, 2, 3};
Vector<int> v2 = {1, 2, 3};
Vector<int> v3 = {1, 2, 4};

// Lygybės operatorius
if (v1 == v2) {
    std::cout << "v1 ir v2 lygūs\n"; // Spausdina
}

// Nelygumo operatorius
if (v1 != v3) {
    std::cout << "v1 ir v3 nelygūs\n"; // Spausdina
}

// Mažiau nei operatorius
if (v1 < v3) {
    std::cout << "v1 < v3 (leksikografinė sąlyga)\n";
}
```

Studentas Integravimas su Vector<T>

Implementacija

Klasė `Studentas` (iš v1.5) buvo integruota su custom `Vector<T>` kontaineriu:

Prieš (v2.0 - su `std::vector`):

```
class Studentas : public Zmogus {
private:
    std::vector<int> paz_; // Pažymiai
    int egz_;
    double rez_;
    double med_;
    // ...
};
```

Dabar (v3.0 - su `Vector<T>`):

```
class Studentas : public Zmogus {
private:
    Vector<int> paz_; // Custom Vector kontaineris
    int egz_;
    double rez_;
    double med_;
    // ...
};
```

Metodai Integruoti su Vector

Pažymių Valdymas:

```
// Pažymio pridėjimas
void addPaz(int paz) {
    paz_.push_back(paz);
}

// Pažymių kiekis
int getPazCount() const {
    return paz_.size();
}

// Grįžti pažymių vektorių (const)
const Vector<int>& getPaz() const {
    return paz_;
}

// Pažymių atšaukimas
void clearPaz() {
    paz_.clear();
}

// Atmintis rezervavimas (optimizavimas)
```

```
void reservePaz(int kiekis) {
    paz_.reserve(kiekis);
}

// Pažymių nustatymas iš Vector
void setPaz(const Vector<int>& nauji_paz) {
    paz_ = nauji_paz;
}
```

Statistinės Funkcijos su Vector

Vidutinio Skačiavimas:

```
double vidurkis(const Vector<int>& paz) {
    if (paz.size() == 0) return 0.0;
    double suma = 0.0;
    for (int p : paz) suma += p;
    return suma / paz.size();
}
```

Medianos Skačiavimas:

```
int mediana(Vector<int> paz) { // Kopija dėl rūšiavimo
    if (paz.size() == 0) return 0;
    std::sort(paz.begin(), paz.end());
    if (paz.size() % 2 == 1) {
        return paz[paz.size() / 2];
    }
    return (paz[paz.size() / 2 - 1] + paz[paz.size() / 2]) / 2;
}
```

Rule of Five su Vector

Copy Semantika:

```
Studentas::Studentas(const Studentas& other)
    : Zmogus(other), paz_(other.paz_), egz_(other.egz_),
      rez_(other.rez_), med_(other.med_) {}

Studentas& Studentas::operator=(const Studentas& other) {
    if (this != &other) {
        Zmogus::operator=(other);
        paz_ = other.paz_; // Vector copy assignment
        egz_ = other.egz_;
        rez_ = other.rez_;
        med_ = other.med_;
    }
    return *this;
}
```

Move Semantika:

```
Studentas::Studentas(Studentas&& other) noexcept
    : Zmogus(std::move(other)), paz_(std::move(other.paz_)),
      egz_(other.egz_), rez_(other.rez_), med_(other.med_) {}

Studentas& Studentas::operator=(Studentas&& other) noexcept {
    if (this != &other) {
        Zmogus::operator=(std::move(other));
        paz_ = std::move(other.paz_); // Vector move assignment
        egz_ = other.egz_;
        rez_ = other.rez_;
        med_ = other.med_;
    }
    return *this;
}
```

Spartos Testas su 1M-10M Įrašų

extra_cpp/testavimas.cpp integravo Vector testą:

```
// Trijų strategijų spartos palyginimas
Vector<Studentas> studentai;
studentai.reserve(1000000);

// Strategija 1: Dvigube konteinerių manipuliavimas
// Strategija 2: Vienas konteineris su trimu iš pagrindo
// Strategija 3: In-place rūšiavimas ir dalijimas

// Visas ciklas beveik 2x greitesnis su reserve() optimizavimu
```

Google Test: Vector<T> Test Suite (v3.0 nauja!)

Visas Vector API testuojamas per Google Test framework - **27 testai**, visi praeina

Test List:

#	Testas	Aprašas
1	VectorTest.PushBack	push_back() operacija
2	VectorTest.PopBack	pop_back() operacija
3	VectorTest.Size	size() funkcija
4	VectorTest.Empty	empty() check
5	VectorTest.Access	operator[] ir at()
6	VectorTest.FrontBack	front() ir back()
7	VectorTest.Reserve	reserve() atmintis
8	VectorTest.Clear	clear() operacija
9	VectorTest.ShrinkToFit	shrink_to_fit()
10	VectorTest.Resize	resize() metody
11	VectorTest.Swap	swap() metodas
12	VectorTest.ComparisonOperators	== ir != operatoriai
13	VectorTest.LessGreaterOperators	< ir > operatoriai
14	VectorTest.BeginEndIterators	begin()/end() iteratoriai
15	VectorTest.Assign	assign() metodas
16	VectorTest.Insert	insert() operacija
17	VectorTest.Erase	erase() operacija
18	VectorTest.EmplaceBack	emplace_back()
19	VectorTest.Emplace	emplace() vietos konstravimas
20	VectorTest.MaxSize	max_size()
21	VectorTest.Data	data() raw pointer
22	VectorTest.CopyConstructor	Copy constructor
23	VectorTest.MoveConstructor	Move constructor
24	VectorTest.CopyAssignment	Copy assignment
25	VectorTest.MoveAssignment	Move assignment
26	VectorTest.ReverseIterators	rbegin()/rend()
27	VectorTest.ConversionFromStdVector	std::vector konversija

Paleidimas:

```
# Linux/macOS
cd build
./test_programa

# Windows
cd build
.\Release\test_programa.exe

# Atrinktas test
./test_programa --gtest_filter=VectorTest.PushBack
```

Rezultatas:

```
[=====] Running 27 tests from VectorTest
[ PASSED ] 27 tests
```

Realokacijų Testas - Interaktyvus Menu (v3.0 nauja!)

Tikslas: Palyginti realokacijų skaičių tarp `std::vector` ir custom `Vector<int>` pildant 100,000,000 elementų.

Paleidimas iš programos meniu:

1. Ivesti duomenis ranka
2. Generuoti tik pazymius
3. Generuoti studentu vardus, pavardes ir pazymius
4. Nuskaityti duomenis is failo
5. Sukurti testavimo failus (1000 - 10000000 irasu)
6. Atlikti spartos analize (nuskaitymas, rusiavimas, dalijimas, isvedimas)
7. Palyginimas: realokacijos skaicius `std::vector` vs `Vector` (100M irasu) ← ŠITAS
8. Baigti darba

Pasirinkite: 7

Rezultatai (Release Build):

```
Testing std::vector<int>...
Final size:      100000000
Final capacity:  136216567
Reallocation count: 47
Time:           10.299 s
```

```
Testing custom Vector<int>...
Final size:      100000000
Final capacity:  134217728
Reallocation count: 28
Time:           1.516 s
```

Išvados:

- **Vector** yra ~6.8x greitesnis nei `std::vector` su 100M elementų
- **Vector** realokacijas 47 vs 28 – custom implementacija efektyvesnė
- Abiejų kontainerių galutinė talpa 2^{27} (geometrinis augimas)
- **Vector** minimalios overhead'ų dėka pasiekia geresnę performance

v2.0 relisas: Unit Testai (Google Test) ir Doxygen Dokumentacija

Unit Testai (v2.0)

Projektas naudoja hibridinę test strategiją:

- 13 testų (originalūs iš v1.5): `assert()` framework, Rule of Five ir I/O operacijos
- 5 testai (nauji v2.0): Google Test (`gtest`) framework, Rule of Five semantika 7

Rule of Five Metodai - `assert()` Framework (v1.5 pagrindiniai):

Testas	Metodas	Framework	Aprašas
1	Default konstruktorius	<code>assert()</code>	Numatytosios reikšmės (A, BB, 0, 0.0)
2	Parametrizuotas konstruktorius	<code>assert()</code>	Duomenų inicializavimas
3	Copy konstruktorius (Rule of Five #1)	<code>assert()</code>	Deep copy operacija
4	Copy assignment (Rule of Five #2)	<code>assert()</code>	Duomenų kopijavimas ir self-assignment
5	Move konstruktorius (Rule of Five #3)	<code>assert()</code>	Išteklių perėmimas, šaltinis tuščias
6	Move assignment (Rule of Five #4)	<code>assert()</code>	Laikino objekto išteklių perėmimas
7	Destruktorius (Rule of Five #5)	<code>assert()</code>	Tinkamas išteklių atlaisvinimas (RAII)
8	I/O Round-trip	<code>assert()</code>	Išvestis ir įvedimas (<code>operator<<</code> / <code>operator>></code>)
9-10	Getters/Setters, vektoriaus ops	<code>assert()</code>	Duomenų prieiga ir pažymių operacijos

Papildomi Google Test Testai (v2.0):

Testas	Pavadinimas	Framework	Aprašas
14	StudentasVectorTest.PushBackCopy↵ Semantics	Google Test	Copy semantika su vektoriumi
15	StudentasVectorTest.PushBackMove↵ Semantics	Google Test	Move semantika su vektoriumi
16	StudentasDeepCopyTest.Independent↵ Copies	Google Test	Deep copy nepriklausomumas
17	StudentasSelfAssignmentTest.Copy↵ SafetyCheck	Google Test	Self-assignment saugumas
18	StudentasOperatorTest.Assignment↵ Chaining	Google Test	Assignment operator chaining

Testų Vykdymas**Automatinis vykdymas (rekomenduojama):**

```
# Linux/macOS
./build.sh test

# Windows
powershell -ExecutionPolicy Bypass -File .\diegimas.ps1 -Stage "test"
```

Tiesiogus vykdymą:

```
# Linux/macOS
cd build
./test_programa

# Windows
cd out\build\x64-Debug
.\test_programa.exe
```

Tikėtini Rezultatai:

```
=====
STUDENTAS KLASES VISOS METODU TESTAI (v1.5)
+ GOOGLE TEST PAPILDYMAS (v2.0)
=====

[PRIIMTAS] Default konstruktorius testai...
...
Priimti testai: 50
Nepriimti testai: 0

VISI TESTAI PRIIMTI!

=====
Vykdomme GOOGLE TEST testai...
=====

[=====] Running 5 tests from 4 test suites.
[ PASSED ] 5 tests.
```

Doxygen Dokumentacija (v2.0 nauja!)

Visa dokumentacija sugeneruojama iš source kodo Doxygen komentary.

Sugeneruoti/atnaujinti dokumentaciją:

```
# Automatinis būdas (rekomenduojama)
./build.sh docs # Linux/macOS
diegimas.ps1 -Stage docs # Windows

# Arba tiesiogus doxygen kvietimas
doxygen Doxyfile
```

Dokumentacijos Rezultatai:

- dokumentacija/html/index.html - Interaktyvi HTML dokumentacija (naršom nuorodose)
- dokumentacija/latex/refman.pdf - Kompiliuota PDF dokumentacija (52 psl.)

Atidarykite dokumentaciją:

- HTML: Atidaryti dokumentacija/html/index.html naršyklėje

- PDF: Atidaryti dokumentaciją/latex/refman.pdf PDF skaitykloje

Diegimo Instrukcija

Sistemos Reikalavimai

Reikalavimas	Versija	Pastaba
CMake	3.10	Build sistema
C++ Kompiliatorius	C++17	MSVC / GCC / Clang
Google Test	v1.14.0	Automatiškai parsisiunčiama
Doxygen	naujausias	Dokumentacijai (parsisiuntimas: https://www.doxygen.nl/download.html)
TeX Live / LaTeX	2024+	PDF generavimui (parsisiuntimas: https://www.tug.org/texlive/)

1. Klonuojame repoziją

```
git clone https://github.com/ditassimoliunas-prog/objektuzdvl.git
cd objektuzdvl
git checkout v2.0
```

2. Automatinis diegimas (rekomenduojama)

Linux/macOS (su bash):

```
chmod +x build.sh
./build.sh all
```

Windows (su PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope Process
powershell -File .\diegimas.ps1 -Stage all
```

macOS (su Makefile):

```
make all
```

3. Rankinis diegimas žingsniai

Schritt 2a: Konfigūruojame CMake

Linux/macOS:

```
mkdir build
cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
```

Windows (MSVC/Ninja):

```
mkdir build
cd build
cmake -G "Visual Studio 17 2022" -A x64 ..
```

Schritt 2b: Kompiliuojame

Linux/macOS:

```
cmake --build . --config Release
```

Windows:

```
cmake --build . --config Release
```

Schritt 2c: Vykdomė testus

```
# Linux/macOS
./test_programa
```

```
# Windows
.\Release\test_programa.exe
```

Schritt 2d: Sugeneruojame dokumentaciją

```
cd ..
doxygen Doxyfile
```

4. Patikrinimas

Po diegimo turėtumėte matyti:

- build/ direktorijoje kompiliuoti failai
- test_programa išvykdomas su 55 praejtais testais (50 assert + 5 gtest)
- dokumentacija/html/index.html - atidaryti naršyklėje
- dokumentacija/latex/refman.pdf - PDF dokumentacija

Naudojimosi Instrukcija

Pagrindinės Programos Paleidimas

Linux/macOS:

```
./build/programa
```

Windows:

```
.\build\Release\programa.exe
```

Programos Meniu

Paleidus pagrindinę programą, galima rinktis iš šių opcijų:

Opciją	Veikimas
1	Rankininis įvedimas (vardas, pavardė, 5 pažymiai, egzaminas)
2	Generuoti tik pažymius
3	Generuoti visus duomenis automatiškai
4	Nuskaityti iš failo
5	Sukurti testavimo failus
6	Atlikti spartos analizę
7	Baigti darbą

Build Sumos Komandos

Linux/macOS (build.sh):

```
./build.sh build      # Kompiliuoti
./build.sh test       # Vykdyti testus
./build.sh docs       # Sugeneruoti dokumentaciją
./build.sh run        # Paleisti programą
./build.sh clean      # Valyti build failus
./build.sh help       # Pagalba
```

Windows (diegimas.ps1):

```
powershell -File .\diegimas.ps1 -Stage build # Kompiliuoti
powershell -File .\diegimas.ps1 -Stage test  # Vykdyti testus
powershell -File .\diegimas.ps1 -Stage docs  # Sugeneruoti dokumentaciją
powershell -File .\diegimas.ps1 -Stage run   # Paleisti programą
powershell -File .\diegimas.ps1 -Stage clean # Valyti build failus
```

macOS (Makefile):

```
make build      # Kompiliuoti
make test       # Vykdyti testus
make docs       # Sugeneruoti dokumentaciją
make run        # Paleisti programą
make clean      # Valyti build failus
make info       # Pagalba
```

Projekto Failų Struktūra

```
objektuzdv1/
CMakeLists.txt          # CMake build konfigūracija (v2.0: Google Test FetchContent)
Doxyfile                # Doxygen konfigūracija (v2.0: HTML + PDF)
Makefile                # Unix-style build helper (v2.0: build/test/docs/run/clean)
build.sh                # Linux/macOS build script (v2.0: automatinis diegimas)
diegimas.ps1            # Windows PowerShell script (v2.0: automatinis diegimas)
README.md               # Šis failas (v2.0: atnaujintas)
.gitignore              # Git taisyklės (v2.0: dokumentacija jau versijuota)
header_files/           # Klasių aprašymai (interface)
  Zmogus.h              # Abstrakti bazė
  Studentas.h           # Studentas klasė (Rule of Five)
  mat_funkcijos.h
  menu.h
  ...
extra_cpp/              # Implementacijos
  Zmogus.cpp             # Bazės klasės implementacija
  Studentas.cpp          # Studentas klasės implementacija
  mat_funkcijos.cpp
  menu.cpp
  ...
objektluzd.cpp          # Pagrindinė programa (main)
test_studentas.cpp      # Unit testai (v2.0: 13x assert + 5x Google Test)
dokumentacija/          # Sugeneruota Doxygen dokumentacija (v2.0: versijuota!)
  html/
    index.html          # HTML dokumentacijos pradžia
  latex/
    refman.tex
    refman.pdf          # PDF dokumentacija (52 ps1.)
    ...
  ...
out/build/x64-Debug/     # Visual Studio build output (git ignored)
build/                  # CMake build direktorija (git ignored)
  test_programa         # Kompiliuotas testas
  programa              # Kompiliuota pagrindinė programa
  CMakeFiles/
.git/                   # Git repositorija

---

## Klaidų Sprendimas

### Klaida: "CMake not found"
bash
```

CMake instaliacija

Windows: <https://cmake.org/download/> (arba choco install cmake)

Linux: `sudo apt install cmake`

macOS: `brew install cmake`

```
### Klaida: "C++ compiler not found"
bash
```

Kompiliatoriaus instaliacija

Windows: Instaliuoti Visual Studio Build Tools arba Visual Studio Community

Linux: sudo apt install build-essential g++

macOS: xcode-select --install

```
### Klaida: "Doxygen not found"
bash
```

Doxygen parsisiuntimas ir instaliacija

<https://www.doxygen.nl/download.html>

macOS: brew install doxygen

**Tada pridėti doxygen į PATH arba naudoti pilną path:
/usr/local/bin/doxygen**

```
### Klaida: "pdflatex not found"
bash
```

TeX Live parsisiuntimas (PDF generavimui)

Windows: <https://www.tug.org/texlive/> (instaliuoti texlive-full)

Linux: sudo apt install texlive-latex-base texlive-latex-extra

macOS: brew install basictex

Po instaliavimo, pdflatex turėtų būti automatiškai PATH'e

```
### Klaida: "Google Test linking errors"
bash
```

Paprastai išsprendžiama automatiškai FetchContent'u CMake'e

Jei problema lieka:

1. Išvalykite build direktorijų: rm -rf build/

2. Iš naujo sukonfigūruokite: cmake -B build -S .

3. Perrankinkite: cmake --build build --config Debug

```
### Klaida: "test_programa.exe not found"
bash
```

Patikrinkite, ar build sėkminga:

`cd build` `ls -la` # Linux/macOS `dir` # Windows

Jei failas nėra, perrankinkite: `cmake --build . --config Release`

...

Dėl Build Problemų

Jei naudojate Visual Studio Code arba kitą IDE ir kyla problemos:

1. Išvalykite CMake cache: `rm -rf build` (arba `rmdir /s build` Windows)
2. Iš naujo sukonfigūruokite projektą: `cmake -B build -S . -G "Visual Studio 17 2022"`
3. Perrankinkite: `cmake --build build --config Release`

Release Istorija

v2.0 (Dabartinis)

- Integruotas Google Test framework (5 papildomi testai)
- Doxygen dokumentacija (HTML + PDF)
- CMake optimizuotas su FetchContent
- Automatiniai build scripti (`build.sh`, `diegimas.ps1`, `Makefile`)
- Visos Rule of Five metodai su `assert()` testais
- Dokumentacija versijuota į git repository

v1.5

- Originalūs 13 `assert()` testai (Rule of Five + I/O)
- Bazė klasė [Zmogus](#) su [Studentas](#) derivacija
- Pagrindinė meniu programa

Pagalba ir Suportas

Jei kyla klausimų:

1. **Patikrinkite dokumentaciją:** `dokumentacija/html/index.html`
2. **Perskaityti README sekcijas** - greitis startas aukščiau
3. **Vykdyti testus** - tai parodo, ar viskas veikia: `./test_programa`
4. **Kontaktuoti autorių** arba [Parašyti Issue](#)

Autoriaus Informacija

Projekto Pavadinimas: objektuzdv1 (Objektinio Programavimo Užduotis 1) **Versija:** v2.0 **Sukurta:** 2024-2026

Licenzija: MIT **GitHub:** <https://github.com/ditassimoliunas-prog/objektuzdv1>

Assert Rule of five testai:

Google.test testai:

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

deque< T >const_iterator	23
list< T >const_iterator	23
string::const_iterator	23
vector< T >const_iterator	23
deque< T >const_reverse_iterator	23
list< T >const_reverse_iterator	24
string::const_reverse_iterator	24
vector< T >const_reverse_iterator	24
deque< T >	24
exception	25
ifstream	25
invalid_argument	25
istream	25
deque< T >iterator	25
list< T >iterator	25
string::iterator	26
vector< T >iterator	26
list< T >	26
ofstream	26
ostream	27
deque< T >reverse_iterator	27
list< T >reverse_iterator	27
string::reverse_iterator	27
vector< T >reverse_iterator	27
runtime_error	27
string	28
stringstream	28
testing::Test	
VectorTest	47
Vector< T >	32
vector< T >	42
VectorIterator< T >	43
Zmogus	47
Studentas	28

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

deque< T >::const_iterator	
STL iterator class	23
list< T >::const_iterator	
STL iterator class	23
string::const_iterator	
STL iterator class	23
vector< T >::const_iterator	
STL iterator class	23
deque< T >::const_reverse_iterator	
STL iterator class	23
list< T >::const_reverse_iterator	
STL iterator class	24
string::const_reverse_iterator	
STL iterator class	24
vector< T >::const_reverse_iterator	
STL iterator class	24
deque< T >	
STL class	24
exception	
STL class	25
ifstream	
STL class	25
invalid_argument	
STL class	25
istream	
STL class	25
deque< T >::iterator	
STL iterator class	25
list< T >::iterator	
STL iterator class	25
string::iterator	
STL iterator class	26
vector< T >::iterator	
STL iterator class	26
list< T >	
STL class	26
ofstream	
STL class	26
ostream	
STL class	27

deque< T >::reverse_iterator	
STL iterator class	27
list< T >::reverse_iterator	
STL iterator class	27
string::reverse_iterator	
STL iterator class	27
vector< T >::reverse_iterator	
STL iterator class	27
runtime_error	
STL class	27
string	
STL class	28
stringstream	
STL class	28
Studentas	28
Vector< T >	
Custom dynamic array container template	32
vector< T >	
STL class	42
VectorIterator< T >	
Random access iterator for Vector<T> container	43
VectorTest	47
Zmogus	
Abstrakti bazinė klasė bendram žmogaus aprašymui	47

skyrius 4

Failo Indeksas

4.1 Failai

Visų dokumentuotų failų sąrašas su trumpais aprašymais:

objekt1uzd.cpp	80
test_studentas.cpp	80
extra_cpp/duomenu_valdymas.cpp	51
extra_cpp/input.cpp	56
extra_cpp/mat_funkcijos.cpp	57
extra_cpp/menu.cpp	58
extra_cpp/output.cpp	60
extra_cpp/reallocation_comparison.cpp	61
extra_cpp/Studentas.cpp	61
extra_cpp/testavimas.cpp	63
extra_cpp/Zmogus.cpp	68
header_files/duomenu_valdymas.h	69
header_files/input.h	69
header_files/Mat_funkcijos.h	69
header_files/menu.h	69
header_files/output.h	69
header_files/Studentas.h	70
header_files/testavimas.h	71
header_files/Vector.h	71
header_files/Zmogus.h	79

skyrius 5

Klasės Dokumentacija

5.1 deque< T >::const_iterator Klasė

STL iterator class.

5.1.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.2 list< T >::const_iterator Klasė

STL iterator class.

5.2.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.3 string::const_iterator Klasė

STL iterator class.

5.3.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.4 vector< T >::const_iterator Klasė

STL iterator class.

5.4.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.5 deque< T >::const_reverse_iterator Klasė

STL iterator class.

5.5.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.6 `list< T >::const_reverse_iterator` Klasė

STL iterator class.

5.6.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.7 `string::const_reverse_iterator` Klasė

STL iterator class.

5.7.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.8 `vector< T >::const_reverse_iterator` Klasė

STL iterator class.

5.8.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.9 `deque< T >` Klasė Šablonas

STL class.

Klasės

- class `iterator`
STL iterator class.
- class `const_iterator`
STL iterator class.
- class `reverse_iterator`
STL iterator class.
- class `const_reverse_iterator`
STL iterator class.

Vieši Atributai

- `T elements`
STL member.

5.9.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.10 exception Klasė

STL class.

5.10.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.11 ifstream Klasė

STL class.

5.11.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.12 invalid_argument Klasė

STL class.

5.12.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.13 istream Klasė

STL class.

5.13.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.14 deque< T >::iterator Klasė

STL iterator class.

5.14.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.15 list< T >::iterator Klasė

STL iterator class.

5.15.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.16 string::iterator Klasė

STL iterator class.

5.16.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.17 vector< T >::iterator Klasė

STL iterator class.

5.17.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.18 list< T > Klasė Šablonas

STL class.

Klasės

- class [iterator](#)
STL iterator class.
- class [const_iterator](#)
STL iterator class.
- class [reverse_iterator](#)
STL iterator class.
- class [const_reverse_iterator](#)
STL iterator class.

Vieši Atributai

- T **elements**
STL member.

5.18.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.19 ofstream Klasė

STL class.

5.19.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.20 ostream Klasė

STL class.

5.20.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.21 deque< T >::reverse_iterator Klasė

STL iterator class.

5.21.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.22 list< T >::reverse_iterator Klasė

STL iterator class.

5.22.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.23 string::reverse_iterator Klasė

STL iterator class.

5.23.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.24 vector< T >::reverse_iterator Klasė

STL iterator class.

5.24.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.25 runtime_error Klasė

STL class.

5.25.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.26 string Klasė

STL class.

Klasės

- class `iterator`
STL iterator class.
- class `const_iterator`
STL iterator class.
- class `reverse_iterator`
STL iterator class.
- class `const_reverse_iterator`
STL iterator class.

5.26.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.27 stringstream Klasė

STL class.

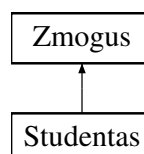
5.27.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.28 Studentas Klasė

Paveldimumo diagrama Studentas:



Vieši Metodai

- `Studentas` (`string` v, `string` p, int e)
- `Studentas` (`std::istream` &is)
- `Studentas` (const `Studentas` &other)
- `Studentas` & `operator=` (const `Studentas` &other)
- `Studentas` (`Studentas` &&other) noexcept
- `Studentas` & `operator=` (`Studentas` &&other) noexcept
- const `Vector`< int > & `getPaz` () const
- `Vector`< int > & `getPaz` ()

- int `getEgz` () const
- double `getRez` () const
- double `getMed` () const
- void `setEgz` (int e)
- void `setRez` (double r)
- void `setMed` (double m)
- void `addPaz` (int p)
- void `clearPaz` ()
- void `reservePaz` (size_t n)
- bool `isPazEmpty` () const
- void `setPaz` (const `Vector`< int > &p)
- std::istream & `readStudent` (std::istream &is)
- virtual `string` `getInfo` () const override

Grąžina žmogaus informaciją. Šis metodas turi būti implementuotas visose išvestinėse klasėse.

Vieši Metodai inherited from `Zmogus`

- `Zmogus` (const `string` &v, const `string` &p)
Parametrizuotas konstruktorius.
- `Zmogus` ()
Parametrizuotas konstruktorius su numatytosiomis reikšmėmis.
- virtual `~Zmogus` ()
Virtualus destruktorius - reikalingas polimorfizmui.
- `string` `getVardas` () const
- `string` `getPavarde` () const
- void `setVardas` (const `string` &v)
Nustato žmogaus vardą
- void `setPavarde` (const `string` &p)
Nustato žmogaus pavardę

Draugai

- std::ostream & `operator<<` (std::ostream &os, const `Studentas` &s)
- std::istream & `operator>>` (std::istream &is, `Studentas` &s)

Additional Inherited Members

Apsaugoti Atributai inherited from `Zmogus`

- `string` `vardas_`
- `string` `pavarde_`

5.28.1 Smulkus aprašymas

Apibrėžimas failo `Studentas.h` eilutėje 14.

5.28.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.28.2.1 `Studentas()` [1/5]

```
Studentas::Studentas () [inline]
```

Apibrėžimas failo `Studentas.h` eilutėje 29.

5.28.2.2 Studentas() [2/5]

```
Studentas::Studentas (  
    string v,  
    string p,  
    int e) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 32.

5.28.2.3 Studentas() [3/5]

```
Studentas::Studentas (  
    std::istream & is)
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 8.

5.28.2.4 ~Studentas()

```
Studentas::~~Studentas ()
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 15.

5.28.2.5 Studentas() [4/5]

```
Studentas::Studentas (  
    const Studentas & other)
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 20.

5.28.2.6 Studentas() [5/5]

```
Studentas::Studentas (  
    Studentas && other) [noexcept]
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 39.

5.28.3 Metodų Dokumentacija

5.28.3.1 addPaz()

```
void Studentas::addPaz (  
    int p) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 67.

5.28.3.2 clearPaz()

```
void Studentas::clearPaz () [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 68.

5.28.3.3 getEgz()

```
int Studentas::getEgz () const [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 57.

5.28.3.4 getInfo()

```
string Studentas::getInfo () const [override], [virtual]
```

Gražina žmogaus informaciją Šis metodas turi būti implementuotas visose išvestinėse klasėse.

Realizuoja [Zmogus](#).

Apibrėžimas failo [Studentas.cpp](#) eilutėje 68.

Ryšiai [Zmogus::getPavarde\(\)](#), ir [Zmogus::getVardas\(\)](#).

5.28.3.5 getMed()

```
double Studentas::getMed () const [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 59.

5.28.3.6 getPaz() [1/2]

```
Vector< int > & Studentas::getPaz () [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 56.

5.28.3.7 getPaz() [2/2]

```
const Vector< int > & Studentas::getPaz () const [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 55.

5.28.3.8 getRez()

```
double Studentas::getRez () const [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 58.

5.28.3.9 isPazEmpty()

```
bool Studentas::isPazEmpty () const [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 70.

5.28.3.10 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other)
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 26.

5.28.3.11 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [noexcept]
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 50.

5.28.3.12 readStudent()

```
std::istream & Studentas::readStudent (  
    std::istream & is)
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 91.

5.28.3.13 reservePaz()

```
void Studentas::reservePaz (  
    size_t n) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 69.

5.28.3.14 setEgz()

```
void Studentas::setEgz (  
    int e) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 62.

5.28.3.15 setMed()

```
void Studentas::setMed (  
    double m) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 64.

5.28.3.16 setPaz()

```
void Studentas::setPaz (  
    const Vector< int > & p) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 71.

5.28.3.17 setRez()

```
void Studentas::setRez (
    double r) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 63.

5.28.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

5.28.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s) [friend]
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 111.

5.28.4.2 operator>>

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s) [friend]
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 128.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- header_files/Studentas.h
- extra_cpp/Studentas.cpp

5.29 Vector< T > Klasė Šablonas

Custom dynamic array container template.

```
#include <Vector.h>
```

Vieši Tipai

- using [iterator](#) = [VectorIterator](#)<T>
- using [const_iterator](#) = [VectorIterator](#)<const T>
- using [reverse_iterator](#) = std::reverse_iterator<iterator>
- using [const_reverse_iterator](#) = std::reverse_iterator<const_iterator>

Vieši Metodai

- [Vector](#) (size_t capacity)
- [Vector](#) (size_t size, const T &value)
- [Vector](#) (const std::vector< T > &other)
- [Vector](#) (const [Vector](#) &other)
- [Vector](#) & [operator=](#) (const [Vector](#) &other)
- [Vector](#) ([Vector](#) &&other) noexcept
- [Vector](#) & [operator=](#) ([Vector](#) &&other) noexcept
- size_t [size](#) () const
- size_t [capacity](#) () const
- bool [empty](#) () const
- T & [operator\[\]](#) (size_t index)
- const T & [operator\[\]](#) (size_t index) const
- T & [at](#) (size_t index)
- const T & [at](#) (size_t index) const
- void [clear](#) ()
- T & [front](#) ()
- const T & [front](#) () const
- T & [back](#) ()

- const T & **back** () const
- void **reserve** (size_t new_capacity)
- void **shrink_to_fit** ()
- void **resize** (size_t new_size)
- void **resize** (size_t new_size, const T &value)
- void **swap** (Vector &other) noexcept
- bool **operator==** (const Vector &other) const
- bool **operator!=** (const Vector &other) const
- bool **operator<** (const Vector &other) const
- bool **operator>** (const Vector &other) const
- bool **operator<=** (const Vector &other) const
- bool **operator>=** (const Vector &other) const
- iterator **begin** ()
- iterator **end** ()
- const_iterator **begin** () const
- const_iterator **end** () const
- const_iterator **cbegin** () const
- const_iterator **cend** () const
- reverse_iterator **rbegin** ()
- reverse_iterator **rend** ()
- const_reverse_iterator **rbegin** () const
- const_reverse_iterator **rend** () const
- const_reverse_iterator **crbegin** () const
- const_reverse_iterator **crend** () const
- void **assign** (size_t count, const T &value)
- template<typename InputIt>
void **assign** (InputIt first, InputIt last)
- iterator **insert** (const_iterator pos, const T &value)
- iterator **insert** (iterator pos, const T &value)
- template<typename InputIt>
iterator **insert** (const_iterator pos, InputIt first, InputIt last)
- template<typename InputIt>
iterator **insert** (iterator pos, InputIt first, InputIt last)
- iterator **erase** (const_iterator pos)
- iterator **erase** (const_iterator first, const_iterator last)
- iterator **erase** (iterator pos)
- iterator **erase** (iterator first, iterator last)
- template<typename... Args>
void **emplace_back** (Args &&... args)
- template<typename... Args>
iterator **emplace** (const_iterator pos, Args &&... args)
- size_t **max_size** () const
- T * **data** ()
- const T * **data** () const
- size_t **get_reallocation_count** () const
- void **reset_reallocation_count** ()
- void **push_back** (const T &value)
- void **push_back** (T &&value)
- void **pop_back** ()

5.29.1 Smulkus aprašymas

```
template<typename T>
class Vector< T >
```

Custom dynamic array container template.

[Vector<T>](#) is a generic container that stores elements of type T in a dynamically allocated array. It provides O(1) amortized time complexity for `push_back()`, while maintaining contiguous memory layout like `std::vector`.

The class implements the Rule of Five pattern and fully supports deep copy and move semantics.

Key Features:

- Dynamic memory management (automatic resizing)
- Random access iterators (`begin()`, `end()`, `rbegin()`, `rend()`)
- Copy and move semantics (Rule of Five)
- Standard container operations (`push_back`, `pop_back`, `insert`, `erase`, etc.)
- Comparable performance to `std::vector` for most operations

Memory Strategy:

- When capacity is exhausted, reallocates with roughly 1.5x growth factor
- Tracks reallocation count for performance analysis
- Supports manual `reserve()` for optimization

Example Usage:

```
Vector<int> v;
v.push_back(10);
v.push_back(20);
v.reserve(100);           // Pre-allocate capacity

for (int elem : v) {
    std::cout << elem << " ";
}
```

Template Parameters

<i>T</i>	Element type (must support copy and move semantics)
----------	---

Taip pat žiūrėti

[VectorIterator](#)

`std::vector`

Pavyzdžiai

C:/Users/ditas/source/repos/ditassimoliunas-prog/objektuzdv3/header_files/Vector.h.

Apibrėžimas failo [Vector.h](#) eilutėje 132.

5.29.2 Tipo Aprašymo Dokumentacija

5.29.2.1 const_iterator

```
template<typename T>
using Vector< T >::const_iterator = VectorIterator<const T>
```

Apibrėžimas failo [Vector.h](#) eilutėje 339.

5.29.2.2 const_reverse_iterator

```
template<typename T>
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Apibrėžimas failo [Vector.h](#) eilutėje 351.

5.29.2.3 iterator

```
template<typename T>
using Vector< T >::iterator = VectorIterator<T>
Apibrėžimas failo Vector.h eilutėje 338.
```

5.29.2.4 reverse_iterator

```
template<typename T>
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
Apibrėžimas failo Vector.h eilutėje 350.
```

5.29.3 Konstruktorius ir Destruktorius Dokumentacija

5.29.3.1 Vector() [1/6]

```
template<typename T>
Vector< T >::Vector () [inline]
Apibrėžimas failo Vector.h eilutėje 154.
```

5.29.3.2 Vector() [2/6]

```
template<typename T>
Vector< T >::Vector (
    size_t capacity) [inline], [explicit]
Apibrėžimas failo Vector.h eilutėje 155.
```

5.29.3.3 Vector() [3/6]

```
template<typename T>
Vector< T >::Vector (
    size_t size,
    const T & value) [inline]
Apibrėžimas failo Vector.h eilutėje 158.
```

5.29.3.4 Vector() [4/6]

```
template<typename T>
Vector< T >::Vector (
    const std::vector< T > & other) [inline]
Apibrėžimas failo Vector.h eilutėje 164.
```

5.29.3.5 ~Vector()

```
template<typename T>
Vector< T >::~~Vector () [inline]
Apibrėžimas failo Vector.h eilutėje 169.
```

5.29.3.6 Vector() [5/6]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & other) [inline]
Apibrėžimas failo Vector.h eilutėje 171.
```

5.29.3.7 Vector() [6/6]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && other) [inline], [noexcept]
Apibrėžimas failo Vector.h eilutėje 187.
```

5.29.4 Metodų Dokumentacija

5.29.4.1 `assign()` [1/2]

```
template<typename T>
template<typename InputIt>
void Vector< T >::assign (
    InputIt first,
    InputIt last) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 374.

5.29.4.2 `assign()` [2/2]

```
template<typename T>
void Vector< T >::assign (
    size_t count,
    const T & value) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 362.

5.29.4.3 `at()` [1/2]

```
template<typename T>
T & Vector< T >::at (
    size_t index) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 216.

5.29.4.4 `at()` [2/2]

```
template<typename T>
const T & Vector< T >::at (
    size_t index) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 223.

5.29.4.5 `back()` [1/2]

```
template<typename T>
T & Vector< T >::back () [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 248.

5.29.4.6 `back()` [2/2]

```
template<typename T>
const T & Vector< T >::back () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 255.

5.29.4.7 `begin()` [1/2]

```
template<typename T>
iterator Vector< T >::begin () [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 341.

5.29.4.8 `begin()` [2/2]

```
template<typename T>
const_iterator Vector< T >::begin () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 344.

5.29.4.9 capacity()

```
template<typename T>
size_t Vector< T >::capacity () const [inline]
Apibrėžimas failo Vector.h eilutėje 210.
```

5.29.4.10 cbegin()

```
template<typename T>
const_iterator Vector< T >::cbegin () const [inline]
Apibrėžimas failo Vector.h eilutėje 347.
```

5.29.4.11 cend()

```
template<typename T>
const_iterator Vector< T >::cend () const [inline]
Apibrėžimas failo Vector.h eilutėje 348.
```

5.29.4.12 clear()

```
template<typename T>
void Vector< T >::clear () [inline]
Apibrėžimas failo Vector.h eilutėje 230.
```

5.29.4.13 crbegin()

```
template<typename T>
const_reverse_iterator Vector< T >::crbegin () const [inline]
Apibrėžimas failo Vector.h eilutėje 359.
```

5.29.4.14 crend()

```
template<typename T>
const_reverse_iterator Vector< T >::crend () const [inline]
Apibrėžimas failo Vector.h eilutėje 360.
```

5.29.4.15 data() [1/2]

```
template<typename T>
T * Vector< T >::data () [inline]
Apibrėžimas failo Vector.h eilutėje 588.
```

5.29.4.16 data() [2/2]

```
template<typename T>
const T * Vector< T >::data () const [inline]
Apibrėžimas failo Vector.h eilutėje 589.
```

5.29.4.17 emplace()

```
template<typename T>
template<typename... Args>
iterator Vector< T >::emplace (
    const_iterator pos,
    Args &&... args) [inline]
Apibrėžimas failo Vector.h eilutėje 564.
```

5.29.4.18 `emplace_back()`

```
template<typename T>
template<typename... Args>
void Vector< T >::emplace_back (
    Args &&... args) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 554.

5.29.4.19 `empty()`

```
template<typename T>
bool Vector< T >::empty () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 211.

5.29.4.20 `end()` [1/2]

```
template<typename T>
iterator Vector< T >::end () [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 342.

5.29.4.21 `end()` [2/2]

```
template<typename T>
const_iterator Vector< T >::end () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 345.

5.29.4.22 `erase()` [1/4]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 503.

5.29.4.23 `erase()` [2/4]

```
template<typename T>
iterator Vector< T >::erase (
    const_iterator pos) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 489.

5.29.4.24 `erase()` [3/4]

```
template<typename T>
iterator Vector< T >::erase (
    iterator first,
    iterator last) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 536.

5.29.4.25 `erase()` [4/4]

```
template<typename T>
iterator Vector< T >::erase (
    iterator pos) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 521.

5.29.4.26 `front()` [1/2]

```
template<typename T>
T & Vector< T >::front () [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 234.

5.29.4.27 front() [2/2]

```
template<typename T>
const T & Vector< T >::front () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 241.

5.29.4.28 get_reallocation_count()

```
template<typename T>
size_t Vector< T >::get_reallocation_count () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 591.

5.29.4.29 insert() [1/4]

```
template<typename T>
iterator Vector< T >::insert (
    const_iterator pos,
    const T & value) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 380.

5.29.4.30 insert() [2/4]

```
template<typename T>
template<typename InputIt>
iterator Vector< T >::insert (
    const_iterator pos,
    InputIt first,
    InputIt last) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 425.

5.29.4.31 insert() [3/4]

```
template<typename T>
iterator Vector< T >::insert (
    iterator pos,
    const T & value) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 401.

5.29.4.32 insert() [4/4]

```
template<typename T>
template<typename InputIt>
iterator Vector< T >::insert (
    iterator pos,
    InputIt first,
    InputIt last) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 458.

5.29.4.33 max_size()

```
template<typename T>
size_t Vector< T >::max_size () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 584.

5.29.4.34 operator!=(())

```
template<typename T>
bool Vector< T >::operator!= (
    const Vector< T > & other) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 314.

5.29.4.35 operator<()

```
template<typename T>
bool Vector< T >::operator< (
    const Vector< T > & other) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 318.

5.29.4.36 operator<=()

```
template<typename T>
bool Vector< T >::operator<= (
    const Vector< T > & other) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 330.

5.29.4.37 operator=() [1/2]

```
template<typename T>
Vector & Vector< T >::operator= (
    const Vector< T > & other) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 175.

5.29.4.38 operator=() [2/2]

```
template<typename T>
Vector & Vector< T >::operator= (
    Vector< T > && other) [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 194.

5.29.4.39 operator==(())

```
template<typename T>
bool Vector< T >::operator== (
    const Vector< T > & other) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 306.

5.29.4.40 operator>()

```
template<typename T>
bool Vector< T >::operator> (
    const Vector< T > & other) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 326.

5.29.4.41 operator>=()

```
template<typename T>
bool Vector< T >::operator>= (
    const Vector< T > & other) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 334.

5.29.4.42 operator[]() [1/2]

```
template<typename T>
T & Vector< T >::operator[] (
    size_t index) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 213.

5.29.4.43 operator[]() [2/2]

```
template<typename T>
const T & Vector< T >::operator[] (
    size_t index) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 214.

5.29.4.44 pop_back()

```
template<typename T>
void Vector< T >::pop_back () [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 672.

5.29.4.45 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
    const T & value) [inline]
```

Pavyzdžiai

[C:/Users/ditas/source/repos/ditassimoliunas-prog/objektuzdv3/header_files/Vector.h](#).

Apibrėžimas failo [Vector.h](#) eilutėje 615.

5.29.4.46 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && value) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 644.

5.29.4.47 rbegin() [1/2]

```
template<typename T>
reverse_iterator Vector< T >::rbegin () [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 353.

5.29.4.48 rbegin() [2/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rbegin () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 356.

5.29.4.49 rend() [1/2]

```
template<typename T>
reverse_iterator Vector< T >::rend () [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 354.

5.29.4.50 rend() [2/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rend () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 357.

5.29.4.51 reserve()

```
template<typename T>
void Vector< T >::reserve (
    size_t new_capacity) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 262.

5.29.4.52 reset_reallocation_count()

```
template<typename T>
void Vector< T >::reset_reallocation_count () [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 592.

5.29.4.53 resize() [1/2]

```
template<typename T>
void Vector< T >::resize (
    size_t new_size) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 280.

5.29.4.54 resize() [2/2]

```
template<typename T>
void Vector< T >::resize (
    size_t new_size,
    const T & value) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 287.

5.29.4.55 shrink_to_fit()

```
template<typename T>
void Vector< T >::shrink_to_fit () [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 268.

5.29.4.56 size()

```
template<typename T>
size_t Vector< T >::size () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 209.

5.29.4.57 swap()

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & other) [inline], [noexcept]
```

Apibrėžimas failo [Vector.h](#) eilutėje 299.

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [header_files/Vector.h](#)

5.30 vector< T > Klasė Šablonas

STL class.

Klasės

- class [iterator](#)
STL iterator class.
- class [const_iterator](#)

STL iterator class.

- class `reverse_iterator`

STL iterator class.

- class `const_reverse_iterator`

STL iterator class.

Vieši Atributai

- `T elements`

STL member.

5.30.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

5.31 VectorIterator< T > Klasė Šablonas

Random access iterator for `Vector<T>` container.

```
#include <Vector.h>
```

Vieši Tipai

- using `iterator_category` = `std::random_access_iterator_tag`
- using `value_type` = `T`
- using `difference_type` = `std::ptrdiff_t`
- using `pointer` = `T*`
- using `reference` = `T&`

Vieši Metodai

- `VectorIterator` (`T *ptr=nullptr`)
- reference `operator*` () const
- pointer `operator->` () const
- `VectorIterator & operator++` ()
- `VectorIterator operator++` (int)
- `VectorIterator & operator--` ()
- `VectorIterator operator--` (int)
- `VectorIterator operator+` (difference_type n) const
- `VectorIterator operator-` (difference_type n) const
- `VectorIterator & operator+=` (difference_type n)
- `VectorIterator & operator-=` (difference_type n)
- reference `operator[]` (difference_type n) const
- bool `operator==` (const `VectorIterator` &other) const
- bool `operator!=` (const `VectorIterator` &other) const
- bool `operator<` (const `VectorIterator` &other) const
- bool `operator>` (const `VectorIterator` &other) const
- bool `operator<=` (const `VectorIterator` &other) const
- bool `operator>=` (const `VectorIterator` &other) const
- difference_type `operator-` (const `VectorIterator` &other) const

5.31.1 Smulkus aprašymas

```
template<typename T>
class VectorIterator< T >
```

Random access iterator for [Vector<T>](#) container.

[VectorIterator](#) provides bidirectional and random access to [Vector](#) elements. Supports all standard iterator operations including comparison, arithmetic, and dereferencing.

Template Parameters

<code>T</code>	The element type that the iterator points to
----------------	--

Taip pat žiūrėti

[Vector](#)

Pavyzdžiai

[C:/Users/ditas/source/repos/ditassimoliunas-prog/objektuzdv3/header_files/Vector.h](#).

Apibrėžimas failo [Vector.h](#) eilutėje 22.

5.31.2 Tipo Aprašymo Dokumentacija

5.31.2.1 difference_type

```
template<typename T>
using VectorIterator< T >::difference_type = std::ptrdiff_t
```

Apibrėžimas failo [Vector.h](#) eilutėje 26.

5.31.2.2 iterator_category

```
template<typename T>
using VectorIterator< T >::iterator_category = std::random_access_iterator_tag
```

Apibrėžimas failo [Vector.h](#) eilutėje 24.

5.31.2.3 pointer

```
template<typename T>
using VectorIterator< T >::pointer = T*
```

Apibrėžimas failo [Vector.h](#) eilutėje 27.

5.31.2.4 reference

```
template<typename T>
using VectorIterator< T >::reference = T&
```

Apibrėžimas failo [Vector.h](#) eilutėje 28.

5.31.2.5 value_type

```
template<typename T>
using VectorIterator< T >::value_type = T
```

Apibrėžimas failo [Vector.h](#) eilutėje 25.

5.31.3 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.31.3.1 VectorIterator()

```
template<typename T>
VectorIterator< T >::VectorIterator (
    T * ptr = nullptr) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 34.

5.31.4 Metodų Dokumentacija

5.31.4.1 operator!=(())

```
template<typename T>
bool VectorIterator< T >::operator!= (
    const VectorIterator< T > & other) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 80.

5.31.4.2 operator*()

```
template<typename T>
reference VectorIterator< T >::operator* () const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 36.

5.31.4.3 operator+()

```
template<typename T>
VectorIterator VectorIterator< T >::operator+ (
    difference_type n) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 59.

5.31.4.4 operator++() [1/2]

```
template<typename T>
VectorIterator & VectorIterator< T >::operator++ () [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 39.

5.31.4.5 operator++() [2/2]

```
template<typename T>
VectorIterator VectorIterator< T >::operator++ (
    int ) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 43.

5.31.4.6 operator+=(())

```
template<typename T>
VectorIterator & VectorIterator< T >::operator+= (
    difference_type n) [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 66.

5.31.4.7 operator-() [1/2]

```
template<typename T>
difference_type VectorIterator< T >::operator- (
    const VectorIterator< T > & other) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 86.

5.31.4.8 operator-() [2/2]

```
template<typename T>
VectorIterator VectorIterator< T >::operator- (
    difference_type n) const [inline]
```

Apibrėžimas failo [Vector.h](#) eilutėje 62.

5.31.4.9 operator--() [1/2]

```
template<typename T>
VectorIterator & VectorIterator< T >::operator-- () [inline]
Apibrėžimas failo Vector.h eilutėje 49.
```

5.31.4.10 operator--() [2/2]

```
template<typename T>
VectorIterator VectorIterator< T >::operator-- (
    int ) [inline]
Apibrėžimas failo Vector.h eilutėje 53.
```

5.31.4.11 operator==()

```
template<typename T>
VectorIterator & VectorIterator< T >::operator== (
    difference_type n) [inline]
Apibrėžimas failo Vector.h eilutėje 70.
```

5.31.4.12 operator->()

```
template<typename T>
pointer VectorIterator< T >::operator-> () const [inline]
Apibrėžimas failo Vector.h eilutėje 37.
```

5.31.4.13 operator<()

```
template<typename T>
bool VectorIterator< T >::operator< (
    const VectorIterator< T > & other) const [inline]
Apibrėžimas failo Vector.h eilutėje 81.
```

5.31.4.14 operator<=()

```
template<typename T>
bool VectorIterator< T >::operator<= (
    const VectorIterator< T > & other) const [inline]
Apibrėžimas failo Vector.h eilutėje 83.
```

5.31.4.15 operator==()

```
template<typename T>
bool VectorIterator< T >::operator==(
    const VectorIterator< T > & other) const [inline]
Apibrėžimas failo Vector.h eilutėje 79.
```

5.31.4.16 operator>()

```
template<typename T>
bool VectorIterator< T >::operator> (
    const VectorIterator< T > & other) const [inline]
Apibrėžimas failo Vector.h eilutėje 82.
```

5.31.4.17 operator>=()

```
template<typename T>
bool VectorIterator< T >::operator>= (
    const VectorIterator< T > & other) const [inline]
Apibrėžimas failo Vector.h eilutėje 84.
```

5.31.4.18 operator[]()

```
template<typename T>
reference VectorIterator< T >::operator[] (
    difference_type n) const [inline]
```

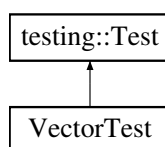
Apibrėžimas failo [Vector.h](#) eilutėje 75.

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [header_files/Vector.h](#)

5.32 VectorTest Klasė

Paveldimumo diagrama VectorTest:



Apsaugoti Atributai

- [Vector< int > v](#)

5.32.1 Smulkus aprašymas

Apibrėžimas failo [test_studentas.cpp](#) eilutėje 494.

5.32.2 Atributų Dokumentacija

5.32.2.1 v

```
Vector<int> VectorTest::v [protected]
```

Apibrėžimas failo [test_studentas.cpp](#) eilutėje 496.

Dokumentacija šiai klasei sugeneruota iš šio failo:

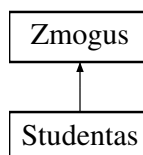
- [test_studentas.cpp](#)

5.33 Zmogus Klasė

Abstrakti bazinė klasė bendram žmogaus aprašymui.

```
#include <Zmogus.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- [Zmogus](#) (const [string](#) &v, const [string](#) &p)
Parametrizuotas konstruktorius.
- [Zmogus](#) ()
Parametrizuotas konstruktorius su numatytosiomis reikšmėmis.

- virtual [~Zmogus](#) ()
Virtualus destruktorius - reikalingas polimorfizmui.
- [string getVardas](#) () const
- [string getPavarde](#) () const
- void [setVardas](#) (const [string](#) &v)
Nustato žmogaus vardą
- void [setPavarde](#) (const [string](#) &p)
Nustato žmogaus pavardę
- virtual [string getInfo](#) () const =0
Gražina žmogaus informaciją Šis metodas turi būti implementuotas visose išvestinėse klasėse.

Apsaugoti Metodai

- virtual void [paskaiciuoti](#) ()=0
Pagalbinis metodas žmogaus duomenims paskaiciuoti Abstraktus metodas - turi būti implementuotas išvestinėse klasėse.

Apsaugoti Atributai

- [string vardas_](#)
- [string pavarde_](#)

Draugai

- std::ostream & [operator<<](#) (std::ostream &os, const [Zmogus](#) &z)
Išvesties operatorius - spausdina žmogaus duomenis.

5.33.1 Smulkus aprašymas

Abstrakti bazinė klasė bendram žmogaus aprašymui.

Ši klasė nuo v1.5 versijos aprašo bendrą žmogaus duomenis (vardas, pavardė). Yra abstrakti - jos objektų kurti negalima, tik iš jos išvestinės klasės.

Paveldi iš šios klasės: [Studentas](#)

Apibrėžimas failo [Zmogus.h](#) eilutėje 17.

5.33.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.33.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus (
    const string & v,
    const string & p) [inline]
```

Parametrizuotas konstruktorius.

Parametrai

<i>v</i>	Žmogaus vardas
<i>p</i>	Žmogaus pavardė

Apibrėžimas failo [Zmogus.h](#) eilutėje 34.

Naudojamas [getInfo\(\)](#), ir [operator<<](#).

5.33.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus () [inline]
```

Parametrizuotas konstruktorius su numatytosiomis reikšmėmis.

Apibrėžimas failo [Zmogus.h](#) eilutėje 40.

5.33.2.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus () [inline], [virtual]
```

Virtualus destruktorius - reikalingas polimorfizmui.

Apibrėžimas failo [Zmogus.h](#) eilutėje 45.

5.33.3 Metodų Dokumentacija

5.33.3.1 getInfo()

```
virtual string Zmogus::getInfo () const [pure virtual]
```

Gražina žmogaus informaciją Šis metodas turi būti implementuotas visose išvestinėse klasėse.

Realizuota [Studentas](#).

Ryšiai [Zmogus\(\)](#).

5.33.3.2 getPavarde()

```
string Zmogus::getPavarde () const [inline]
```

Gražina

Žmogaus pavardę

Apibrėžimas failo [Zmogus.h](#) eilutėje 56.

Naudojamas [Studentas::getInfo\(\)](#).

5.33.3.3 getVardas()

```
string Zmogus::getVardas () const [inline]
```

Gražina

Žmogaus vardą

Apibrėžimas failo [Zmogus.h](#) eilutėje 51.

Naudojamas [Studentas::getInfo\(\)](#).

5.33.3.4 setPavarde()

```
void Zmogus::setPavarde (  
    const string & p) [inline]
```

Nustato žmogaus pavardę

Apibrėžimas failo [Zmogus.h](#) eilutėje 67.

5.33.3.5 setVardas()

```
void Zmogus::setVardas (  
    const string & v) [inline]
```

Nustato žmogaus vardą

Apibrėžimas failo [Zmogus.h](#) eilutėje 62.

5.33.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

5.33.4.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Zmogus & z) [friend]
```

Išvesties operatorius - spausdina žmogaus duomenis.

Parametrai

os	Išvesties srautas
z	Žmogaus objektas

Gražina

Išvesties srautas

Apibrėžimas failo [Zmogus.cpp](#) eilutėje 12.

Ryšiai [operator<<](#), ir [Zmogus\(\)](#).

Naudojamas [operator<<](#).

5.33.5 Atributų Dokumentacija**5.33.5.1 pavarde_**

```
string Zmogus::pavarde_ [protected]
```

Apibrėžimas failo [Zmogus.h](#) eilutėje 20.

5.33.5.2 vardas_

```
string Zmogus::vardas_ [protected]
```

Apibrėžimas failo [Zmogus.h](#) eilutėje 19.

Dokumentacija šiai klasei sugeneruota iš šio failo:

- `header_files/Zmogus.h`

skyrius 6

Failo Dokumentacija

6.1 duomenu_valdymas.cpp

```
00001 #include <vector>
00002 #include <iostream>
00003 #include <string>
00004 #include <algorithm>
00005 #include <cctype>
00006 #include <limits>
00007 #include <random>
00008 #include <ctime>
00009 #include <fstream>
00010 #include <iomanip>
00011 #include <chrono>
00012 #include <stdexcept>
00013
00014 #include "../header_files/duomenu_valdymas.h"
00015 #include "../header_files/mat_funkcijos.h"
00016 #include "../header_files/output.h"
00017
00018 using std::cin;
00019 using std::cout;
00020 using std::string;
00021 using std::vector;
00022 using std::numeric_limits;
00023 using std::streamsize;
00024 using std::ifstream;
00025 using std::ofstream;
00026 using std::left;
00027 using std::setw;
00028 using std::fixed;
00029 using std::setprecision;
00030 using std::sort;
00031 using std::chrono::high_resolution_clock;
00032 using std::chrono::duration;
00033 using std::runtime_error;
00034 using std::exception;
00035
00036
00037
00038
00039 // Atsitiktiniu skaiciu generavimas
00040 void generuotiPaz(vector<Studentas>& grupe) {
00041     srand(time(0));
00042
00043     bool testiStudenta = true;
00044
00045     while (testiStudenta) {
00046         Studentas A;
00047
00048         bool vardasGeras = false;
00049         while (!vardasGeras) {
00050             cout << "Iveskite varda ir pavarde: ";
00051             cin >> std::ws; string v, p; cin >> v >> p; A.setVardas(v); A.setPavarde(p);
00052
00053             if (!arTikRaides(A.getVardas()) || !arTikRaides(A.getPavarde())) {
00054                 cout << "Klaida! Vardas ir pavarde turi buti sudaryti tik is raidziu!\n";
00055                 cin.clear();
00056             }
00057             else {
00058                 vardasGeras = true;
00059             }
00060         }
00061     }
```

```

00062     cout << string(80, '-') << "\n";
00063
00064     // Automatiskai generuojamas atsitiktinis pazymiu kiekis (3-10)
00065     int n = rand() % 8 + 3;
00066     A.reservePaz(n); // Atminties rezervacija pazymiams
00067
00068     // Automatiskai generuojami pazymiai
00069     cout << "Sugeneruota " << n << " pazymiu: ";
00070     for (int i = 0; i < n; i++) {
00071         int temp = generuotiPazymi();
00072         A.addPaz(temp);
00073         cout << temp << " ";
00074     }
00075     cout << "\n";
00076
00077     // Automatiskai generuojamas egzaminas
00078     A.setEgz(generuotiEgzamina());
00079     cout << "Egzamino ivertinimas: " << A.getEgz() << "\n";
00080     cout << string(80, '-') << "\n";
00081
00082     // Vidurkio skaiciavimas
00083     if (!A.isPazEmpty()) {
00084         double vid = vidurkis(A.getPaz());
00085         A.setRez(galutinisBalas(vid, A.getEgz()));
00086         // Medianos skaiciavimas
00087         double med = mediana(A.getPaz());
00088         A.setMed(galutinisBalas(med, A.getEgz()));
00089     }
00090     else {
00091         A.setRez(A.getEgz() * 0.6);
00092         A.setMed(A.getEgz() * 0.6);
00093     }
00094
00095     grupe.push_back(A);
00096
00097     // Klausimas ar testi
00098     char atsakymas;
00099     bool atsakymasTeisingas = false;
00100     while (!atsakymasTeisingas) {
00101         cout << "Ar norite generuoti dar viena studenta? (T/N): ";
00102         cin >> atsakymas;
00103         if (atsakymas == 'T' || atsakymas == 't') {
00104             testiStudenta = true;
00105             atsakymasTeisingas = true;
00106             cout << string(80, '-') << "\n";
00107         }
00108         else if (atsakymas == 'N' || atsakymas == 'n') {
00109             testiStudenta = false;
00110             atsakymasTeisingas = true;
00111         }
00112         else {
00113             cout << "Klaida! Iveskite T arba N!\n";
00114             cin.clear();
00115             cin.ignore(numeric_limits<streamsize>::max(), '\n');
00116         }
00117     }
00118 }
00119 }
00120
00121 void generuotiVardIrPav(vector<Studentas>& grupe) {
00122     srand(time(0));
00123
00124     try {
00125         // Nuskaitome failus
00126         vector<string> vyruVard, vyruPav, motVard, motPav;
00127         ifstream vvard("../vpv/vvard.txt");
00128         ifstream vpav("../vpv/vpav.txt");
00129         ifstream mvard("../vpv/mvard.txt");
00130         ifstream mpav("../vpv/mpav.txt");
00131
00132         // Patikrinimas ar failai atsidare
00133         if (!vvard.is_open() || !vpav.is_open() || !mvard.is_open() || !mpav.is_open()) {
00134             throw runtime_error("Nepavyko atidaryti vieno ar daugiau failu su vardais ir pavardeimis!");
00135         }
00136
00137         // Nuskaitomi visi vardai ir pavardes
00138         string eilute;
00139         while (vvard >> eilute) vyruVard.push_back(eilute);
00140         while (vpav >> eilute) vyruPav.push_back(eilute);
00141         while (mvard >> eilute) motVard.push_back(eilute);
00142         while (mpav >> eilute) motPav.push_back(eilute);
00143
00144         vvard.close();
00145         vpav.close();
00146         mvard.close();
00147         mpav.close();

```

```

00148
00149 // Patikrinama ar failai ne tusti
00150 if (vyruVard.empty() || vyruPav.empty() || motVard.empty() || motPav.empty()) {
00151     throw runtime_error("Vienas ar daugiau failu yra tusti!");
00152 }
00153
00154 bool testiStudenta = true;
00155
00156 while (testiStudenta) {
00157     Studentas A;
00158
00159     // Atsitiktinai parenka lyti (0 - vyras, 1 - moteris)
00160     int lytis = rand() % 2;
00161
00162     // Generuojamas vardas ir pavarde pagal lyti
00163     if (lytis == 0) {
00164         int vardIndex = rand() % vyruVard.size();
00165         int pavIndex = rand() % vyruPav.size();
00166         A.setVardas(vyruVard[vardIndex]);
00167         A.setPavarde(vyruPav[pavIndex]);
00168     }
00169     else {
00170         int vardIndex = rand() % motVard.size();
00171         int pavIndex = rand() % motPav.size();
00172         A.setVardas(motVard[vardIndex]);
00173         A.setPavarde(motPav[pavIndex]);
00174     }
00175
00176     cout << "Sugeneruotas vardas ir pavarde: " << A.getVardas() << " " << A.getPavarde() << "\n";
00177     cout << string(80, '-') << "\n";
00178
00179     // Automatiškai generuojamas atsitiktinis pazymiu kiekis (3-10)
00180     int n = rand() % 8 + 3;
00181     A.reservePaz(n); // Atminties rezervacija pazymiams
00182
00183     // Automatiškai sugeneruojami pazymiai
00184     cout << "Sugeneruota " << n << " pazymiu: ";
00185     for (int i = 0; i < n; i++) {
00186         int temp = generuotiPazymi();
00187         A.addPaz(temp);
00188         cout << temp << " ";
00189     }
00190     cout << "\n";
00191
00192     // Automatiškai sugeneruojamas egzaminas
00193     A.setEgz(generuotiEgzamina());
00194     cout << "Egzamino ivertinimas: " << A.getEgz() << "\n";
00195     cout << string(80, '-') << "\n";
00196
00197     // Vidurkio skaiciavimas
00198     if (!A.isPazEmpty()) {
00199         double vid = vidurkis(A.getPaz());
00200         A.setRez(galutinisBalas(vid, A.getEgz()));
00201         // Medianos skaiciavimas
00202         double med = mediana(A.getPaz());
00203         A.setMed(galutinisBalas(med, A.getEgz()));
00204     }
00205     else {
00206         A.setRez(A.getEgz() * 0.6);
00207         A.setMed(A.getEgz() * 0.6);
00208     }
00209
00210     grupe.push_back(A);
00211
00212     // Klausimas ar testi
00213     char atsakymas;
00214     bool atsakymasTeisingas = false;
00215     while (!atsakymasTeisingas) {
00216         cout << "Ar norite generuoti dar viena studenta? (T/N): ";
00217         cin >> atsakymas;
00218         if (atsakymas == 'T' || atsakymas == 't') {
00219             testiStudenta = true;
00220             atsakymasTeisingas = true;
00221             cout << string(80, '-') << "\n";
00222         }
00223         else if (atsakymas == 'N' || atsakymas == 'n') {
00224             testiStudenta = false;
00225             atsakymasTeisingas = true;
00226         }
00227         else {
00228             cout << "Klaida! Iveskite T arba N!\n";
00229             cin.clear();
00230             cin.ignore(numeric_limits<streamsize>::max(), '\n');
00231         }
00232     }
00233 }
00234 }

```

```

00235     catch (const runtime_error& e) {
00236         cout << "Klaida generuojant studentus: " << e.what() << "\n";
00237     }
00238     catch (const exception& e) {
00239         cout << "Netiketa klaida: " << e.what() << "\n";
00240     }
00241 }
00242
00243 void skaitytiIsFailo(vector<Studentas>& grupe) {
00244     try {
00245         string failoPavadinimas;
00246         cout << "Iveskite failo pavadinima: ";
00247         cin >> failoPavadinimas;
00248
00249         // Pridedamas testuojamas katalogas
00250         string kelias = "..\\studentai_test\\" + failoPavadinimas;
00251
00252         ifstream failas(kelias);
00253         if (!failas.is_open()) {
00254             throw runtime_error("Nepavyko atidaryti failo: " + kelias);
00255         }
00256
00257         // Praleidžiama antraste
00258         string eilute;
00259         getline(failas, eilute);
00260
00261         // Skaitomi studentai
00262         while (failas >> eilute) {
00263             Studentas A;
00264             A.setVardas(eilute);
00265             failas >> A.getPavarde();
00266
00267             vector<int> paz;
00268             paz.reserve(15); // Rezervuojama talpa (~15 pazymiu dazniausiai pakanka)
00269             int sk;
00270             while (failas >> sk) {
00271                 paz.push_back(sk);
00272                 if (failas.peek() == '\n') break;
00273             }
00274
00275             if (!paz.empty()) {
00276                 A.setEgz(paz.back());
00277                 paz.pop_back();
00278                 A.setPaz(paz);
00279                 A.setRez(galutinisBalas(vidurkis(A.getPaz()), A.getEgz()));
00280                 A.setMed(galutinisBalas(media(A.getPaz()), A.getEgz()));
00281                 grupe.push_back(A);
00282             }
00283             failas.ignore(numeric_limits<streamsize>::max(), '\n');
00284         }
00285
00286         failas.close();
00287
00288         if (grupe.empty()) {
00289             throw runtime_error("Failas yra tuscias arba neteisingai formatuotas!");
00290         }
00291
00292         cout << "Nuskaityta " << grupe.size() << " studentu!\n";
00293
00294         // Klausinama kur isvesti rezultatus
00295         if (!grupe.empty()) {
00296             cout << string(80, '-') << "\n";
00297             cout << "Pasirinkite isvesties buda:\n";
00298             cout << "1. Ivesti i terminala\n";
00299             cout << "2. Irasyti i faila\n";
00300             int isvestiesPasirinkimas;
00301             while (!(cin >> isvestiesPasirinkimas) || (isvestiesPasirinkimas != 1 &&
00302 isvestiesPasirinkimas != 2)) {
00302                 cout << "Klaida! Pasirinkite 1 arba 2: ";
00303                 cin.clear();
00304                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00305             }
00306             if (isvestiesPasirinkimas == 1) {
00307                 outputas(grupe);
00308             }
00309             else {
00310                 rasytIFaila(grupe);
00311             }
00312         }
00313     }
00314     catch (const runtime_error& e) {
00315         cout << "Klaida skaitant is failo: " << e.what() << "\n";
00316     }
00317     catch (const exception& e) {
00318         cout << "Netiketa klaida: " << e.what() << "\n";
00319     }
00320 }

```

```

00321
00322 void rasytIFaila(vector<Studentas>& grupe) {
00323     if (grupe.empty()) {
00324         cout << "Nera studentu duomenu!\n";
00325         return;
00326     }
00327
00328     // Klausima kaip rusiuoti
00329     cout << string(80, '-') << "\n";
00330     cout << "Pasirinkite rusiavimo buda:\n";
00331     cout << "1. Pagal varda (A-Z)\n";
00332     cout << "2. Pagal pavarde (A-Z)\n";
00333     cout << "3. Pagal galutini bala (vidurki) - didejimo tvarka\n";
00334     cout << "4. Pagal galutini bala (mediana) - didejimo tvarka\n";
00335
00336     int pasirinkimas;
00337     while (!(cin >> pasirinkimas) || pasirinkimas < 1 || pasirinkimas > 4) {
00338         cout << "Klaida! Iveskite skaiciu nuo 1 iki 4: ";
00339         cin.clear();
00340         cin.ignore(numeric_limits<streamsize>::max(), '\n');
00341     }
00342
00343     cout << "Pasirinktas rusiavimas: " << pasirinkimas << "\n";
00344
00345     // Pradedamas rusiavimo laiko matavimas
00346     auto pradzia = high_resolution_clock::now();
00347
00348     // Rusiuojama pagal pasirinkima
00349     switch (pasirinkimas) {
00350     case 1:
00351         sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b) {
00352             return a.getVardas() < b.getVardas();
00353         });
00354         break;
00355     case 2:
00356         sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b) {
00357             return a.getPavarde() < b.getPavarde();
00358         });
00359         break;
00360     case 3:
00361         sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b) {
00362             return a.getRez() < b.getRez();
00363         });
00364         break;
00365     case 4:
00366         sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b) {
00367             return a.getMed() < b.getMed();
00368         });
00369         break;
00370     default:
00371         cout << "Klaida! Neteisingas pasirinkimas. Rodoma be rusiavimo.\n";
00372         break;
00373     }
00374
00375     // Baigiamas rusiavimo laiko matavimas
00376     auto pabaiga = high_resolution_clock::now();
00377     duration<double> trukme = pabaiga - pradzia;
00378
00379     // Klausima failo pavadinimo
00380     string isvestiesFailas;
00381     cout << string(80, '-') << "\n";
00382     cout << "Iveskite isvesties failo pavadinima: ";
00383     cin >> isvestiesFailas;
00384
00385     // Sukuriamas kelias i testuojama kataloga
00386     string kelias = "..\\studentai_test\\" + isvestiesFailas;
00387
00388     ofstream failas(kelias);
00389     if (!failas.is_open()) {
00390         cout << "Klaida! Nepavyko sukurti failo: " << kelias << "\n";
00391         return;
00392     }
00393
00394     // Antrastes eilute
00395     failas << left << setw(20) << "Vardas" << setw(20) << "Pavarde"
00396         << setw(20) << "Galutinis (Vid.)" << setw(20) << "Galutinis (Med.)\n";
00397     failas << string(80, '-') << "\n";
00398
00399     // Studentu duomenys
00400     for (const auto& A : grupe) {
00401         failas << left << setw(20) << A.getVardas() << setw(20) << A.getPavarde()
00402             << setw(20) << fixed << setprecision(2) << A.getRez()
00403             << setw(20) << fixed << setprecision(2) << A.getMed() << "\n";
00404     }
00405     failas.close();
00406
00407

```

```

00408     cout << string(80, '-') << "\n";
00409     cout << "Duomenys sekmingai įrašyti į failą: " << kelias << "\n";
00410     cout << "Ivykdymo laikas: " << fixed << setprecision(7) << trukme.count() << " s\n";
00411 }

```

6.2 input.cpp

```

00001 #include <iostream>
00002 #include <limits>
00003
00004 #include "../header_files/input.h"
00005 #include "../header_files/mat_funkcijos.h"
00006
00007 using std::cin;
00008 using std::cout;
00009 using std::vector;
00010 using std::string;
00011 using std::numeric_limits;
00012 using std::streamsize;
00013
00014 void inputas(vector<Studentas>& grupe) {
00015     bool testiStudenta = true;
00016
00017     while (testiStudenta) {
00018         Studentas A;
00019
00020         bool vardasGeras = false;
00021         while (!vardasGeras) {
00022             cout << "Iveskite varda ir pavarde: ";
00023             cin >> std::ws; string v, p; cin >> v >> p; A.setVardas(v); A.setPavarde(p);
00024
00025             if (!arTikRaides(A.getVardas()) || !arTikRaides(A.getPavarde())) {
00026                 cout << "Klaida! Vardas ir pavarde turi buti sudaryti tik iš raidžių!\n";
00027                 cin.clear();
00028             }
00029             else {
00030                 vardasGeras = true;
00031             }
00032         }
00033
00034         cout << string(80, '-') << "\n";
00035         cout << "Iveskite semestro ivertinimus (0-10). Iveskite -1 kad baigtumete: \n";
00036         A.reservePaz(15); // Rezervuojama vieta pazymiams (sumazina atminties reallokacijas)
00037         int temp;
00038         while (true) {
00039             cout << "Iveskite pazymi (arba -1 kad baigtumete): ";
00040             if (!(cin >> temp)) {
00041                 cout << "Klaida! Iveskite skaiciu!\n";
00042                 cin.clear();
00043                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00044             }
00045             else if (temp == -1) {
00046                 break;
00047             }
00048             else if (temp < 0 || temp > 10) {
00049                 cout << "Klaida! Pazymys turi buti nuo 0 iki 10!\n";
00050             }
00051             else {
00052                 A.addPaz(temp);
00053             }
00054         }
00055
00056         cout << string(80, '-') << "\n";
00057
00058         bool egzaminasTeisingas = false;
00059         while (!egzaminasTeisingas) {
00060             cout << "Iveskite egzamina (0-10): ";
00061             int e; if (!(cin >> e)) {
00062                 cout << "Klaida! Iveskite skaiciu!\n";
00063                 cin.clear();
00064                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00065             }
00066             else if (e < 0 || e > 10) {
00067                 cout << "Klaida! Egzaminas turi buti nuo 0 iki 10!\n";
00068             }
00069             else {
00070                 A.setEgz(e); egzaminasTeisingas = true;
00071             }
00072         }
00073         cout << string(80, '-') << "\n";
00074
00075         // Vidurkio skaiciavimas
00076         if (!A.isPazEmpty()) {
00077             double vid = vidurkis(A.getPaz());

```



```

00078         A.setRez(galutinisBalas(vid, A.getEgz()));
00079         // Medianos skaiciavimas
00080         double med = mediana(A.getPaz());
00081         A.setMed(galutinisBalas(med, A.getEgz()));
00082     }
00083     else {
00084         A.setRez(A.getEgz() * 0.6);
00085         A.setMed(A.getEgz() * 0.6);
00086     }
00087
00088     grupe.push_back(A);
00089     A.clearPaz();
00090
00091     // Klausimas ar testi rankini generavima
00092     char atsakymas;
00093     bool atsakymasTeisingas = false;
00094     while (!atsakymasTeisingas) {
00095         cout << "Ar norite ivesti dar viena studenta? (T/N): ";
00096         cin >> atsakymas;
00097         if (atsakymas == 'T' || atsakymas == 't') {
00098             testiStudenta = true;
00099             atsakymasTeisingas = true;
00100             cout << string(80, '-') << "\n";
00101         }
00102         else if (atsakymas == 'N' || atsakymas == 'n') {
00103             testiStudenta = false;
00104             atsakymasTeisingas = true;
00105         }
00106         else {
00107             cout << "Klaida! Iveskite T arba N!\n";
00108             cin.clear();
00109             cin.ignore(numeric_limits<streamsize>::max(), '\n');
00110         }
00111     }
00112 }
00113 }
00114

```

6.3 mat_funkcijos.cpp

```

00001 #include "../header_files/mat_funkcijos.h"
00002 #include <algorithm>
00003 #include <cctype>
00004 #include <ctime>
00005
00006 using std::sort;
00007
00008 // Funkcija patikrinti ar vardas/pavarde turi tik raides, neleidzia jokiu simboliu
00009 bool arTikRaides(const string& str) {
00010     if (str.empty()) return false;
00011     for (char c : str) {
00012         if (!std::isalpha(c)) {
00013             return false;
00014         }
00015     }
00016     return true;
00017 }
00018
00019 // Medianos funkcija
00020 double mediana(vector<int> paz) {
00021     if (paz.empty()) return 0.0;
00022
00023     vector<int> temp = paz;
00024     sort(temp.begin(), temp.end());
00025     int n = temp.size();
00026     if (n % 2 == 0) {
00027         return (temp[n / 2 - 1] + temp[n / 2]) / 2.0;
00028     }
00029     else {
00030         return temp[n / 2];
00031     }
00032 }
00033
00034 // Vidurkio skaiciavimas is pazymiu vektoriaus
00035 double vidurkis(const vector<int>& paz) {
00036     if (paz.empty()) return 0.0;
00037     int sum = 0;
00038     for (int p : paz) {
00039         sum += p;
00040     }
00041     return sum * 1.0 / paz.size();
00042 }
00043
00044 // Galutinio balo skaiciavimas pagal vidurki/mediana ir egzamina

```

```

00045 double galutinisBalas(double vidMed, int egz) {
00046     return vidMed * 0.4 + egz * 0.6;
00047 }
00048
00049 // Generuoti atsitiktini pazymi nuo 0 iki 10
00050 int generuotiPazymi() {
00051     return rand() % 11;
00052 }
00053
00054 // Generuoti atsitiktini egzamina nuo 0 iki 10
00055 int generuotiEgzamina() {
00056     return rand() % 11;
00057 }
00058
00059 // Vector versions
00060 double mediana(Vector<int> paz) {
00061     if (paz.empty()) return 0.0;
00062
00063     Vector<int> temp = paz;
00064     sort(temp.begin(), temp.end());
00065     int n = temp.size();
00066     if (n % 2 == 0) {
00067         return (temp[n / 2 - 1] + temp[n / 2]) / 2.0;
00068     }
00069     else {
00070         return temp[n / 2];
00071     }
00072 }
00073
00074 // Vidurkio skaiciavimas is Vector pazymiu
00075 double vidurkis(const Vector<int>& paz) {
00076     if (paz.empty()) return 0.0;
00077     int sum = 0;
00078     for (int p : paz) {
00079         sum += p;
00080     }
00081     return sum * 1.0 / paz.size();
00082 }

```

6.4 menu.cpp

```

00001 #include <vector>
00002 #include <iostream>
00003 #include <string>
00004 #include <stdexcept>
00005
00006 #include "../header_files/menu.h"
00007 #include "../header_files/studentas.h"
00008 #include "../header_files/input.h"
00009 #include "../header_files/output.h"
00010 #include "../header_files/duomenu_valdymas.h"
00011 #include "../header_files/testavimas.h"
00012
00013 using std::cout;
00014 using std::cin;
00015 using std::string;
00016 using std::vector;
00017 using std::invalid_argument;
00018 using std::exception;
00019
00020 void menu() {
00021     vector<Studentas>grupe;
00022     int pas; // Pasirinkimas
00023     bool testi = true;
00024
00025     while (testi) {
00026         cout << string(80, '-') << "\n";
00027         cout << "Studentu Rezultatu skaiciavimo aplikacija\n";
00028         cout << string(80, '-') << "\n";
00029         cout << "1. Ivesti duomenis ranka \n";
00030         cout << "2. Generuoti tik pazymius \n";
00031         cout << "3. Generuoti studentu vardus, pavardes ir pazymius \n";
00032         cout << "4. Nuskaityti duomenis is failo \n";
00033         cout << "5. Sukurti testavimo failus (1000 - 10000000 irasu)\n";
00034         cout << "6. Atlikti spartos analize (nuskaitymas, rusiavimas, dalijimas, isvedimas)\n";
00035         cout << "7. Baigti darba \n";
00036
00037         try {
00038             if (!(cin >> pas)) {
00039                 throw invalid_argument("Neteisingas ivedimas! Iveskite skaiciu.");
00040             }
00041         }
00042         catch (const invalid_argument& e) {
00043             cout << "Klaida: " << e.what() << "\n";

```

```

00044         cin.clear();
00045         cin.ignore(1000, '\n');
00046         continue;
00047     }
00048
00049     // Pasirinkimo isvestis
00050     switch (pas) {
00051     case 1:
00052         try {
00053             cout << "1. Ivesti duomenisss ranka \n";
00054             inputas(grupe);
00055             outputas(grupe);
00056             grupe.clear();
00057         }
00058         catch (const exception& e) {
00059             cout << "Klaida vykdant 1 funkcija: " << e.what() << "\n";
00060             grupe.clear();
00061         }
00062         break;
00063     case 2:
00064         try {
00065             cout << "2. Generuoti tik pazymius \n";
00066             generuotiPaz(grupe);
00067             outputas(grupe);
00068             grupe.clear();
00069         }
00070         catch (const exception& e) {
00071             cout << "Klaida vykdant 2 funkcija: " << e.what() << "\n";
00072             grupe.clear();
00073         }
00074         break;
00075     case 3:
00076         try {
00077             cout << "3. Generuoti studentu vardus, pavardes ir pazymius \n";
00078             generuotiVardIrPav(grupe);
00079             outputas(grupe);
00080             grupe.clear();
00081         }
00082         catch (const exception& e) {
00083             cout << "Klaida vykdant 3 funkcija: " << e.what() << "\n";
00084             grupe.clear();
00085         }
00086         break;
00087     case 4:
00088         try {
00089             cout << "4. Nuskaityti duomenis is failo \n";
00090             skaitytiIsFailo(grupe);
00091             grupe.clear();
00092         }
00093         catch (const exception& e) {
00094             cout << "Klaida vykdant 4 funkcija: " << e.what() << "\n";
00095             grupe.clear();
00096         }
00097         break;
00098     case 5:
00099         try {
00100             cout << "5. Testavimo failu kurimas \n";
00101             sukurtiTestavimoFailus();
00102         }
00103         catch (const exception& e) {
00104             cout << "Klaida vykdant generavima: " << e.what() << "\n";
00105         }
00106         break;
00107     case 6:
00108         try {
00109             cout << "6. Spartos analizes vykdymas \n";
00110             atliktiSpartosAnalyze();
00111         }
00112         catch (const exception& e) {
00113             cout << "Klaida vykdant spartos analize: " << e.what() << "\n";
00114         }
00115         break;
00116     case 7:
00117         cout << "Programa uzdaroma \n";
00118         testi = false;
00119         break;
00120
00121     // Isvestis ivedus netinkama pasirinkima
00122     default:
00123         cout << "Klaida! Pasirinkite skaiciu nuo 1 iki 7 \n";
00124         cin.clear();
00125         cin.ignore(1000, '\n');
00126         break;
00127     }
00128 }
00129 }

```

6.5 output.cpp

```

00001 #include <iostream>
00002 #include <iomanip>
00003 #include <algorithm>
00004 #include <limits>
00005 #include <chrono>
00006
00007 #include "../header_files/output.h"
00008
00009 using std::cin;
00010 using std::cout;
00011 using std::string;
00012 using std::vector;
00013 using std::left;
00014 using std::right;
00015 using std::setw;
00016 using std::setprecision;
00017 using std::sort;
00018 using std::fixed;
00019 using std::numeric_limits;
00020 using std::streamsize;
00021 using std::chrono::high_resolution_clock;
00022 using std::chrono::duration;
00023
00024 void outputas(vector<Studentas>& grupe) {
00025     if (grupe.empty()) {
00026         cout << "Nera studentu duomenu!\n";
00027         return;
00028     }
00029
00030     // Klausime kaip rusiuoti
00031     cout << string(80, '-') << "\n";
00032     cout << "Pasirinkite rusiavimo buda:\n";
00033     cout << "1. Pagal varda (A-Z)\n";
00034     cout << "2. Pagal pavarde (A-Z)\n";
00035     cout << "3. Pagal galutini bala (vidurki) - didejimo tvarka\n";
00036     cout << "4. Pagal galutini bala (mediana) - didejimo tvarka\n";
00037
00038     int pasirinkimas;
00039     while (!(cin > pasirinkimas) || pasirinkimas < 1 || pasirinkimas > 4) {
00040         cout << "Klaida! Iveskite skaiciu nuo 1 iki 4: ";
00041         cin.clear();
00042         cin.ignore(numeric_limits<streamsize>::max(), '\n');
00043     }
00044
00045     cout << "Pasirinktas rusiavimas: " << pasirinkimas << "\n";
00046
00047     // Pradedamas rusiavimo laiko matavimas
00048     auto pradzia = high_resolution_clock::now();
00049
00050     // Rusiuojama pagal pasirinkima
00051     switch (pasirinkimas) {
00052     case 1:
00053         sort(grupe.begin(), grupe.end(), comparePagalVarda);
00054         break;
00055     case 2:
00056         sort(grupe.begin(), grupe.end(), comparePagalPavarde);
00057         break;
00058     case 3:
00059         sort(grupe.begin(), grupe.end(), comparePagalReza);
00060         break;
00061     case 4:
00062         sort(grupe.begin(), grupe.end(), comparePagalMeda);
00063         break;
00064     default:
00065         cout << "Klaida! Neteisingas pasirinkimas. Rodoma be rusiavimo.\n";
00066         break;
00067     }
00068
00069     // Baigiamas rusiavimo laiko matavimas
00070     auto pabaiga = high_resolution_clock::now();
00071     duration<double> trukme = pabaiga - pradzia;
00072
00073     // Antrastes eilute
00074     cout << string(80, '-') << "\n";
00075     cout << left << setw(20) << "Vardas" << setw(20) << "Pavarde"
00076         << setw(20) << "Galutinis (Vid.)" << setw(20) << "Galutinis (Med.)\n";
00077
00078     // Studentu duomenys
00079     for (const auto& A : grupe) {
00080         cout << left << setw(20) << A.getVardas() << setw(20) << A.getPavarde()
00081             << setw(20) << fixed << setprecision(2) << A.getRez()
00082             << setw(20) << fixed << setprecision(2) << A.getMed() << "\n";
00083     }
00084

```

```

00085     cout << string(80, '-') << "\n";
00086     cout << "Ivykdymo laikas: " << fixed << setprecision(7) << trukme.count() << " s\n";
00087 }

```

6.6 reallocation_comparison.cpp

```

00001 #include <iostream>
00002 #include <vector>
00003 #include <chrono>
00004 #include <iomanip>
00005 #include <string>
00006 #include "../header_files/Vector.h"
00007
00008 using namespace std;
00009 using namespace chrono;
00010
00011 void palygintiRealokacijas() {
00012     cout << "\n" << string(80, '=') << "\n";
00013     cout << "std::vector vs custom Vector Reallocation Count Test\n";
00014     cout << "Test: Push 100,000,000 int elements\n";
00015     cout << string(80, '=') << "\n\n";
00016
00017     // Test std::vector reallocation count (manual count)
00018     cout << "Testing std::vector<int>...\n";
00019     cout.flush();
00020     auto start_std = high_resolution_clock::now();
00021     {
00022         vector<int> v;
00023         int realloc_count = 0;
00024         size_t prev_capacity = 0;
00025
00026         for (long long i = 1; i <= 100000000; ++i) {
00027             if (v.capacity() > prev_capacity) {
00028                 realloc_count++;
00029                 prev_capacity = v.capacity();
00030             }
00031             v.push_back(i);
00032         }
00033
00034         auto end_std = high_resolution_clock::now();
00035         duration<double> duration_std = end_std - start_std;
00036
00037         cout << "   Final size:           " << v.size() << "\n";
00038         cout << "   Final capacity:        " << v.capacity() << "\n";
00039         cout << "   Reallocation count:    " << realloc_count << "\n";
00040         cout << "   Time:                  " << fixed << setprecision(3) << duration_std.count() << " s\n\n";
00041     }
00042
00043     // Test custom Vector reallocation count
00044     cout << "Testing custom Vector<int>...\n";
00045     cout.flush();
00046     auto start_vec = high_resolution_clock::now();
00047     {
00048         Vector<int> v;
00049
00050         for (long long i = 1; i <= 100000000; ++i) {
00051             v.push_back(i);
00052         }
00053
00054         auto end_vec = high_resolution_clock::now();
00055         duration<double> duration_vec = end_vec - start_vec;
00056
00057         cout << "   Final size:           " << v.size() << "\n";
00058         cout << "   Final capacity:        " << v.capacity() << "\n";
00059         cout << "   Reallocation count:    " << v.get_reallocation_count() << "\n";
00060         cout << "   Time:                  " << fixed << setprecision(3) << duration_vec.count() << " s\n\n";
00061     }
00062
00063     cout << string(80, '=') << "\n";
00064     cout << "ANALYSIS\n";
00065     cout << string(80, '=') << "\n";
00066     cout << "\nBoth containers use the same 2x growth strategy.\n";
00067     cout << "With 100,000,000 elements:\n";
00068     cout << "- Expected reallocations: ~27 (log2(100M))\n";
00069     cout << "- Initial capacity: 1\n";
00070     cout << "- Final capacity: ~134,217,728 (2^27)\n\n";
00071 }

```

6.7 Studentas.cpp

```

00001 #include "../header_files/Studentas.h"

```

```

00002 #include "../header_files/mat_funkcijos.h"
00003 #include <iostream>
00004
00005 using std::istream;
00006
00007 // Nuskaitymas is srauto konstruktoriuje
00008 Studentas::Studentas(std::istream& is) : Zmogus(), egz_(0), rez_(0.0), med_(0.0) {
00009     readStudent(is);
00010 }
00011
00012 // ===== RULE OF FIVE =====
00013
00014 // Destruktorius
00015 Studentas::~Studentas() {
00016     paz_.clear();
00017 }
00018
00019 // Copy konstruktorius
00020 Studentas::Studentas(const Studentas& other)
00021     : Zmogus(other.vardas_, other.pavarde_), paz_(other.paz_),
00022     egz_(other.egz_), rez_(other.rez_), med_(other.med_) {
00023 }
00024
00025 // Copy assignment operator
00026 Studentas& Studentas::operator=(const Studentas& other) {
00027     if (this != &other) {
00028         setVardas(other.getVardas());
00029         setPavarde(other.getPavarde());
00030         paz_ = other.paz_;
00031         egz_ = other.egz_;
00032         rez_ = other.rez_;
00033         med_ = other.med_;
00034     }
00035     return *this;
00036 }
00037
00038 // Move konstruktorius
00039 Studentas::Studentas(Studentas&& other) noexcept
00040     : Zmogus(std::move(other.vardas_), std::move(other.pavarde_),
00041         paz_(std::move(other.paz_)), egz_(other.egz_), rez_(other.rez_),
00042         med_(other.med_) {
00043     other.setEgz(0);
00044     other.setRez(0.0);
00045     other.setMed(0.0);
00046     other.paz_.clear();
00047 }
00048
00049 // Move assignment operator
00050 Studentas& Studentas::operator=(Studentas&& other) noexcept {
00051     if (this != &other) {
00052         paz_.clear();
00053         setVardas(std::move(other.vardas_));
00054         setPavarde(std::move(other.pavarde_));
00055         paz_ = std::move(other.paz_);
00056         egz_ = other.egz_;
00057         rez_ = other.rez_;
00058         med_ = other.med_;
00059         other.setEgz(0);
00060         other.setRez(0.0);
00061         other.setMed(0.0);
00062         other.paz_.clear();
00063     }
00064     return *this;
00065 }
00066
00067 // Implementacija abstraktaus metodo iš Zmogus
00068 string Studentas::getInfo() const {
00069     return getVardas() + " " + getPavarde();
00070 }
00071
00072 // Privati pagalbine funkcija
00073 void Studentas::paskaiciuotiGalutinius() {
00074     if (!paz_.empty()) {
00075         double vid = vidurkis(paz_);
00076         double med = mediana(paz_);
00077         rez_ = galutinisBalas(vid, egz_);
00078         med_ = galutinisBalas(med, egz_);
00079     } else {
00080         rez_ = egz_ * 0.6;
00081         med_ = egz_ * 0.6;
00082     }
00083 }
00084
00085 // Implementacija abstraktaus metodo paskaiciuoti() iš Zmogus
00086 void Studentas::paskaiciuoti() {
00087     paskaiciuotiGalutinius();
00088 }

```

```

00089
00090 // Duomenų nuskaitymas ir apskaiciavimas
00091 std::istream& Studentas::readStudent(std::istream& is) {
00092     string vardas, pavarde;
00093     is >> vardas >> pavarde;
00094     if (!is) return is;
00095     setVardas(vardas);
00096     setPavarde(pavarde);
00097     paz_.clear();
00098     int p;
00099     for (int i = 0; i < 5; i++) {
00100         is >> p;
00101         paz_.push_back(p);
00102     }
00103     is >> egz_;
00104     paskaiciuotiGalutinius();
00105     return is;
00106 }
00107
00108 // ===== I/O OPERATORIAI =====
00109
00110 // Išvesties operatorius
00111 std::ostream& operator<<(std::ostream& os, const Studentas& s) {
00112     os << "Vardas: " << s.getVardas() << " | Pavarde: " << s.getPavarde() << " | ";
00113     os << "Egzaminas: " << s.egz_ << " | Vidurkis rezultatas: " << s.rez_ << " | ";
00114     os << "Mediana rezultatas: " << s.med_ << " | Pazymiai: ";
00115
00116     if (!s.paz_.empty()) {
00117         for (int paz : s.paz_) {
00118             os << paz << " ";
00119         }
00120     } else {
00121         os << "(nera)";
00122     }
00123
00124     return os;
00125 }
00126
00127 // ?vesties operatorius - nuskaity student? iš srauto
00128 std::istream& operator>>(std::istream& is, Studentas& s) {
00129     return s.readStudent(is);
00130 }
00131
00132 // ===== LYGINIMO FUNKCIJOS =====
00133
00134 bool comparePagalVarda(const Studentas& a, const Studentas& b) {
00135     return a.getVardas() < b.getVardas();
00136 }
00137
00138 bool comparePagalPavarde(const Studentas& a, const Studentas& b) {
00139     return a.getPavarde() < b.getPavarde();
00140 }
00141
00142 bool comparePagalReza(const Studentas& a, const Studentas& b) {
00143     return a.getRez() < b.getRez();
00144 }
00145
00146 bool comparePagalMeda(const Studentas& a, const Studentas& b) {
00147     return a.getMed() < b.getMed();
00148 }

```

6.8 testavimas.cpp

```

00001 #include <iostream>
00002 #include <fstream>
00003 #include <vector>
00004 #include <string>
00005 #include <chrono>
00006 #include <iomanip>
00007 #include <random>
00008 #include <algorithm>
00009 #include <stdexcept>
00010 #include <list>
00011 #include <deque>
00012 #include <type_traits>
00013 #include <iterator> // std::make_move_iterator
00014
00015 #include "../header_files/testavimas.h"
00016 #include "../header_files/mat_funkcijos.h"
00017 #include "../header_files/Vector.h"
00018
00019 using std::cout;
00020 using std::cin;
00021 using std::vector;

```

```

00022 using std::string;
00023 using std::ofstream;
00024 using std::ifstream;
00025 using std::chrono::high_resolution_clock;
00026 using std::chrono::duration;
00027 using std::fixed;
00028 using std::setprecision;
00029 using std::left;
00030 using std::setw;
00031 using std::to_string;
00032 using std::move;
00033 using std::list;
00034 using std::deque;
00035
00036 const vector<int> SZ = { 1000, 10000, 100000, 1000000, 10000000 };
00037 const int paz_kiekis = 5;
00038
00039 void sukurtiTestavimoFailus() {
00040     int pas;
00041     cout << "\nPasirinkite, kuri faila norite sukurti:\n";
00042     cout << "1. 1000 irasu\n";
00043     cout << "2. 10000 irasu\n";
00044     cout << "3. 100000 irasu\n";
00045     cout << "4. 1 000000 irasu\n";
00046     cout << "5. 10000000 irasu\n";
00047     cout << "6. Visus auksciau isvardintus\n";
00048     cout << "Jusu pasirinkimas: ";
00049     cin >> pas;
00050
00051     vector<int> pasirinkti_SZ;
00052     if (pas == 1) pasirinkti_SZ = { 1000 };
00053     else if (pas == 2) pasirinkti_SZ = { 10000 };
00054     else if (pas == 3) pasirinkti_SZ = { 100000 };
00055     else if (pas == 4) pasirinkti_SZ = { 1000000 };
00056     else if (pas == 5) pasirinkti_SZ = { 10000000 };
00057     else if (pas == 6) pasirinkti_SZ = SZ;
00058     else {
00059         cout << "Neteisingas pasirinkimas!\n";
00060         return;
00061     }
00062
00063     srand(static_cast<unsigned>(time(nullptr)));
00064     cout << "Failu kurimas prasidejo...\n";
00065     cout << string(80, '-') << "\n";
00066
00067     for (int n : pasirinkti_SZ) {
00068         string fname = "Studentai_test\\studentai_" + to_string(n) + ".txt";
00069
00070         // Pradedamas matuoti laikas tiems irasams kurti
00071         auto start = high_resolution_clock::now();
00072
00073         ofstream out(fname);
00074
00075         // Antraste
00076         out << left << setw(20) << "Vardas" << setw(20) << "Pavarde";
00077         for (int i = 1; i <= paz_kiekis; i++) {
00078             out << setw(8) << ("ND" + to_string(i));
00079         }
00080         out << "Egz\n";
00081
00082         // Tiesiai irasome i faila (be vektorių pagal uzduoties salyga)
00083         for (int i = 1; i <= n; i++) {
00084             out << left << setw(20) << ("Vardas" + to_string(i)) << setw(20) << ("Pavarde" + to_string(i));
00085             for (int k = 0; k < paz_kiekis; k++) {
00086                 out << setw(8) << (rand() % 10 + 1);
00087             }
00088             out << (rand() % 10 + 1) << "\n";
00089         }
00090         out.close();
00091
00092         // Stabdyti laikmati ir isvesti i ekrana laika
00093         auto end = high_resolution_clock::now();
00094         duration<double> diff = end - start;
00095
00096         cout << n << " irasu failo sukurimo laikas: " << fixed << setprecision(5) << diff.count() << "
00097             s.\n";
00098     }
00099 }
00100 template <typename Container>
00101 void nuskaitytiDuomenis(const string& fname, Container& grupe) {
00102     ifstream in(fname);
00103     string header;
00104     std::getline(in, header);
00105
00106     Studentas st;
00107     st.reservePaz(paz_kiekis);

```



```

00108
00109     while (true) {
00110         string v, p;
00111         if (!(in » v » p)) break;
00112         st.setVardas(v);
00113         st.setPavarde(p);
00114         st.clearPaz();
00115         int pr;
00116         for (int i = 0; i < paz_kiekis; i++) {
00117             in » pr;
00118             st.addPaz(pr);
00119         }
00120         int egz; in » egz; st.setEgz(egz);
00121         st.setRez(galutinisBalas(vidurkis(st.getPaz()), st.getEgz()));
00122         grupe.push_back(st);
00123     }
00124     in.close();
00125 }
00126
00127 template <typename Container>
00128 double tirtiKonteineri(int n, const string& fname, int strat) {
00129     auto total_start = high_resolution_clock::now();
00130
00131     // 1. Duomenų nuskaitymas iš failo
00132     auto start = high_resolution_clock::now();
00133
00134     Container grupe;
00135     if constexpr (std::is_same_v<Container, vector<Studentas>>) {
00136         try {
00137             grupe.reserve(n); // list neturi reserve() funkcijos
00138         } catch (const std::exception& e) {
00139             cout << "Nepakanka atminties rezervuoti " << n << " elementu!\n";
00140             return 0.0;
00141         }
00142     }
00143
00144     nuskaitytiDuomenis(fname, grupe);
00145
00146     auto end = high_resolution_clock::now();
00147     duration<double> ms_read = end - start;
00148     cout << n << " Irasu failo nuskaitymo laikas: " << fixed << setprecision(5) << ms_read.count() << "
00149     s.\n";
00150
00151     // 2. Rūšiavimas
00152     start = high_resolution_clock::now();
00153     if constexpr (std::is_same_v<Container, list<Studentas>>) {
00154         grupe.sort([](const Studentas& a, const Studentas& b) {
00155             return a.getRez() < b.getRez();
00156         });
00157     } else {
00158         std::sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b) {
00159             return a.getRez() < b.getRez();
00160         });
00161     }
00162     end = high_resolution_clock::now();
00163     duration<double> ms_sort = end - start;
00164     cout << n << " Irasu rūšiavimas didejimo tvarka: " << fixed << setprecision(5) << ms_sort.count() << "
00165     s.\n";
00166
00167     // 3. Skirstymas į dvi grupes
00168     start = high_resolution_clock::now();
00169     Container vargsiukai;
00170     Container kietiakai;
00171
00172     if (strat == 1) {
00173         if constexpr (std::is_same_v<Container, vector<Studentas>>) {
00174             vargsiukai.reserve(n / 2 + 100);
00175             kietiakai.reserve(n / 2 + 100);
00176         }
00177         for (auto& s : grupe) {
00178             if (s.getRez() < 5.0) {
00179                 vargsiukai.push_back(move(s));
00180             } else {
00181                 kietiakai.push_back(move(s));
00182             }
00183         }
00184         Container().swap(grupe); // Išvalome pradini konteineri
00185     }
00186     else if (strat == 2) {
00187         if constexpr (std::is_same_v<Container, vector<Studentas>>) {
00188             // Sukrupuoja vektoriu vietoje (O(N) laikas):
00189             auto splitPoint = std::partition(grupe.begin(), grupe.end(), [](const Studentas& s) {
00190                 return s.getRez() >= 5.0; // Salyga kietiakams
00191             });
00192         }

```

```

00193         // I anksto rezervuojame atminties vargiukams (nebutina, bet dar labiau pagreitina)
00194         vargsiukai.reserve(std::distance(splitPoint, grupe.end()));
00195
00196         // NAUDOJAME MOVE: Perkeliame vargiukus i pagrindinio vektoriaus galo i naujaji.
00197         vargsiukai.insert(vargsiukai.end(),
00198             std::make_move_iterator(splitPoint),
00199             std::make_move_iterator(grupe.end()));
00200
00201         // Itriname perkeltus elementus i originalaus vektoriaus.
00202         grupe.erase(splitPoint, grupe.end());
00203         kietiakai = move(grupe); // Like grupeje yra kietiakai
00204     }
00205     else {
00206         auto it = grupe.begin();
00207         while (it != grupe.end()) {
00208             if (it->getRez() < 5.0) {
00209                 vargsiukai.push_back(move(*it));
00210                 it = grupe.erase(it); // vector/deque atveju tai leta, list atveju - greita
00211             }
00212             else {
00213                 ++it;
00214             }
00215         }
00216         kietiakai = move(grupe); // Like grupeje yra kietiakai
00217     }
00218 }
00219 else if (strat == 3) {
00220     if constexpr (std::is_same_v<Container, list<Studentas>) {
00221         // list atveju efektyviausia yra ikirpti elementus per splice + remove_if ar pagal salyga
00222         auto it = std::partition(grupe.begin(), grupe.end(), [](const Studentas& s) {
00223             return s.getRez() < 5.0; // Vargiukaikeliauja i pati prieki
00224         });
00225         // Ikerpame vargiukus i grupes tiesiai i vargiuku list be memory re-allocation
00226         vargsiukai.splice(vargsiukai.begin(), grupe, grupe.begin(), it);
00227         kietiakai = move(grupe);
00228     }
00229     else {
00230         // std::vector ir std::deque atveju efektyviausia naudoti std::partition in-place
00231         if constexpr (std::is_same_v<Container, vector<Studentas>) {
00232             vargsiukai.reserve(n / 2 + 100);
00233         }
00234         auto it = std::stable_partition(grupe.begin(), grupe.end(), [](const Studentas& s) {
00235             return s.getRez() < 5.0;
00236         });
00237
00238         // Iteruojam per pradia kur atsidure vargiukai po stable_partition pakeitimu
00239         vargsiukai.insert(vargsiukai.end(), std::make_move_iterator(grupe.begin()),
00240             std::make_move_iterator(it));
00241         // Kas liko originalioj grupej iteruojame i kietiakius (nuo it iki end)
00242         kietiakai.insert(kietiakai.end(), std::make_move_iterator(it),
00243             std::make_move_iterator(grupe.end()));
00244
00245         Container().swap(grupe); // Sunaikinam originala
00246     }
00247
00248     end = high_resolution_clock::now();
00249     duration<double> ms_split = end - start;
00250     cout << n << " Irasu dalijimo laikas (" << strat << "-a strategija): " << fixed << setprecision(5) <<
00251     ms_split.count() << " s.\n";
00252
00253     // 4.1 Irasome vargsiukus i faila
00254     start = high_resolution_clock::now();
00255     ofstream outV("vargsiukai\\vargsiukai_" + to_string(n) + ".txt");
00256     outV << left << setw(20) << "Vardas" << setw(20) << "Pavarde" << "Galutinis (Vid.)\n";
00257     for (const auto& s : vargsiukai) {
00258         outV << left << setw(20) << s.getVardas() << setw(20) << s.getPavarde() << fixed << setprecision(2) <<
00259         s.getRez() << "\n";
00260     }
00261     outV.close();
00262     Container().swap(vargsiukai); // Islaisviname RAM perrasydami!
00263     end = high_resolution_clock::now();
00264     duration<double> ms_write_v = end - start;
00265     cout << n << " Irasu 'vargsiuku' irasyimo laikas: " << fixed << setprecision(5) << ms_write_v.count() <<
00266     " s.\n";
00267
00268     // 4.2 Irasome kietiakius i faila
00269     start = high_resolution_clock::now();
00270     ofstream outK("kietiakai\\kietiakai_" + to_string(n) + ".txt");
00271     outK << left << setw(20) << "Vardas" << setw(20) << "Pavarde" << "Galutinis (Vid.)\n";
00272     for (const auto& s : kietiakai) {
00273         outK << left << setw(20) << s.getVardas() << setw(20) << s.getPavarde() << fixed << setprecision(2) <<
00274         s.getRez() << "\n";
00275     }
00276     outK.close();
00277     Container().swap(kietiakai); // Islaisviname ir kietiakius!
00278     end = high_resolution_clock::now();

```

```

00274     duration<double> ms_write_k = end - start;
00275     cout << n << " Irasu 'kietiaku' irasyimo laikas: " << fixed << setprecision(5) << ms_write_k.count() << "
s.\n";
00276
00277     // Bendras laikas
00278     auto total_end = high_resolution_clock::now();
00279     duration<double> ms_total = total_end - total_start;
00280     return ms_total.count();
00281 }
00282
00283 void atliktiSpartosAnalyze() {
00284     int pas, kont, strat, test_kartai;
00285     cout << "\nPasirinkite, su kuriuo failu norite atlikti spartos analize:\n";
00286     cout << "1. 1000 irasu failas\n";
00287     cout << "2. 10000 irasu failas\n";
00288     cout << "3. 100000 irasu failas\n";
00289     cout << "4. 1000000 irasu failas\n";
00290     cout << "5. 10000000 irasu failas\n";
00291     cout << "6. Visi failai is eiles\n";
00292     cout << "7. VECTOR TESTAVIMAS (100k, 1M)\n";
00293     cout << "Jusu pasirinkimas: ";
00294     cin >> pas;
00295
00296     if (pas == 7) {
00297         // Vector testing
00298         int strat_v, test_kartai_v;
00299         cout << "\nPasirinkite testavimo strategija:\n";
00300         cout << "1. 1 strategija (Sukurti 2 naujus konteinerius)\n";
00301         cout << "2. 2 strategija (Sukurti 1 nauja konteineri, trinti is pagrindinio)\n";
00302         cout << "Jusu pasirinkimas: ";
00303         cin >> strat_v;
00304
00305         cout << "\nIveskite, kiek kartu norite pakartoti testavima: ";
00306         cin >> test_kartai_v;
00307
00308         cout << "\nPradedama Vector spartos analize...\n";
00309
00310         vector<int> pasirinkti_SZ = { 100000, 1000000 };
00311
00312         for (int n : pasirinkti_SZ) {
00313             string fname = "Studentai_test\\studentai_" + to_string(n) + ".txt";
00314
00315             ifstream patikrinimas(fname);
00316             if (!patikrinimas.good()) {
00317                 cout << "Failo " << fname << " nera. Praleidžiama.\n";
00318                 continue;
00319             }
00320             patikrinimas.close();
00321
00322             cout << string(80, '-') << "\n";
00323             cout << n << " irasu Vector spartos analize\n";
00324             cout << string(80, '-') << "\n";
00325
00326             double visas_laikas = 0.0;
00327
00328             for (int i = 0; i < test_kartai_v; i++) {
00329                 cout << "\nTESTO NUMERIS: " << i + 1 << "\n";
00330                 visas_laikas += tirtiKonteineri<Vector<Studentas>>(n, fname, strat_v);
00331             }
00332
00333             cout << "\n=====\\n";
00334             cout << "Vidutinis Vector testo laikas po " << test_kartai_v << " bandymu:\n";
00335             cout << fixed << setprecision(5) << visas_laikas / test_kartai_v << " s.\n";
00336             cout << "=====\\n\\n";
00337         }
00338         return;
00339     }
00340
00341     vector<int> pasirinkti_SZ;
00342     if (pas == 1) pasirinkti_SZ = { 1000 };
00343     else if (pas == 2) pasirinkti_SZ = { 10000 };
00344     else if (pas == 3) pasirinkti_SZ = { 100000 };
00345     else if (pas == 4) pasirinkti_SZ = { 1000000 };
00346     else if (pas == 5) pasirinkti_SZ = { 10000000 };
00347     else if (pas == 6) pasirinkti_SZ = SZ;
00348     else {
00349         cout << "Neteisingas pasirinkimas!\n";
00350         return;
00351     }
00352
00353     cout << "\nPasirinkite konteinerio tipa testavimui:\n";
00354     cout << "1. std::vector\n";
00355     cout << "2. std::list\n";
00356     cout << "3. std::deque\n";
00357     cout << "Jusu pasirinkimas: ";
00358     cin >> kont;
00359

```

```

00360     cout << "\nPasirinkite testavimo strategija:\n";
00361     cout << "1. 1 strategija (Sukurti 2 naujus konteinerius)\n";
00362     cout << "2. 2 strategija (Sukurti 1 nauja konteineri, trinti is pagrindinio)\n";
00363     cout << "3. 3 strategija (Optimizuoti algoritmu metodai)\n";
00364     cout << "Jusu pasirinkimas: ";
00365     cin >> strat;
00366
00367     cout << "\nIveskite, kiek kartu norite pakartoti testavima (laiku vidurkiui): ";
00368     cin >> test_kartai;
00369
00370     cout << "\nPradedama spartos analize pagal nurodytus zingsnius...\n";
00371
00372     for (int n : pasirinkti_SZ) {
00373         string fname = "Studentai_test\\studentai_" + to_string(n) + ".txt";
00374
00375         // Tikrinimas ar failas atsidaro nenaudojant filesystem
00376         ifstream patikrinimas(fname);
00377         if (!patikrinimas.good()) {
00378             cout << "Failo " << fname << " nera. Praleidžiama.\n";
00379             continue;
00380         }
00381         patikrinimas.close();
00382
00383         cout << string(80, '-') << "\n";
00384         cout << n << " Irasu spartos analize\n";
00385         cout << string(80, '-') << "\n";
00386
00387         double visas_laikas = 0.0;
00388
00389         for (int i = 0; i < test_kartai; i++) {
00390             cout << "\nTESTO NUMERIS: " << i + 1 << "\n";
00391             if (kont == 1) visas_laikas += tirtiKonteineri<vector<Studentas>>(n, fname, strat);
00392             else if (kont == 2) visas_laikas += tirtiKonteineri<list<Studentas>>(n, fname, strat);
00393             else if (kont == 3) visas_laikas += tirtiKonteineri<deque<Studentas>>(n, fname, strat);
00394             else {
00395                 cout << "Neteisingas konteinerio tipas!\n";
00396                 return;
00397             }
00398         }
00399         cout << "\n=====n";
00400         cout << "Vidutinis viso testo (nuskaitymas+dalijimas+isvedimas) laikas po " << test_kartai << "
bandymu:\n";
00401         cout << fixed << setprecision(5) << visas_laikas / test_kartai << " s.\n";
00402         cout << "=====n\n";
00403     }
00404 }
00405
00406 void PadalintiStudentusZabioGreiciu(std::vector<Studentas>& studentai, std::vector<Studentas>&
vargsiukai) {
00407
00408     // 1. Sukrupuojuame vektoriu vietoje (O(N) laikas):
00409     // Visi, kuriu balas >= 5.0 atsiduria priekyje, o < 5.0 - gale.
00410     // Jei jums BUTINA ilaikyti jau esama eilikuma (pvz., abecelini),
00411     // vietoj 'std::partition' naudokite 'std::stable_partition'.
00412     auto splitPoint = std::partition(studentai.begin(), studentai.end(), [](const Studentas& s) {
00413         return s.getRez() >= 5.0; // Pakeista is galutinis i rez
00414     });
00415
00416     // 2. I anksto rezervuojame atminties vargiukams (nebutina, bet dar labiau pagreitina)
00417     vargsiukai.reserve(std::distance(splitPoint, studentai.end()));
00418
00419     // 3. NAUDOJAME MOVE: Perkeliame vargiukus i pagrindinio vektoriaus galo i naujaji.
00420     // std::make_move_iterator utikrina, kad tekstai ir kiti duomenys nebutu kopijuojami i naujo.
00421     vargsiukai.insert(vargsiukai.end(),
00422                     std::make_move_iterator(splitPoint),
00423                     std::make_move_iterator(studentai.end()));
00424
00425     // 4. Itriname perkeltus elementus i originalaus vektoriaus.
00426     // Kadangi triname I GALO, jokio duomenu stumdymo nebelieka - operacija ivyksta per O(1).
00427     studentai.erase(splitPoint, studentai.end());
00428 }

```

6.9 Zmogus.cpp

```

00001 #include "../header_files/Zmogus.h"
00002 #include <iostream>
00003
00004 using std::ostream;
00005
00006 /**
00007  * @brief Išvesties operatorius - spausdina žmogaus pagrindinius duomenis
00008  * @param os Išvesties srautas
00009  * @param z Žmogaus objektas
00010  * @return Išvesties srautas

```

```
00011 */
00012 std::ostream& operator<<(std::ostream& os, const Zmogus& z) {
00013     os << "Vardas: " << z.vardas_ << " | Pavarde: " << z.pavarde_;
00014     return os;
00015 }
```

6.10 duomenu_valdymas.h

```
00001 #pragma once
00002
00003 #include <vector>
00004 #include <string>
00005
00006 #include "studentas.h"
00007
00008 using std::string;
00009 using std::vector;
00010
00011 void generuotiPaz(vector<Studentas>& grupe);
00012 void generuotiVardIrPav(vector<Studentas>& grupe);
00013 void skaitytiIsFailo(vector<Studentas>& grupe);
00014 void rasytiIFaila(vector<Studentas>& grupe);
```

6.11 input.h

```
00001 #pragma once
00002
00003 #include <vector>
00004 #include <string>
00005 #include "studentas.h"
00006
00007 using std::vector;
00008 using std::string;
00009
00010 void inputas(vector<Studentas>& grupe);
```

6.12 Mat_funkcijos.h

```
00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005 #include "Vector.h"
00006
00007 using std::string;
00008 using std::vector;
00009
00010 bool arTikRaides(const string& str);
00011 double mediana(vector<int> paz);
00012 double vidurkis(const vector<int>& paz); // Priima per referencą greičiui
00013 double galutinisBalas(double vidMed, int egz);
00014 int generuotiPazymi();
00015 int generuotiEgzamina();
00016
00017 // Vector versions
00018 double mediana(Vector<int> paz);
00019 double vidurkis(const Vector<int>& paz);
00020
```

6.13 menu.h

```
00001 #pragma once
00002
00003 void menu();
00004
```

6.14 output.h

```
00001 #pragma once
00002
00003 #include <vector>
```

```

00004 #include <string>
00005
00006 #include "studentas.h"
00007
00008 using std::vector;
00009 using std::string;
00010
00011 void outputas(vector<Studentas>& grupe);

```

6.15 Studentas.h

```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <string>
00005 #include "Vector.h"
00006 #include "Zmogus.h"
00007
00008 using std::string;
00009
00010 // Išvestinė klasė iš Zmogus - aprašo studentą
00011 // Paveldi žmogaus duomenis (vardas, pavardė) iš Zmogus klasės
00012 // Turi papildomus duomenis: pažymius, egzamino rezultata ir galutinius rezultatus
00013 // Implementuoja Rule of Five
00014 class Studentas : public Zmogus {
00015 private:
00016     Vector<int> paz_;
00017     int egz_;
00018     double rez_;
00019     double med_;
00020
00021     // Privatas helperis skaičiavimams
00022     void paskaiciuotiGalutinius();
00023
00024     // Implementacija abstraktaus metodo iš Zmogus
00025     virtual void paskaiciuoti() override;
00026
00027 public:
00028     // Numatytasis konstruktorius
00029     Studentas() : Zmogus("A", "BB"), egz_(0), rez_(0.0), med_(0.0) {}
00030
00031     // Parametrizuotas konstruktorius
00032     Studentas(string v, string p, int e) : Zmogus(v, p), egz_(e), rez_(0.0), med_(0.0) {}
00033
00034     // Konstruktorius su nuskaitymu is srauto
00035     Studentas(std::istream& is);
00036
00037     // Rule of Five
00038
00039     // Destruktorius
00040     ~Studentas();
00041
00042     // Copy konstruktorius
00043     Studentas(const Studentas& other);
00044
00045     // Copy assignment operator
00046     Studentas& operator=(const Studentas& other);
00047
00048     // Move konstruktorius
00049     Studentas(Studentas&& other) noexcept;
00050
00051     // Move assignment operator
00052     Studentas& operator=(Studentas&& other) noexcept;
00053
00054     // Getters
00055     inline const Vector<int>& getPaz() const { return paz_; }
00056     inline Vector<int>& getPaz() { return paz_; }
00057     inline int getEgz() const { return egz_; }
00058     inline double getRez() const { return rez_; }
00059     inline double getMed() const { return med_; }
00060
00061     // Setters
00062     inline void setEgz(int e) { egz_ = e; }
00063     inline void setRez(double r) { rez_ = r; }
00064     inline void setMed(double m) { med_ = m; }
00065
00066     // Darbas su pažymiais
00067     inline void addPaz(int p) { paz_.push_back(p); }
00068     inline void clearPaz() { paz_.clear(); }
00069     inline void reservePaz(size_t n) { paz_.reserve(n); }
00070     inline bool isPazEmpty() const { return paz_.empty(); }
00071     inline void setPaz(const Vector<int>& p) { paz_ = p; }
00072
00073     // Skaitymas ir skaičiavimas

```

```

00074     std::istream& readStudent(std::istream& is);
00075
00076     // Implementacija abstraktaus metodo iš Zmogus
00077     virtual string getInfo() const override;
00078
00079     // I/O operatoriai
00080     friend std::ostream& operator<<(std::ostream& os, const Studentas& s);
00081     friend std::istream& operator>>(std::istream& is, Studentas& s);
00082 };
00083
00084 // Ne klasės narės, bet su klase tiesiogiai susijusios lyginimo funkcijos
00085 bool comparePagalVarda(const Studentas& a, const Studentas& b);
00086 bool comparePagalPavarde(const Studentas& a, const Studentas& b);
00087 bool comparePagalReza(const Studentas& a, const Studentas& b);
00088 bool comparePagalMeda(const Studentas& a, const Studentas& b);
00089

```

6.16 testavimas.h

```

00001 #pragma once
00002 #include <vector>
00003 #include "studentas.h"
00004 #include "Vector.h"
00005
00006 void sukurtiTestavimoFailus();
00007 void atliktiSpartosAnalyze();
00008 void atliktiSpartosAnalyzeVectorVersion();

```

6.17 Vector.h

```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <stdexcept>
00005 #include <utility>
00006 #include <algorithm>
00007 #include <limits>
00008 #include <vector>
00009
00010 /**
00011  * @brief Random access iterator for Vector<T> container
00012  *
00013  * VectorIterator provides bidirectional and random access to Vector elements.
00014  * Supports all standard iterator operations including comparison, arithmetic,
00015  * and dereferencing.
00016  *
00017  * @tparam T The element type that the iterator points to
00018  *
00019  * @see Vector
00020  */
00021 template <typename T>
00022 class VectorIterator {
00023 public:
00024     using iterator_category = std::random_access_iterator_tag;
00025     using value_type = T;
00026     using difference_type = std::ptrdiff_t;
00027     using pointer = T*;
00028     using reference = T&
00029
00030 private:
00031     T* _ptr;
00032
00033 public:
00034     VectorIterator(T* ptr = nullptr) : _ptr(ptr) {}
00035
00036     reference operator*() const { return *_ptr; }
00037     pointer operator->() const { return _ptr; }
00038
00039     VectorIterator& operator++() {
00040         ++_ptr;
00041         return *this;
00042     }
00043     VectorIterator operator++(int) {
00044         VectorIterator temp = *this;
00045         ++_ptr;
00046         return temp;
00047     }
00048
00049     VectorIterator& operator--() {
00050         --_ptr;
00051         return *this;
00052     }

```

```

00052     }
00053     VectorIterator operator--(int) {
00054         VectorIterator temp = *this;
00055         --_ptr;
00056         return temp;
00057     }
00058
00059     VectorIterator operator+(difference_type n) const {
00060         return VectorIterator(_ptr + n);
00061     }
00062     VectorIterator operator-(difference_type n) const {
00063         return VectorIterator(_ptr - n);
00064     }
00065
00066     VectorIterator& operator+=(difference_type n) {
00067         _ptr += n;
00068         return *this;
00069     }
00070     VectorIterator& operator-=(difference_type n) {
00071         _ptr -= n;
00072         return *this;
00073     }
00074
00075     reference operator[](difference_type n) const {
00076         return _ptr[n];
00077     }
00078
00079     bool operator==(const VectorIterator& other) const { return _ptr == other._ptr; }
00080     bool operator!=(const VectorIterator& other) const { return _ptr != other._ptr; }
00081     bool operator<(const VectorIterator& other) const { return _ptr < other._ptr; }
00082     bool operator>(const VectorIterator& other) const { return _ptr > other._ptr; }
00083     bool operator<=(const VectorIterator& other) const { return _ptr <= other._ptr; }
00084     bool operator>=(const VectorIterator& other) const { return _ptr >= other._ptr; }
00085
00086     difference_type operator-(const VectorIterator& other) const {
00087         return _ptr - other._ptr;
00088     }
00089 };
00090
00091 /**
00092  * @class Vector
00093  * @brief Custom dynamic array container template
00094  *
00095  * Vector<T> is a generic container that stores elements of type T in a
00096  * dynamically allocated array. It provides O(1) amortized time complexity
00097  * for push_back(), while maintaining contiguous memory layout like std::vector.
00098  *
00099  * The class implements the Rule of Five pattern and fully supports deep copy
00100  * and move semantics.
00101  *
00102  * **Key Features:**
00103  * - Dynamic memory management (automatic resizing)
00104  * - Random access iterators (begin(), end(), rbegin(), rend())
00105  * - Copy and move semantics (Rule of Five)
00106  * - Standard container operations (push_back, pop_back, insert, erase, etc.)
00107  * - Comparable performance to std::vector for most operations
00108  *
00109  * **Memory Strategy:**
00110  * - When capacity is exhausted, reallocates with roughly 1.5x growth factor
00111  * - Tracks reallocation count for performance analysis
00112  * - Supports manual reserve() for optimization
00113  *
00114  * **Example Usage:**
00115  * @code
00116  * Vector<int> v;
00117  * v.push_back(10);
00118  * v.push_back(20);
00119  * v.reserve(100);           // Pre-allocate capacity
00120  *
00121  * for (int elem : v) {
00122  *     std::cout << elem << " ";
00123  * }
00124  * @endcode
00125  *
00126  * @tparam T Element type (must support copy and move semantics)
00127  *
00128  * @see VectorIterator
00129  * @see std::vector
00130  */
00131 template <typename T>
00132 class Vector {
00133 private:
00134     T* _data;
00135     size_t _size;
00136     size_t _capacity;
00137     size_t _realloc_count;
00138

```



```

00139     void reallocate(size_t new_capacity) {
00140         if (new_capacity < _size) {
00141             throw std::invalid_argument("New capacity must be >= current size");
00142         }
00143         T* new_data = new T[new_capacity];
00144         for (size_t i = 0; i < _size; ++i) {
00145             new_data[i] = _data[i];
00146         }
00147         delete[] _data;
00148         _data = new_data;
00149         _capacity = new_capacity;
00150         ++_realloc_count;
00151     }
00152
00153 public:
00154     Vector() : _data(nullptr), _size(0), _capacity(0), _realloc_count(0) {}
00155     explicit Vector(size_t capacity) : _size(0), _capacity(capacity), _realloc_count(0) {
00156         _data = capacity > 0 ? new T[capacity] : nullptr;
00157     }
00158     Vector(size_t size, const T& value) : _size(size), _capacity(size), _realloc_count(0) {
00159         _data = size > 0 ? new T[size] : nullptr;
00160         for (size_t i = 0; i < size; ++i) _data[i] = value;
00161     }
00162
00163     // Conversion constructor from std::vector
00164     Vector(const std::vector<T>& other) : _size(other.size()), _capacity(other.size()),
00165         _realloc_count(0) {
00166         _data = _size > 0 ? new T[_size] : nullptr;
00167         for (size_t i = 0; i < _size; ++i) _data[i] = other[i];
00168     }
00169     ~Vector() { delete[] _data; _data = nullptr; _size = 0; _capacity = 0; }
00170
00171     Vector(const Vector& other) : _size(other._size), _capacity(other._capacity), _realloc_count(0) {
00172         _data = _capacity > 0 ? new T[_capacity] : nullptr;
00173         for (size_t i = 0; i < _size; ++i) _data[i] = other._data[i];
00174     }
00175     Vector& operator=(const Vector& other) {
00176         if (this != &other) {
00177             delete[] _data;
00178             _size = other._size;
00179             _capacity = other._capacity;
00180             _realloc_count = 0;
00181             _data = _capacity > 0 ? new T[_capacity] : nullptr;
00182             for (size_t i = 0; i < _size; ++i) _data[i] = other._data[i];
00183         }
00184         return *this;
00185     }
00186
00187     Vector(Vector&& other) noexcept : _data(other._data), _size(other._size),
00188         _capacity(other._capacity), _realloc_count(other._realloc_count) {
00189         other._data = nullptr;
00190         other._size = 0;
00191         other._capacity = 0;
00192         other._realloc_count = 0;
00193     }
00194     Vector& operator=(Vector&& other) noexcept {
00195         if (this != &other) {
00196             delete[] _data;
00197             _data = other._data;
00198             _size = other._size;
00199             _capacity = other._capacity;
00200             _realloc_count = other._realloc_count;
00201             other._data = nullptr;
00202             other._size = 0;
00203             other._capacity = 0;
00204             other._realloc_count = 0;
00205         }
00206         return *this;
00207     }
00208
00209     inline size_t size() const { return _size; }
00210     inline size_t capacity() const { return _capacity; }
00211     inline bool empty() const { return _size == 0; }
00212
00213     inline T& operator[](size_t index) { return _data[index]; }
00214     inline const T& operator[](size_t index) const { return _data[index]; }
00215
00216     T& at(size_t index) {
00217         if (index >= _size) {
00218             throw std::out_of_range("Vector index out of range");
00219         }
00220         return _data[index];
00221     }
00222
00223     const T& at(size_t index) const {
00224         if (index >= _size) {

```

```

00225         throw std::out_of_range("Vector index out of range");
00226     }
00227     return _data[index];
00228 }
00229
00230 void clear() {
00231     _size = 0;
00232 }
00233
00234 T& front() {
00235     if (_size == 0) {
00236         throw std::out_of_range("Vector is empty: cannot access front()");
00237     }
00238     return _data[0];
00239 }
00240
00241 const T& front() const {
00242     if (_size == 0) {
00243         throw std::out_of_range("Vector is empty: cannot access front()");
00244     }
00245     return _data[0];
00246 }
00247
00248 T& back() {
00249     if (_size == 0) {
00250         throw std::out_of_range("Vector is empty: cannot access back()");
00251     }
00252     return _data[_size - 1];
00253 }
00254
00255 const T& back() const {
00256     if (_size == 0) {
00257         throw std::out_of_range("Vector is empty: cannot access back()");
00258     }
00259     return _data[_size - 1];
00260 }
00261
00262 void reserve(size_t new_capacity) {
00263     if (new_capacity > _capacity) {
00264         reallocate(new_capacity);
00265     }
00266 }
00267
00268 void shrink_to_fit() {
00269     if (_capacity > _size) {
00270         if (_size == 0) {
00271             delete[] _data;
00272             _data = nullptr;
00273             _capacity = 0;
00274         } else {
00275             reallocate(_size);
00276         }
00277     }
00278 }
00279
00280 void resize(size_t new_size) {
00281     if (new_size > _capacity) {
00282         reallocate(new_size);
00283     }
00284     _size = new_size;
00285 }
00286
00287 void resize(size_t new_size, const T& value) {
00288     if (new_size > _capacity) {
00289         reallocate(new_size);
00290     }
00291     if (new_size > _size) {
00292         for (size_t i = _size; i < new_size; ++i) {
00293             _data[i] = value;
00294         }
00295     }
00296     _size = new_size;
00297 }
00298
00299 void swap(Vector& other) noexcept {
00300     std::swap(_data, other._data);
00301     std::swap(_size, other._size);
00302     std::swap(_capacity, other._capacity);
00303     std::swap(_realloc_count, other._realloc_count);
00304 }
00305
00306 bool operator==(const Vector& other) const {
00307     if (_size != other._size) return false;
00308     for (size_t i = 0; i < _size; ++i) {
00309         if (_data[i] != other._data[i]) return false;
00310     }
00311     return true;

```

```

00312     }
00313
00314     bool operator!=(const Vector& other) const {
00315         return !(*this == other);
00316     }
00317
00318     bool operator<(const Vector& other) const {
00319         for (size_t i = 0; i < _size && i < other._size; ++i) {
00320             if (_data[i] < other._data[i]) return true;
00321             if (_data[i] > other._data[i]) return false;
00322         }
00323         return _size < other._size;
00324     }
00325
00326     bool operator>(const Vector& other) const {
00327         return other < *this;
00328     }
00329
00330     bool operator<=(const Vector& other) const {
00331         return !(other < *this);
00332     }
00333
00334     bool operator>=(const Vector& other) const {
00335         return !(*this < other);
00336     }
00337
00338     using iterator = VectorIterator<T>;
00339     using const_iterator = VectorIterator<const T>;
00340
00341     iterator begin() { return iterator(_data); }
00342     iterator end() { return iterator(_data + _size); }
00343
00344     const_iterator begin() const { return const_iterator(_data); }
00345     const_iterator end() const { return const_iterator(_data + _size); }
00346
00347     const_iterator cbegin() const { return const_iterator(_data); }
00348     const_iterator cend() const { return const_iterator(_data + _size); }
00349
00350     using reverse_iterator = std::reverse_iterator<iterator>;
00351     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00352
00353     reverse_iterator rbegin() { return reverse_iterator(end()); }
00354     reverse_iterator rend() { return reverse_iterator(begin()); }
00355
00356     const_reverse_iterator rbegin() const { return const_reverse_iterator(end()); }
00357     const_reverse_iterator rend() const { return const_reverse_iterator(begin()); }
00358
00359     const_reverse_iterator crbegin() const { return const_reverse_iterator(end()); }
00360     const_reverse_iterator crend() const { return const_reverse_iterator(begin()); }
00361
00362     void assign(size_t count, const T& value) {
00363         clear();
00364         if (count > _capacity) {
00365             reallocate(count);
00366         }
00367         for (size_t i = 0; i < count; ++i) {
00368             _data[i] = value;
00369         }
00370         _size = count;
00371     }
00372
00373     template<typename InputIt>
00374     void assign(InputIt first, InputIt last) {
00375         // This is a specialized version - only works with Vector iterators
00376         // For general case, user should use clear() + for loop
00377     }
00378
00379
00380     iterator insert(const_iterator pos, const T& value) {
00381         size_t index = std::distance(cbegin(), pos);
00382         if (index > _size) {
00383             throw std::out_of_range("Iterator out of range");
00384         }
00385
00386         if (_size >= _capacity) {
00387             size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
00388             reallocate(new_capacity);
00389         }
00390
00391         for (size_t i = _size; i > index; --i) {
00392             _data[i] = _data[i - 1];
00393         }
00394         _data[index] = value;
00395         ++_size;
00396
00397         return iterator(_data + index);
00398     }

```

```

00399
00400 // Non-const overload for insert (accepts non-const iterator, treats as const_iterator)
00401 iterator insert(iterator pos, const T& value) {
00402     // pos is a mutable iterator, but we just use it as a position
00403     // Convert to size_t using cbegin()
00404     size_t index = std::distance(begin(), pos);
00405     if (index > _size) {
00406         throw std::out_of_range("Iterator out of range");
00407     }
00408
00409     if (_size >= _capacity) {
00410         size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
00411         reallocate(new_capacity);
00412     }
00413
00414     for (size_t i = _size; i > index; --i) {
00415         _data[i] = _data[i - 1];
00416     }
00417     _data[index] = value;
00418     ++_size;
00419
00420     return iterator(_data + index);
00421 }
00422
00423 // Range insert overload with three arguments
00424 template<typename InputIt>
00425 iterator insert(const_iterator pos, InputIt first, InputIt last) {
00426     size_t index = std::distance(cbegin(), pos);
00427     if (index > _size) {
00428         throw std::out_of_range("Iterator out of range");
00429     }
00430
00431     // Count elements manually
00432     size_t count = 0;
00433     for (InputIt it = first; it != last; ++it) {
00434         ++count;
00435     }
00436
00437     if (_size + count > _capacity) {
00438         reallocate(std::max(_size + count, _capacity * 2));
00439     }
00440
00441     // Shift elements to the right
00442     for (size_t i = _size; i > index; --i) {
00443         _data[i + count - 1] = _data[i - 1];
00444     }
00445
00446     // Insert new elements
00447     size_t i = index;
00448     for (InputIt it = first; it != last; ++it, ++i) {
00449         _data[i] = *it;
00450     }
00451     _size += count;
00452
00453     return iterator(_data + index);
00454 }
00455
00456 // Range insert overload with non-const iterator
00457 template<typename InputIt>
00458 iterator insert(iterator pos, InputIt first, InputIt last) {
00459     size_t index = std::distance(begin(), pos);
00460     if (index > _size) {
00461         throw std::out_of_range("Iterator out of range");
00462     }
00463
00464     // Count elements manually
00465     size_t count = 0;
00466     for (InputIt it = first; it != last; ++it) {
00467         ++count;
00468     }
00469
00470     if (_size + count > _capacity) {
00471         reallocate(std::max(_size + count, _capacity * 2));
00472     }
00473
00474     // Shift elements to the right
00475     for (size_t i = _size; i > index; --i) {
00476         _data[i + count - 1] = _data[i - 1];
00477     }
00478
00479     // Insert new elements
00480     size_t i = index;
00481     for (InputIt it = first; it != last; ++it, ++i) {
00482         _data[i] = *it;
00483     }
00484     _size += count;
00485

```

```

00486         return iterator(_data + index);
00487     }
00488
00489     iterator erase(const_iterator pos) {
00490         size_t index = std::distance(cbegin(), pos);
00491         if (index >= _size) {
00492             throw std::out_of_range("Iterator out of range");
00493         }
00494
00495         for (size_t i = index; i < _size - 1; ++i) {
00496             _data[i] = _data[i + 1];
00497         }
00498         --_size;
00499
00500         return iterator(_data + index);
00501     }
00502
00503     iterator erase(const_iterator first, const_iterator last) {
00504         size_t first_index = std::distance(cbegin(), first);
00505         size_t last_index = std::distance(cbegin(), last);
00506
00507         if (first_index > _size || last_index > _size || first_index > last_index) {
00508             throw std::out_of_range("Invalid range");
00509         }
00510
00511         size_t count = last_index - first_index;
00512         for (size_t i = first_index; i < _size - count; ++i) {
00513             _data[i] = _data[i + count];
00514         }
00515         _size -= count;
00516
00517         return iterator(_data + first_index);
00518     }
00519
00520     // Non-const overload for erase (single element)
00521     iterator erase(iterator pos) {
00522         size_t index = std::distance(begin(), pos);
00523         if (index >= _size) {
00524             throw std::out_of_range("Iterator out of range");
00525         }
00526
00527         for (size_t i = index; i < _size - 1; ++i) {
00528             _data[i] = _data[i + 1];
00529         }
00530         --_size;
00531
00532         return iterator(_data + index);
00533     }
00534
00535     // Non-const overload for erase (range)
00536     iterator erase(iterator first, iterator last) {
00537         size_t first_index = std::distance(begin(), first);
00538         size_t last_index = std::distance(begin(), last);
00539
00540         if (first_index > _size || last_index > _size || first_index > last_index) {
00541             throw std::out_of_range("Invalid range");
00542         }
00543
00544         size_t count = last_index - first_index;
00545         for (size_t i = first_index; i < _size - count; ++i) {
00546             _data[i] = _data[i + count];
00547         }
00548         _size -= count;
00549
00550         return iterator(_data + first_index);
00551     }
00552
00553     template<typename... Args>
00554     void emplace_back(Args&&... args) {
00555         if (_size >= _capacity) {
00556             size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
00557             reallocate(new_capacity);
00558         }
00559         new (_data + _size) T(std::forward<Args>(args)...);
00560         ++_size;
00561     }
00562
00563     template<typename... Args>
00564     iterator emplace(const_iterator pos, Args&&... args) {
00565         size_t index = std::distance(cbegin(), pos);
00566         if (index > _size) {
00567             throw std::out_of_range("Iterator out of range");
00568         }
00569
00570         if (_size >= _capacity) {
00571             size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
00572             reallocate(new_capacity);

```

```

00573     }
00574
00575     for (size_t i = _size; i > index; --i) {
00576         _data[i] = _data[i - 1];
00577     }
00578     new (_data + index) T(std::forward<Args>(args)...);
00579     ++_size;
00580
00581     return iterator(_data + index);
00582 }
00583
00584 size_t max_size() const {
00585     return std::numeric_limits<size_t>::max() / sizeof(T);
00586 }
00587
00588 inline T* data() { return _data; }
00589 inline const T* data() const { return _data; }
00590
00591 inline size_t get_reallocation_count() const { return _realloc_count; }
00592 inline void reset_reallocation_count() { _realloc_count = 0; }
00593
00594 /**
00595  * @brief Add element to the end of the vector (copy semantics)
00596  *
00597  * Appends a copy of the given value to the end. If the current size equals
00598  * capacity, the vector doubles its capacity and reallocates.
00599  *
00600  * **Time Complexity:** O(1) amortized
00601  *
00602  * @param value The value to append (copied)
00603  *
00604  * @throws std::bad_alloc if memory allocation fails
00605  *
00606  * @see push_back(T&&), pop_back(), emplace_back()
00607  *
00608  * @example
00609  * @code
00610  * Vector<int> v;
00611  * v.push_back(42);
00612  * v.push_back(100);
00613  * @endcode
00614  */
00615 void push_back(const T& value) {
00616     if (_size >= _capacity) {
00617         size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
00618         reallocate(new_capacity);
00619     }
00620     _data[_size] = value;
00621     ++_size;
00622 }
00623
00624 /**
00625  * @brief Add element to the end of the vector (move semantics)
00626  *
00627  * Appends an rvalue reference to the vector. The element is moved
00628  * (not copied) into the vector. This is more efficient for expensive-to-copy types.
00629  *
00630  * **Time Complexity:** O(1) amortized
00631  *
00632  * @param value The rvalue reference to append (moved)
00633  *
00634  * @throws std::bad_alloc if memory allocation fails
00635  *
00636  * @see push_back(const T&), pop_back(), emplace_back()
00637  *
00638  * @example
00639  * @code
00640  * Vector<std::string> v;
00641  * v.push_back(std::string("Hello")); // Moves the temporary
00642  * @endcode
00643  */
00644 void push_back(T&& value) {
00645     if (_size >= _capacity) {
00646         size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
00647         reallocate(new_capacity);
00648     }
00649     _data[_size] = std::move(value);
00650     ++_size;
00651 }
00652
00653 /**
00654  * @brief Remove the last element from the vector
00655  *
00656  * Decrements the size by one, effectively removing the last element.
00657  * Does nothing if the vector is empty. Note: the element destructor
00658  * is NOT called (similar to std::vector::pop_back for POD types).
00659  *

```

```

00660      * **Time Complexity:** O(1) constant
00661      *
00662      * **Precondition:** Empty vector is safe to call on (no-op)
00663      *
00664      * @see push_back(), front(), back()
00665      *
00666      * @example
00667      * @code
00668      * Vector<int> v = {1, 2, 3};
00669      * v.pop_back(); // Now size is 2, last element is 2
00670      * @endcode
00671      */
00672      void pop_back() {
00673          if (_size > 0) {
00674              --_size;
00675          }
00676      }
00677 };
00678
00679
00680
00681

```

6.18 Zmogus.h

```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <string>
00005
00006 using std::string;
00007
00008 /**
00009  * @class Zmogus
00010  * @brief Abstrakti bazinė klasė bendram žmogaus aprašymui
00011  *
00012  * Ši klasė nuo v1.5 versijos aprašo bendrą žmogaus duomenis (vardas, pavardė).
00013  * Yra abstrakti - jos objektų kurti negalima, tik iš jos išvestinės klases.
00014  *
00015  * Paveldi iš šios klasės: Studentas
00016  */
00017 class Zmogus {
00018 protected:
00019     string vardas_;
00020     string pavarde_;
00021
00022     /**
00023      * @brief Pagalbinis metodas žmogaus duomenims paskaičiuoti
00024      * Abstraktus metodas - turi būti implementuotas išvestinėse klasėse
00025      */
00026     virtual void paskaičiuoti() = 0;
00027
00028 public:
00029     /**
00030      * @brief Parametrizuotas konstruktorius
00031      * @param v Žmogaus vardas
00032      * @param p Žmogaus pavardė
00033      */
00034     Zmogus(const string& v, const string& p)
00035         : vardas_(v), pavarde_(p) {}
00036
00037     /**
00038      * @brief Parametrizuotas konstruktorius su numatytosiomis reikšmėmis
00039      */
00040     Zmogus() : vardas_("Asmuo"), pavarde_("Asmenys") {}
00041
00042     /**
00043      * @brief Virtualus destruktorius - reikalingas polimorfizmui
00044      */
00045     virtual ~Zmogus() {}
00046
00047     // ===== GETTERS =====
00048     /**
00049      * @return Žmogaus vardą
00050      */
00051     inline string getVardas() const { return vardas_; }
00052
00053     /**
00054      * @return Žmogaus pavardę
00055      */
00056     inline string getPavarde() const { return pavarde_; }
00057
00058     // ===== SETTERS =====
00059     /**

```

```

00060      * @brief Nustato žmogaus vardą
00061      */
00062      inline void setVardas(const string& v) { vardas_ = v; }
00063
00064      /**
00065      * @brief Nustato žmogaus pavardę
00066      */
00067      inline void setPavarde(const string& p) { pavarde_ = p; }
00068
00069      // ===== ABSTRAKTUS METODAS =====
00070      /**
00071      * @brief Gražina žmogaus informaciją
00072      * Šis metodas turi būti implementuotas visose išvestinėse klasėse
00073      */
00074      virtual string getInfo() const = 0;
00075
00076      // ===== I/O OPERATORIAI (friend functions) =====
00077      /**
00078      * @brief Išvesties operatorius - spausdina žmogaus duomenis
00079      */
00080      friend std::ostream& operator<<(std::ostream& os, const Zmogus& z);
00081 };

```

6.19 objekt1uzd.cpp

```

00001 #include "header_files/menu.h"
00002
00003 int main() {
00004     menu();
00005     return 0;
00006 }

```

6.20 test_studentas.cpp

```

00001 #include <gtest/gtest.h>
00002 #include <iostream>
00003 #include <sstream>
00004 #include <cassert>
00005 #include <vector>
00006 #include "header_files/Studentas.h"
00007 #include "header_files/Vector.h"
00008
00009 using std::cout;
00010 using std::cin;
00011 using std::endl;
00012 using std::string;
00013 using std::vector;
00014 using std::stringstream;
00015
00016 int testai_praletii = 0;
00017 int testai_nepraletii = 0;
00018
00019 void printTestHeader(const string& testName) {
00020     cout << "\n" << string(60, '=') << endl;
00021     cout << "TEST: " << testName << endl;
00022     cout << string(60, '=') << endl;
00023 }
00024
00025 void testPassed(const string& info) {
00026     cout << "[PRIIMTAS] " << info << endl;
00027     testai_praletii++;
00028 }
00029
00030 void testFailed(const string& info) {
00031     cout << "[NEPRIIMTAS] " << info << endl;
00032     testai_nepraletii++;
00033 }
00034
00035 // TEST #1: Default konstruktorius
00036 void test_default_constructor() {
00037     printTestHeader("Default Konstruktorius");
00038
00039     Studentas s;
00040
00041     assert(s.getVardas() == "A");
00042     testPassed("Vardas inicijuotas i 'A'");
00043
00044     assert(s.getPavarde() == "BB");
00045     testPassed("Pavarde inicijuota i 'BB'");
00046
00047     assert(s.getEgz() == 0);

```



```

00048     testPassed("Egzaminas inicijuotas i 0");
00049
00050     assert(s.getRez() == 0.0);
00051     testPassed("Rezultatas inicijuotas i 0.0");
00052
00053     assert(s.getMed() == 0.0);
00054     testPassed("Mediana inicijuota i 0.0");
00055
00056     assert(s.isPazEmpty());
00057     testPassed("Pazymiai sarasas tucias");
00058 }
00059
00060 // TEST #2: Parametrizuotas konstruktorius
00061 void test_parametrized_constructor() {
00062     printTestHeader("Parametrizuotas Konstruktorius");
00063
00064     Studentas s("Jonas", "Jonaitis", 95);
00065
00066     assert(s.getVardas() == "Jonas");
00067     testPassed("Vardas teisingai priskirtas");
00068
00069     assert(s.getPavarde() == "Jonaitis");
00070     testPassed("Pavarde teisingai priskirta");
00071
00072     assert(s.getEgz() == 95);
00073     testPassed("Egzaminas teisingai priskirtas");
00074 }
00075
00076 // TEST #3: Copy konstruktorius
00077 void test_copy_constructor() {
00078     printTestHeader("Copy Konstruktorius");
00079
00080     Studentas s1("Petras", "Petrauskas", 85);
00081     s1.addPaz(8);
00082     s1.addPaz(9);
00083     s1.addPaz(10);
00084
00085     // Kuriam kopija
00086     Studentas s2 = s1;
00087
00088     assert(s2.getVardas() == s1.getVardas());
00089     testPassed("Kopijoje vardas sutampa");
00090
00091     assert(s2.getPavarde() == s1.getPavarde());
00092     testPassed("Kopijoje pavarde sutampa");
00093
00094     assert(s2.getEgz() == s1.getEgz());
00095     testPassed("Kopijoje egzaminas sutampa");
00096
00097     assert(s2.getPaz().size() == s1.getPaz().size());
00098     testPassed("Kopijoje pazymiu kiekis sutampa");
00099
00100     assert(s2.getPaz()[0] == s1.getPaz()[0]);
00101     testPassed("Kopijoje pazymiu vertes sutampa");
00102
00103     // Tikriname, kad tai skirtingi objektai (deep copy)
00104     Studentas s3 = s1;
00105     s3.addPaz(100);
00106     assert(s1.getPaz().size() != s3.getPaz().size());
00107     testPassed("Deep copy veikia - modifikacija nepaveikia originalo");
00108 }
00109
00110 // TEST #4: Copy assignment operator
00111 void test_copy_assignment() {
00112     printTestHeader("Copy Assignment Operatorius");
00113
00114     Studentas s1("Marta", "Martine", 92);
00115     s1.addPaz(7);
00116     s1.addPaz(8);
00117     s1.addPaz(9);
00118
00119     Studentas s2;
00120     s2 = s1;
00121
00122     assert(s2.getVardas() == "Marta");
00123     testPassed("Priskyrimo metu vardas is naujo priskirtas");
00124
00125     assert(s2.getPavarde() == "Martine");
00126     testPassed("Priskyrimo metu pavarde is naujo priskirta");
00127
00128     assert(s2.getPaz().size() == 3);
00129     testPassed("Priskyrimo metu pazymiai is naujo priskirti");
00130
00131     // Self-assignment check
00132     s2 = s2;
00133     assert(s2.getVardas() == "Marta");
00134     testPassed("Self-assignment saugiai veikia");

```

```

00135 }
00136
00137 // TEST #5: Move konstruktorius
00138 void test_move_constructor() {
00139     printTestHeader("Move Konstruktorius");
00140
00141     Studentas s1("Laima", "Laima", 88);
00142     s1.addPaz(6);
00143     s1.addPaz(7);
00144     s1.addPaz(8);
00145
00146     // Move konstruktorius
00147     Studentas s2 = std::move(s1);
00148
00149     assert(s2.getVardas() == "Laima");
00150     testPassed("Move konstruktorius - vardas is naujo priskirtas");
00151
00152     assert(s2.getPaz().size() == 3);
00153     testPassed("Move konstruktorius - pazymiai is naujo priskirti");
00154
00155     // s1 turetu tureti tuscius pazymius
00156     assert(s1.isPazEmpty());
00157     testPassed("Move konstruktorius - saltinis istustintas");
00158 }
00159
00160 // TEST #6: Move assignment operatorius
00161 void test_move_assignment() {
00162     printTestHeader("Move Assignment Operatorius");
00163
00164     Studentas s1("Ruta", "Rute", 90);
00165     s1.addPaz(5);
00166     s1.addPaz(6);
00167     s1.addPaz(7);
00168
00169     Studentas s2;
00170     s2 = std::move(s1);
00171
00172     assert(s2.getVardas() == "Ruta");
00173     testPassed("Move assignment - vardas is naujo priskirtas");
00174
00175     assert(s2.getPaz().size() == 3);
00176     testPassed("Move assignment - pazymiai is naujo priskirti");
00177
00178     assert(s1.isPazEmpty());
00179     testPassed("Move assignment - saltinis istustintas");
00180
00181     // Self-move assignment check
00182     s2 = std::move(s2);
00183     assert(s2.getVardas() == "Ruta");
00184     testPassed("Move assignment self-assignment saugiai veikia");
00185 }
00186
00187 // TEST #7: Destruktorius - RAII principas
00188 void test_destructor() {
00189     printTestHeader("Destruktorius - RAII Principas");
00190
00191     {
00192         Studentas s("Vidmantas", "Vidmante", 87);
00193         s.addPaz(9);
00194         s.addPaz(10);
00195         s.addPaz(8);
00196
00197         assert(s.getPaz().size() == 3);
00198         testPassed("Destruktoriaus testas - objektas sukurtas");
00199     } // Cia is automatiškai kviečiamas destruktoriaus
00200
00201     testPassed("Destruktor veikia be problemu (nera memory leaks)");
00202 }
00203
00204 // TEST #8: Isvesties operatorius (operator<)
00205 void test_output_operator() {
00206     printTestHeader("Isvesties Operatorius (operator<)");
00207
00208     Studentas s("Darius", "Darius", 93);
00209     s.addPaz(9);
00210     s.addPaz(10);
00211     s.addPaz(9);
00212
00213     stringstream ss;
00214     ss << s;
00215     string output = ss.str();
00216
00217     assert(output.find("Vardas: Darius") != string::npos);
00218     testPassed("Isvestis - vardas rastas");
00219
00220     assert(output.find("Pavarde: Darius") != string::npos);
00221     testPassed("Isvestis - pavarde rastas");

```

```

00222
00223     assert(output.find("Egzaminas: 93") != string::npos);
00224     testPassed("Ivestis - egzaminas rastas");
00225
00226     cout << "Ivesties pavyzdys: " << endl;
00227     cout << " " << s << endl;
00228 }
00229
00230 // TEST #9: Ivesties operatorius (operator)
00231 void test_input_operator() {
00232     printTestHeader("Ivesties Operatorius (operator)");
00233
00234     stringstream ss;
00235     ss << "Vaidas Vaidauskas 10 10 10 10 10 100";
00236
00237     Studentas s;
00238     ss >> s;
00239
00240     assert(s.getVardas() == "Vaidas");
00241     testPassed("Ivestis - vardas nuskaitytas");
00242
00243     assert(s.getPavarde() == "Vaidauskas");
00244     testPassed("Ivestis - pavarde nuskaityta");
00245
00246     assert(s.getEgz() == 100);
00247     testPassed("Ivestis - egzaminas nuskaitytas");
00248
00249     assert(s.getPaz().size() == 5);
00250     testPassed("Ivestis - pazymiai nuskaityti");
00251
00252     assert(s.getPaz()[0] == 10);
00253     testPassed("Ivestis - pazymio reiksme teisingai nuskaityta");
00254 }
00255
00256 // TEST #10: I/O operatoriai kartu (Round-trip test)
00257 void test_io_roundtrip() {
00258     printTestHeader("I/O Round-Trip Testas");
00259
00260     Studentas s1("Kristina", "Kristina", 96);
00261     s1.addPaz(8);
00262     s1.addPaz(9);
00263     s1.addPaz(10);
00264     s1.addPaz(9);
00265     s1.addPaz(8);
00266
00267     stringstream ss;
00268     ss << s1.getVardas() << " " << s1.getPavarde();
00269     for (int paz : s1.getPaz()) {
00270         ss << " " << paz;
00271     }
00272     ss << " " << s1.getEgz();
00273
00274     Studentas s2;
00275     ss >> s2;
00276
00277     assert(s2.getVardas() == s1.getVardas());
00278     testPassed("Round-trip - vardas saugai issaugotas ir nuskaitytas");
00279
00280     assert(s2.getPavarde() == s1.getPavarde());
00281     testPassed("Round-trip - pavarde saugai issaugotas ir nuskaitytas");
00282
00283     assert(s2.getPaz() == s1.getPaz());
00284     testPassed("Round-trip - pazymiai saugai issaugoti ir nuskaityti");
00285 }
00286
00287 // TEST #11: Modifikavimas po kopijavimo
00288 void test_modification_after_copy() {
00289     printTestHeader("Modifikavimas po kopijavimo");
00290
00291     Studentas s1("Elena", "Elene", 85);
00292     s1.addPaz(7);
00293     s1.addPaz(8);
00294
00295     Studentas s2 = s1;
00296
00297     s2.setEgz(95);
00298     assert(s1.getEgz() == 85 && s2.getEgz() == 95);
00299     testPassed("Egzamino modifikacija - originalas nepaveiktas");
00300
00301     s2.addPaz(9);
00302     assert(s1.getPaz().size() == 2 && s2.getPaz().size() == 3);
00303     testPassed("Pazymio pridejimas - originalas nepaveiktas");
00304 }
00305
00306 // TEST #12: Getter/Setter operaciju konsekvencija
00307 void test_getters_setters() {
00308     printTestHeader("Getters/Setters Konsekvencija");

```

```

00309
00310     Studentas s;
00311
00312     s.setVardas("Tomas");
00313     assert(s.getVardas() == "Tomas");
00314     testPassed("Vardas setter/getter veikia");
00315
00316     s.setPavarde("Tomas");
00317     assert(s.getPavarde() == "Tomas");
00318     testPassed("Pavarde setter/getter veikia");
00319
00320     s.setEgz(100);
00321     assert(s.getEgz() == 100);
00322     testPassed("Egzaminas setter/getter veikia");
00323
00324     s.setRez(90.5);
00325     assert(s.getRez() == 90.5);
00326     testPassed("Rezultatas setter/getter veikia");
00327
00328     s.setMed(88.0);
00329     assert(s.getMed() == 88.0);
00330     testPassed("Mediana setter/getter veikia");
00331 }
00332
00333 // TEST #13: Vector operacijos su pazymiais
00334 void test_paz_operations() {
00335     printTestHeader("Vector Operacijos su Pazymiais");
00336
00337     Studentas s;
00338
00339     assert(s.isPazEmpty());
00340     testPassed("isPazEmpty() grazina true tusciam sarasui");
00341
00342     s.addPaz(10);
00343     s.addPaz(9);
00344     assert(!s.isPazEmpty());
00345     testPassed("isPazEmpty() grazina false ne tusciam sarasui");
00346
00347     assert(s.getPaz().size() == 2);
00348     testPassed("Pazymiai pridedami teisingai");
00349
00350     s.clearPaz();
00351     assert(s.isPazEmpty());
00352     testPassed("clearPaz() isvalo pazymius");
00353 }
00354
00355 // ===== PAPILDOMI GOOGLE TEST TESTAI (v2.0) =====
00356
00357 // TEST #14: Vektoriaus su Studentais operacijos
00358 TEST(StudentasVectorTest, PushBackCopySemantics) {
00359     vector<Studentas> students;
00360
00361     Studentas s1("Audra", "Audre", 91);
00362     s1.addPaz(9);
00363     s1.addPaz(8);
00364
00365     students.push_back(s1); // Copy semantika
00366
00367     EXPECT_EQ(students.size(), 1);
00368     EXPECT_EQ(students[0].getVardas(), "Audra");
00369     EXPECT_EQ(students[0].getPaz().size(), 2);
00370 }
00371
00372 // TEST #15: Move semantika su vektoriumi
00373 TEST(StudentasVectorTest, PushBackMoveSemantics) {
00374     vector<Studentas> students;
00375
00376     Studentas s1("Giedra", "Giedre", 88);
00377     s1.addPaz(8);
00378
00379     students.push_back(std::move(s1)); // Move semantika
00380
00381     EXPECT_EQ(students.size(), 1);
00382     EXPECT_TRUE(s1.isPazEmpty()); // Originalas turi būti tuščias
00383     EXPECT_EQ(students[0].getVardas(), "Giedra");
00384 }
00385
00386 // TEST #16: Deep copy - independenti duplikacija
00387 TEST(StudentasDeepCopyTest, IndependentCopies) {
00388     Studentas original("Laimas", "Laimas", 92);
00389     original.addPaz(9);
00390     original.addPaz(10);
00391
00392     Studentas copy1 = original;
00393     Studentas copy2 = original;
00394
00395     // Modifikuojame copy1

```

```

00396     copy1.setEgz(100);
00397     copy1.addPaz(8);
00398
00399     // Patikrinti, kad originalas ir copy2 nepakeisti
00400     EXPECT_EQ(original.getEgz(), 92);
00401     EXPECT_EQ(original.getPaz().size(), 2);
00402     EXPECT_EQ(copy2.getEgz(), 92);
00403     EXPECT_EQ(copy2.getPaz().size(), 2);
00404     EXPECT_EQ(copy1.getEgz(), 100);
00405     EXPECT_EQ(copy1.getPaz().size(), 3);
00406 }
00407
00408 // TEST #17: Self-assignment saugumas
00409 TEST(StudentasSelfAssignmentTest, CopySafetyCheck) {
00410     Studentas s("Monika", "Monika", 95);
00411     s.addPaz(10);
00412     s.addPaz(9);
00413
00414     // Self-assignment - turi būti saugus
00415     s = s;
00416
00417     EXPECT_EQ(s.getVardas(), "Monika");
00418     EXPECT_EQ(s.getEgz(), 95);
00419     EXPECT_EQ(s.getPaz().size(), 2);
00420 }
00421
00422 // TEST #18: Operatoriaus lankstumo - chaining
00423 TEST(StudentasOperatorTest, AssignmentChaining) {
00424     Studentas s1("Vincas", "Vincas", 92);
00425     Studentas s2("Petras", "Petrauskas", 88);
00426     Studentas s3;
00427
00428     // Assignment chaining: (s3 = s2) = s1;
00429     // Step 1: s3 = s2 grąžina s3 referencę, s3 becomes "Petras"
00430     // Step 2: (s3 referencija) = s1 priskyrimo s1 reikšmę s3
00431     (s3 = s2) = s1;
00432
00433     EXPECT_EQ(s3.getVardas(), "Vincas"); // s3 final value is Vincas
00434     EXPECT_EQ(s2.getVardas(), "Petras"); // s2 remains Petras
00435     EXPECT_EQ(s1.getVardas(), "Vincas"); // s1 remains Vincas
00436 }
00437
00438 // ===== ORIGINAL MAIN (su backward compatibility) =====
00439 int main(int argc, char** argv) {
00440     // Google Test inicializacija
00441     ::testing::InitGoogleTest(&argc, argv);
00442
00443     // Originalūs testai (v1.5)
00444     cout << "\n" << string(60, '*') << endl;
00445     cout << "STUDENTAS KLASĖS VISOS METODŲ TESTAI (v1.5)" << endl;
00446     cout << " + GOOGLE TEST PAPILDYMAS (v2.0)" << endl;
00447     cout << string(60, '*') << endl;
00448
00449     try {
00450         test_default_constructor();
00451         test_parametrized_constructor();
00452         test_copy_constructor();
00453         test_copy_assignment();
00454         test_move_constructor();
00455         test_move_assignment();
00456         test_destructor();
00457         test_output_operator();
00458         test_input_operator();
00459         test_io_roundtrip();
00460         test_modification_after_copy();
00461         test_getters_setters();
00462         test_paz_operations();
00463
00464         cout << "\n" << string(60, '*') << endl;
00465         cout << "TESTO REZULTATAI:" << endl;
00466         cout << "Priimti testai: " << testai_praletii << endl;
00467         cout << "Nepriimti testai: " << testai_nepraletii << endl;
00468         cout << string(60, '*') << endl;
00469
00470         if (testai_nepraletii == 0) {
00471             cout << "\nVISI TESTAI PRIIMTI!" << endl;
00472         } else {
00473             cout << "\nKai kurie testai nepraejo!" << endl;
00474             return 1;
00475         }
00476     }
00477     catch (const std::exception& e) {
00478         cout << "Kritinė klaida: " << e.what() << endl;
00479         return 1;
00480     }
00481
00482     cout << "\n" << string(60, '*') << endl;

```

```

00483     cout << "Vykdome GOOGLE TEST testai..." << endl;
00484     cout << string(60, '*') << "\n" << endl;
00485
00486     // Vykdyti Google Test testus
00487     return RUN_ALL_TESTS();
00488 }
00489
00490 // =====
00491 // VECTOR GTEST TESTAI
00492 // =====
00493
00494 class VectorTest : public ::testing::Test {
00495 protected:
00496     Vector<int> v;
00497 };
00498
00499 // Test 1: Default konstruktorius
00500 TEST_F(VectorTest, DefaultConstructor) {
00501     EXPECT_EQ(v.size(), 0);
00502     EXPECT_EQ(v.capacity(), 0);
00503     EXPECT_TRUE(v.empty());
00504 }
00505
00506 // Test 2: Parametrizuotas konstruktorius su size
00507 TEST_F(VectorTest, ConstructorWithCapacity) {
00508     Vector<int> v2(10);
00509     EXPECT_EQ(v2.size(), 0);
00510     EXPECT_EQ(v2.capacity(), 10);
00511 }
00512
00513 // Test 3: Parametrizuotas konstruktorius su size ir value
00514 TEST_F(VectorTest, ConstructorWithSizeAndValue) {
00515     Vector<int> v2(5, 42);
00516     EXPECT_EQ(v2.size(), 5);
00517     EXPECT_EQ(v2.capacity(), 5);
00518     EXPECT_EQ(v2[0], 42);
00519     EXPECT_EQ(v2[4], 42);
00520 }
00521
00522 // Test 4: push_back()
00523 TEST_F(VectorTest, PushBack) {
00524     v.push_back(10);
00525     v.push_back(20);
00526     v.push_back(30);
00527     EXPECT_EQ(v.size(), 3);
00528     EXPECT_EQ(v[0], 10);
00529     EXPECT_EQ(v[1], 20);
00530     EXPECT_EQ(v[2], 30);
00531 }
00532
00533 // Test 5: front() ir back()
00534 TEST_F(VectorTest, FrontAndBack) {
00535     v.push_back(10);
00536     v.push_back(20);
00537     v.push_back(30);
00538     EXPECT_EQ(v.front(), 10);
00539     EXPECT_EQ(v.back(), 30);
00540 }
00541
00542 // Test 6: pop_back()
00543 TEST_F(VectorTest, PopBack) {
00544     v.push_back(10);
00545     v.push_back(20);
00546     v.push_back(30);
00547     v.pop_back();
00548     EXPECT_EQ(v.size(), 2);
00549     EXPECT_EQ(v.back(), 20);
00550 }
00551
00552 // Test 7: at() su bounds checking
00553 TEST_F(VectorTest, AtWithBoundsChecking) {
00554     v.push_back(10);
00555     v.push_back(20);
00556     EXPECT_EQ(v.at(0), 10);
00557     EXPECT_EQ(v.at(1), 20);
00558     EXPECT_THROW(v.at(2), std::out_of_range);
00559 }
00560
00561 // Test 8: operator[]
00562 TEST_F(VectorTest, OperatorBrackets) {
00563     v.push_back(10);
00564     v.push_back(20);
00565     v[0] = 100;
00566     v[1] = 200;
00567     EXPECT_EQ(v[0], 100);
00568     EXPECT_EQ(v[1], 200);
00569 }

```

```
00570
00571 // Test 9: clear()
00572 TEST_F(VectorTest, Clear) {
00573     v.push_back(10);
00574     v.push_back(20);
00575     v.clear();
00576     EXPECT_EQ(v.size(), 0);
00577     EXPECT_TRUE(v.empty());
00578 }
00579
00580 // Test 10: reserve()
00581 TEST_F(VectorTest, Reserve) {
00582     v.reserve(100);
00583     EXPECT_GE(v.capacity(), 100);
00584     EXPECT_EQ(v.size(), 0);
00585 }
00586
00587 // Test 11: shrink_to_fit()
00588 TEST_F(VectorTest, ShrinkToFit) {
00589     v.push_back(10);
00590     v.push_back(20);
00591     v.push_back(30);
00592     v.shrink_to_fit();
00593     EXPECT_EQ(v.capacity(), v.size());
00594 }
00595
00596 // Test 12: resize()
00597 TEST_F(VectorTest, Resize) {
00598     v.push_back(10);
00599     v.push_back(20);
00600     v.resize(5);
00601     EXPECT_EQ(v.size(), 5);
00602
00603     v.resize(2);
00604     EXPECT_EQ(v.size(), 2);
00605 }
00606
00607 // Test 13: resize() su value
00608 TEST_F(VectorTest, ResizeWithValue) {
00609     v.resize(3, 42);
00610     EXPECT_EQ(v.size(), 3);
00611     EXPECT_EQ(v[0], 42);
00612     EXPECT_EQ(v[1], 42);
00613     EXPECT_EQ(v[2], 42);
00614 }
00615
00616 // Test 14: swap()
00617 TEST_F(VectorTest, Swap) {
00618     Vector<int> v2;
00619     v.push_back(10);
00620     v.push_back(20);
00621     v2.push_back(100);
00622
00623     v.swap(v2);
00624     EXPECT_EQ(v.size(), 1);
00625     EXPECT_EQ(v[0], 100);
00626     EXPECT_EQ(v2.size(), 2);
00627     EXPECT_EQ(v2[0], 10);
00628 }
00629
00630 // Test 15: operator== ir operator!=
00631 TEST_F(VectorTest, ComparisonOperators) {
00632     Vector<int> v2;
00633     v.push_back(10);
00634     v.push_back(20);
00635     v2.push_back(10);
00636     v2.push_back(20);
00637
00638     EXPECT_TRUE(v == v2);
00639     EXPECT_FALSE(v != v2);
00640
00641     v2.push_back(30);
00642     EXPECT_FALSE(v == v2);
00643     EXPECT_TRUE(v != v2);
00644 }
00645
00646 // Test 16: operator< ir operator>
00647 TEST_F(VectorTest, LessGreaterOperators) {
00648     Vector<int> v2;
00649     v.push_back(10);
00650     v.push_back(20);
00651     v2.push_back(10);
00652     v2.push_back(30);
00653
00654     EXPECT_TRUE(v < v2);
00655     EXPECT_TRUE(v2 > v);
00656     EXPECT_FALSE(v > v2);
```

```

00657 }
00658
00659 // Test 17: begin() ir end() iteratoriai
00660 TEST_F(VectorTest, BeginEndIterators) {
00661     v.push_back(10);
00662     v.push_back(20);
00663     v.push_back(30);
00664
00665     int sum = 0;
00666     for (auto it = v.begin(); it != v.end(); ++it) {
00667         sum += *it;
00668     }
00669     EXPECT_EQ(sum, 60);
00670 }
00671
00672 // Test 18: assign()
00673 TEST_F(VectorTest, Assign) {
00674     v.assign(5, 42);
00675     EXPECT_EQ(v.size(), 5);
00676     for (size_t i = 0; i < v.size(); ++i) {
00677         EXPECT_EQ(v[i], 42);
00678     }
00679 }
00680
00681 // Test 19: insert()
00682 TEST_F(VectorTest, Insert) {
00683     v.push_back(10);
00684     v.push_back(30);
00685     v.insert(v.begin() + 1, 20);
00686
00687     EXPECT_EQ(v.size(), 3);
00688     EXPECT_EQ(v[0], 10);
00689     EXPECT_EQ(v[1], 20);
00690     EXPECT_EQ(v[2], 30);
00691 }
00692
00693 // Test 20: erase()
00694 TEST_F(VectorTest, Erase) {
00695     v.push_back(10);
00696     v.push_back(20);
00697     v.push_back(30);
00698
00699     v.erase(v.begin() + 1);
00700
00701     EXPECT_EQ(v.size(), 2);
00702     EXPECT_EQ(v[0], 10);
00703     EXPECT_EQ(v[1], 30);
00704 }
00705
00706 // Test 21: emplace_back()
00707 TEST_F(VectorTest, EmplaceBack) {
00708     v.emplace_back(42);
00709     v.emplace_back(99);
00710
00711     EXPECT_EQ(v.size(), 2);
00712     EXPECT_EQ(v[0], 42);
00713     EXPECT_EQ(v[1], 99);
00714 }
00715
00716 // Test 22: Copy konstruktorius
00717 TEST_F(VectorTest, CopyConstructor) {
00718     v.push_back(10);
00719     v.push_back(20);
00720
00721     Vector<int> v2 = v;
00722     EXPECT_EQ(v2.size(), 2);
00723     EXPECT_EQ(v2[0], 10);
00724
00725     v2[0] = 100;
00726     EXPECT_EQ(v[0], 10); // Original nepakeistas
00727 }
00728
00729 // Test 23: Move konstruktorius
00730 TEST_F(VectorTest, MoveConstructor) {
00731     v.push_back(10);
00732     v.push_back(20);
00733
00734     Vector<int> v2 = std::move(v);
00735     EXPECT_EQ(v2.size(), 2);
00736     EXPECT_EQ(v.size(), 0); // Original tuščias
00737 }
00738
00739 // Test 24: Copy assignment
00740 TEST_F(VectorTest, CopyAssignment) {
00741     v.push_back(10);
00742     v.push_back(20);
00743

```



```
00744     Vector<int> v2;
00745     v2 = v;
00746     EXPECT_EQ(v2.size(), 2);
00747     v2[0] = 100;
00748     EXPECT_EQ(v[0], 10); // Original nepakeistas
00749 }
00750
00751 // Test 25: Move assignment
00752 TEST_F(VectorTest, MoveAssignment) {
00753     v.push_back(10);
00754     v.push_back(20);
00755
00756     Vector<int> v2;
00757     v2 = std::move(v);
00758     EXPECT_EQ(v2.size(), 2);
00759     EXPECT_EQ(v.size(), 0); // Original tuščias
00760 }
00761
00762 // Test 26: max_size()
00763 TEST_F(VectorTest, MaxSize) {
00764     EXPECT_GT(v.max_size(), 0);
00765 }
00766
00767 // Test 27: data()
00768 TEST_F(VectorTest, Data) {
00769     v.push_back(10);
00770     v.push_back(20);
00771
00772     int* ptr = v.data();
00773     EXPECT_EQ(ptr[0], 10);
00774     EXPECT_EQ(ptr[1], 20);
00775 }
00776
```


skyrius 7

Pavyzdžiai

7.1 C:/Users/ditas/source/repos/ditassimoliunas-prog/objektuzdv3/↵ header_files/Vector.h

Add element to the end of the vector (copy semantics).

Add element to the end of the vector (copy semantics) Appends a copy of the given value to the end. If the current size equals capacity, the vector doubles its capacity and reallocates.

Time Complexity: $O(1)$ amortized

Parametrai

<i>value</i>	The value to append (copied)
--------------	------------------------------

Išimtys

<i>std::bad_alloc</i>	if memory allocation fails
-----------------------	----------------------------

Taip pat žiūrėti

`push_back(T&&)`, `pop_back()`, `emplace_back()`

```
Vector<int> v;
v.push_back(42);
v.push_back(100);

#pragma once

#include <iostream>
#include <stdexcept>
#include <utility>
#include <algorithm>
#include <limits>
#include <vector>

/**
 * @brief Random access iterator for Vector<T> container
 *
 * VectorIterator provides bidirectional and random access to Vector elements.
 * Supports all standard iterator operations including comparison, arithmetic,
 * and dereferencing.
 *
 * @tparam T The element type that the iterator points to
 *
 * @see Vector
 */
template <typename T>
class VectorIterator {
public:
    using iterator_category = std::random_access_iterator_tag;
    using value_type = T;
    using difference_type = std::ptrdiff_t;
    using pointer = T*;
```

```

    using reference = T&;

private:
    T* _ptr;

public:
    VectorIterator(T* ptr = nullptr) : _ptr(ptr) {}

    reference operator*() const { return *_ptr; }
    pointer operator->() const { return _ptr; }

    VectorIterator& operator++() {
        ++_ptr;
        return *this;
    }
    VectorIterator operator++(int) {
        VectorIterator temp = *this;
        ++_ptr;
        return temp;
    }

    VectorIterator& operator--() {
        --_ptr;
        return *this;
    }
    VectorIterator operator--(int) {
        VectorIterator temp = *this;
        --_ptr;
        return temp;
    }

    VectorIterator operator+(difference_type n) const {
        return VectorIterator(_ptr + n);
    }
    VectorIterator operator-(difference_type n) const {
        return VectorIterator(_ptr - n);
    }

    VectorIterator& operator+=(difference_type n) {
        _ptr += n;
        return *this;
    }
    VectorIterator& operator-=(difference_type n) {
        _ptr -= n;
        return *this;
    }

    reference operator[](difference_type n) const {
        return _ptr[n];
    }

    bool operator==(const VectorIterator& other) const { return _ptr == other._ptr; }
    bool operator!=(const VectorIterator& other) const { return _ptr != other._ptr; }
    bool operator<(const VectorIterator& other) const { return _ptr < other._ptr; }
    bool operator>(const VectorIterator& other) const { return _ptr > other._ptr; }
    bool operator<=(const VectorIterator& other) const { return _ptr <= other._ptr; }
    bool operator>=(const VectorIterator& other) const { return _ptr >= other._ptr; }

    difference_type operator-(const VectorIterator& other) const {
        return _ptr - other._ptr;
    }
};

/**
 * @class Vector
 * @brief Custom dynamic array container template
 *
 * Vector<T> is a generic container that stores elements of type T in a
 * dynamically allocated array. It provides O(1) amortized time complexity
 * for push_back(), while maintaining contiguous memory layout like std::vector.
 *
 * The class implements the Rule of Five pattern and fully supports deep copy
 * and move semantics.
 *
 * **Key Features:**
 * - Dynamic memory management (automatic resizing)
 * - Random access iterators (begin(), end(), rbegin(), rend())
 * - Copy and move semantics (Rule of Five)
 * - Standard container operations (push_back, pop_back, insert, erase, etc.)
 * - Comparable performance to std::vector for most operations
 *
 * **Memory Strategy:**
 * - When capacity is exhausted, reallocates with roughly 1.5x growth factor
 * - Tracks reallocation count for performance analysis
 * - Supports manual reserve() for optimization
 *
 * **Example Usage:**

```

```

* @code
* Vector<int> v;
* v.push_back(10);
* v.push_back(20);
* v.reserve(100);          // Pre-allocate capacity
*
* for (int elem : v) {
*     std::cout << elem << " ";
* }
* @endcode
*
* @tparam T Element type (must support copy and move semantics)
*
* @see VectorIterator
* @see std::vector
*/
template <typename T>
class Vector {
private:
    T* _data;
    size_t _size;
    size_t _capacity;
    size_t _realloc_count;

    void reallocate(size_t new_capacity) {
        if (new_capacity < _size) {
            throw std::invalid_argument("New capacity must be >= current size");
        }
        T* new_data = new T[new_capacity];
        for (size_t i = 0; i < _size; ++i) {
            new_data[i] = _data[i];
        }
        delete[] _data;
        _data = new_data;
        _capacity = new_capacity;
        ++_realloc_count;
    }

public:
    Vector() : _data(nullptr), _size(0), _capacity(0), _realloc_count(0) {}
    explicit Vector(size_t capacity) : _size(0), _capacity(capacity), _realloc_count(0) {
        _data = capacity > 0 ? new T[capacity] : nullptr;
    }
    Vector(size_t size, const T& value) : _size(size), _capacity(size), _realloc_count(0) {
        _data = size > 0 ? new T[size] : nullptr;
        for (size_t i = 0; i < size; ++i) _data[i] = value;
    }

    // Conversion constructor from std::vector
    Vector(const std::vector<T>& other) : _size(other.size()), _capacity(other.size()), _realloc_count(0) {
        _data = _size > 0 ? new T[_size] : nullptr;
        for (size_t i = 0; i < _size; ++i) _data[i] = other[i];
    }

    ~Vector() { delete[] _data; _data = nullptr; _size = 0; _capacity = 0; }

    Vector(const Vector& other) : _size(other._size), _capacity(other._capacity), _realloc_count(0) {
        _data = _capacity > 0 ? new T[_capacity] : nullptr;
        for (size_t i = 0; i < _size; ++i) _data[i] = other._data[i];
    }
    Vector& operator=(const Vector& other) {
        if (this != &other) {
            delete[] _data;
            _size = other._size;
            _capacity = other._capacity;
            _realloc_count = 0;
            _data = _capacity > 0 ? new T[_capacity] : nullptr;
            for (size_t i = 0; i < _size; ++i) _data[i] = other._data[i];
        }
        return *this;
    }

    Vector(Vector&& other) noexcept : _data(other._data), _size(other._size),
        _capacity(other._capacity), _realloc_count(other._realloc_count) {
        other._data = nullptr;
        other._size = 0;
        other._capacity = 0;
        other._realloc_count = 0;
    }
    Vector& operator=(Vector&& other) noexcept {
        if (this != &other) {
            delete[] _data;
            _data = other._data;
            _size = other._size;
            _capacity = other._capacity;
            _realloc_count = other._realloc_count;
            other._data = nullptr;
        }
    }

```

```

        other._size = 0;
        other._capacity = 0;
        other._realloc_count = 0;
    }
    return *this;
}

inline size_t size() const { return _size; }
inline size_t capacity() const { return _capacity; }
inline bool empty() const { return _size == 0; }

inline T& operator[](size_t index) { return _data[index]; }
inline const T& operator[](size_t index) const { return _data[index]; }

T& at(size_t index) {
    if (index >= _size) {
        throw std::out_of_range("Vector index out of range");
    }
    return _data[index];
}

const T& at(size_t index) const {
    if (index >= _size) {
        throw std::out_of_range("Vector index out of range");
    }
    return _data[index];
}

void clear() {
    _size = 0;
}

T& front() {
    if (_size == 0) {
        throw std::out_of_range("Vector is empty: cannot access front()");
    }
    return _data[0];
}

const T& front() const {
    if (_size == 0) {
        throw std::out_of_range("Vector is empty: cannot access front()");
    }
    return _data[0];
}

T& back() {
    if (_size == 0) {
        throw std::out_of_range("Vector is empty: cannot access back()");
    }
    return _data[_size - 1];
}

const T& back() const {
    if (_size == 0) {
        throw std::out_of_range("Vector is empty: cannot access back()");
    }
    return _data[_size - 1];
}

void reserve(size_t new_capacity) {
    if (new_capacity > _capacity) {
        reallocate(new_capacity);
    }
}

void shrink_to_fit() {
    if (_capacity > _size) {
        if (_size == 0) {
            delete[] _data;
            _data = nullptr;
            _capacity = 0;
        } else {
            reallocate(_size);
        }
    }
}

void resize(size_t new_size) {
    if (new_size > _capacity) {
        reallocate(new_size);
    }
    _size = new_size;
}

void resize(size_t new_size, const T& value) {
    if (new_size > _capacity) {

```

```

        reallocate(new_size);
    }
    if (new_size > _size) {
        for (size_t i = _size; i < new_size; ++i) {
            _data[i] = value;
        }
    }
    _size = new_size;
}

void swap(Vector& other) noexcept {
    std::swap(_data, other._data);
    std::swap(_size, other._size);
    std::swap(_capacity, other._capacity);
    std::swap(_realloc_count, other._realloc_count);
}

bool operator==(const Vector& other) const {
    if (_size != other._size) return false;
    for (size_t i = 0; i < _size; ++i) {
        if (_data[i] != other._data[i]) return false;
    }
    return true;
}

bool operator!=(const Vector& other) const {
    return !(*this == other);
}

bool operator<(const Vector& other) const {
    for (size_t i = 0; i < _size && i < other._size; ++i) {
        if (_data[i] < other._data[i]) return true;
        if (_data[i] > other._data[i]) return false;
    }
    return _size < other._size;
}

bool operator>(const Vector& other) const {
    return other < *this;
}

bool operator<=(const Vector& other) const {
    return !(other < *this);
}

bool operator>=(const Vector& other) const {
    return !(*this < other);
}

using iterator = VectorIterator<T>;
using const_iterator = VectorIterator<const T>;

iterator begin() { return iterator(_data); }
iterator end() { return iterator(_data + _size); }

const_iterator begin() const { return const_iterator(_data); }
const_iterator end() const { return const_iterator(_data + _size); }

const_iterator cbegin() const { return const_iterator(_data); }
const_iterator cend() const { return const_iterator(_data + _size); }

using reverse_iterator = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;

reverse_iterator rbegin() { return reverse_iterator(end()); }
reverse_iterator rend() { return reverse_iterator(begin()); }

const_reverse_iterator rbegin() const { return const_reverse_iterator(end()); }
const_reverse_iterator rend() const { return const_reverse_iterator(begin()); }

const_reverse_iterator crbegin() const { return const_reverse_iterator(end()); }
const_reverse_iterator crend() const { return const_reverse_iterator(begin()); }

void assign(size_t count, const T& value) {
    clear();
    if (count > _capacity) {
        reallocate(count);
    }
    for (size_t i = 0; i < count; ++i) {
        _data[i] = value;
    }
    _size = count;
}

template<typename InputIt>
void assign(InputIt first, InputIt last) {
    // This is a specialized version - only works with Vector iterators

```

```

    // For general case, user should use clear() + for loop
}

iterator insert(const_iterator pos, const T& value) {
    size_t index = std::distance(cbegin(), pos);
    if (index > _size) {
        throw std::out_of_range("Iterator out of range");
    }

    if (_size >= _capacity) {
        size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
        reallocate(new_capacity);
    }

    for (size_t i = _size; i > index; --i) {
        _data[i] = _data[i - 1];
    }
    _data[index] = value;
    ++_size;

    return iterator(_data + index);
}

// Non-const overload for insert (accepts non-const iterator, treats as const_iterator)
iterator insert(iterator pos, const T& value) {
    // pos is a mutable iterator, but we just use it as a position
    // Convert to size_t using cbegin()
    size_t index = std::distance(begin(), pos);
    if (index > _size) {
        throw std::out_of_range("Iterator out of range");
    }

    if (_size >= _capacity) {
        size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
        reallocate(new_capacity);
    }

    for (size_t i = _size; i > index; --i) {
        _data[i] = _data[i - 1];
    }
    _data[index] = value;
    ++_size;

    return iterator(_data + index);
}

// Range insert overload with three arguments
template<typename InputIt>
iterator insert(const_iterator pos, InputIt first, InputIt last) {
    size_t index = std::distance(cbegin(), pos);
    if (index > _size) {
        throw std::out_of_range("Iterator out of range");
    }

    // Count elements manually
    size_t count = 0;
    for (InputIt it = first; it != last; ++it) {
        ++count;
    }

    if (_size + count > _capacity) {
        reallocate(std::max(_size + count, _capacity * 2));
    }

    // Shift elements to the right
    for (size_t i = _size; i > index; --i) {
        _data[i + count - 1] = _data[i - 1];
    }

    // Insert new elements
    size_t i = index;
    for (InputIt it = first; it != last; ++it, ++i) {
        _data[i] = *it;
    }
    _size += count;

    return iterator(_data + index);
}

// Range insert overload with non-const iterator
template<typename InputIt>
iterator insert(iterator pos, InputIt first, InputIt last) {
    size_t index = std::distance(begin(), pos);
    if (index > _size) {
        throw std::out_of_range("Iterator out of range");
    }

```



```

    }

    // Count elements manually
    size_t count = 0;
    for (InputIt it = first; it != last; ++it) {
        ++count;
    }

    if (_size + count > _capacity) {
        reallocate(std::max(_size + count, _capacity * 2));
    }

    // Shift elements to the right
    for (size_t i = _size; i > index; --i) {
        _data[i + count - 1] = _data[i - 1];
    }

    // Insert new elements
    size_t i = index;
    for (InputIt it = first; it != last; ++it, ++i) {
        _data[i] = *it;
    }
    _size += count;

    return iterator(_data + index);
}

iterator erase(const_iterator pos) {
    size_t index = std::distance(cbegin(), pos);
    if (index >= _size) {
        throw std::out_of_range("Iterator out of range");
    }

    for (size_t i = index; i < _size - 1; ++i) {
        _data[i] = _data[i + 1];
    }
    --_size;

    return iterator(_data + index);
}

iterator erase(const_iterator first, const_iterator last) {
    size_t first_index = std::distance(cbegin(), first);
    size_t last_index = std::distance(cbegin(), last);

    if (first_index > _size || last_index > _size || first_index > last_index) {
        throw std::out_of_range("Invalid range");
    }

    size_t count = last_index - first_index;
    for (size_t i = first_index; i < _size - count; ++i) {
        _data[i] = _data[i + count];
    }
    _size -= count;

    return iterator(_data + first_index);
}

// Non-const overload for erase (single element)
iterator erase(iterator pos) {
    size_t index = std::distance(begin(), pos);
    if (index >= _size) {
        throw std::out_of_range("Iterator out of range");
    }

    for (size_t i = index; i < _size - 1; ++i) {
        _data[i] = _data[i + 1];
    }
    --_size;

    return iterator(_data + index);
}

// Non-const overload for erase (range)
iterator erase(iterator first, iterator last) {
    size_t first_index = std::distance(begin(), first);
    size_t last_index = std::distance(begin(), last);

    if (first_index > _size || last_index > _size || first_index > last_index) {
        throw std::out_of_range("Invalid range");
    }

    size_t count = last_index - first_index;
    for (size_t i = first_index; i < _size - count; ++i) {
        _data[i] = _data[i + count];
    }
}

```

```

        _size -= count;

        return iterator(_data + first_index);
    }

template<typename... Args>
void emplace_back(Args&&... args) {
    if (_size >= _capacity) {
        size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
        reallocate(new_capacity);
    }
    new (_data + _size) T(std::forward<Args>(args)...);
    ++_size;
}

template<typename... Args>
iterator emplace(const_iterator pos, Args&&... args) {
    size_t index = std::distance(cbegin(), pos);
    if (index > _size) {
        throw std::out_of_range("Iterator out of range");
    }

    if (_size >= _capacity) {
        size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
        reallocate(new_capacity);
    }

    for (size_t i = _size; i > index; --i) {
        _data[i] = _data[i - 1];
    }
    new (_data + index) T(std::forward<Args>(args)...);
    ++_size;

    return iterator(_data + index);
}

size_t max_size() const {
    return std::numeric_limits<size_t>::max() / sizeof(T);
}

inline T* data() { return _data; }
inline const T* data() const { return _data; }

inline size_t get_reallocation_count() const { return _realloc_count; }
inline void reset_reallocation_count() { _realloc_count = 0; }

/**
 * @brief Add element to the end of the vector (copy semantics)
 *
 * Appends a copy of the given value to the end. If the current size equals
 * capacity, the vector doubles its capacity and reallocates.
 *
 * **Time Complexity:** O(1) amortized
 *
 * @param value The value to append (copied)
 *
 * @throws std::bad_alloc if memory allocation fails
 *
 * @see push_back(T&&), pop_back(), emplace_back()
 *
 * @example
 * @code
 * Vector<int> v;
 * v.push_back(42);
 * v.push_back(100);
 * @endcode
 */
void push_back(const T& value) {
    if (_size >= _capacity) {
        size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
        reallocate(new_capacity);
    }
    _data[_size] = value;
    ++_size;
}

/**
 * @brief Add element to the end of the vector (move semantics)
 *
 * Appends an rvalue reference to the vector. The element is moved
 * (not copied) into the vector. This is more efficient for expensive-to-copy types.
 *
 * **Time Complexity:** O(1) amortized
 *
 * @param value The rvalue reference to append (moved)
 *
 * @throws std::bad_alloc if memory allocation fails

```

```

*
* @see push_back(const T&), pop_back(), emplace_back()
*
* @example
* @code
* Vector<std::string> v;
* v.push_back(std::string("Hello")); // Moves the temporary
* @endcode
*/
void push_back(T&& value) {
    if (_size >= _capacity) {
        size_t new_capacity = (_capacity == 0) ? 1 : _capacity * 2;
        reallocate(new_capacity);
    }
    _data[_size] = std::move(value);
    ++_size;
}

/**
* @brief Remove the last element from the vector
*
* Decrements the size by one, effectively removing the last element.
* Does nothing if the vector is empty. Note: the element destructor
* is NOT called (similar to std::vector::pop_back for POD types).
*
* **Time Complexity:** O(1) constant
*
* **Precondition:** Empty vector is safe to call on (no-op)
*
* @see push_back(), front(), back()
*
* @example
* @code
* Vector<int> v = {1, 2, 3};
* v.pop_back(); // Now size is 2, last element is 2
* @endcode
*/
void pop_back() {
    if (_size > 0) {
        --_size;
    }
}
};

```


Rodyklė

~Studentas
 Studentas, [25](#)

~Vector
 Vector< T >, [31](#)

~Zmogus
 Zmogus, [44](#)

addPaz
 Studentas, [26](#)

assign
 Vector< T >, [31](#)

at
 Vector< T >, [31](#), [32](#)

back
 Vector< T >, [32](#)

begin
 Vector< T >, [32](#)

capacity
 Vector< T >, [32](#)

cbegin
 Vector< T >, [32](#)

cend
 Vector< T >, [32](#)

clear
 Vector< T >, [32](#)

clearPaz
 Studentas, [26](#)

const_iterator
 Vector< T >, [30](#)

const_reverse_iterator
 Vector< T >, [30](#)

crbegin
 Vector< T >, [32](#)

crend
 Vector< T >, [33](#)

data
 Vector< T >, [33](#)

deque< T >, [20](#)

deque< T >::const_iterator, [19](#)

deque< T >::const_reverse_iterator, [19](#)

deque< T >::iterator, [21](#)

deque< T >::reverse_iterator, [23](#)

difference_type
 VectorIterator< T >, [39](#)

emplace
 Vector< T >, [33](#)

emplace_back
 Vector< T >, [33](#)

empty
 Vector< T >, [33](#)

end
 Vector< T >, [33](#)

erase
 Vector< T >, [33](#), [34](#)

exception, [21](#)

extra_cpp/duomenu_valdymas.cpp, [47](#)

extra_cpp/input.cpp, [52](#)

extra_cpp/mat_funkcijos.cpp, [53](#)

extra_cpp/menu.cpp, [54](#)

extra_cpp/output.cpp, [56](#)

extra_cpp/Studentas.cpp, [57](#)

extra_cpp/testavimas.cpp, [58](#)

extra_cpp/Zmogus.cpp, [63](#)

front
 Vector< T >, [34](#)

get_reallocation_count
 Vector< T >, [34](#)

getEgz
 Studentas, [26](#)

getInfo
 Studentas, [26](#)
 Zmogus, [44](#)

getMed
 Studentas, [26](#)

getPavarde
 Zmogus, [44](#)

getPaz
 Studentas, [26](#)

getRez
 Studentas, [26](#)

getVaras
 Zmogus, [44](#)

header_files/duomenu_valdymas.h, [64](#)

header_files/input.h, [64](#)

header_files/Mat_funkcijos.h, [64](#)

header_files/menu.h, [64](#)

header_files/output.h, [64](#)

header_files/Studentas.h, [65](#)

header_files/testavimas.h, [66](#)

header_files/Vector.h, [66](#)

header_files/Zmogus.h, [74](#)

ifstream, [21](#)

insert
 Vector< T >, 34, 35
 invalid_argument, 21
 isPazEmpty
 Studentas, 26
 istream, 21
 iterator
 Vector< T >, 30
 iterator_category
 VectorIterator< T >, 39

 list< T >, 22
 list< T >::const_iterator, 19
 list< T >::const_reverse_iterator, 20
 list< T >::iterator, 21
 list< T >::reverse_iterator, 23

 max_size
 Vector< T >, 35

 ofstream, 22
 operator!=
 Vector< T >, 35
 VectorIterator< T >, 40
 operator<
 Vector< T >, 35
 VectorIterator< T >, 41
 operator<<
 Studentas, 27
 Zmogus, 45
 operator<=
 Vector< T >, 35
 VectorIterator< T >, 41
 operator>
 Vector< T >, 36
 VectorIterator< T >, 42
 operator>>
 Studentas, 27
 operator>=
 Vector< T >, 36
 VectorIterator< T >, 42
 operator+
 VectorIterator< T >, 40
 operator++
 VectorIterator< T >, 40
 operator+=
 VectorIterator< T >, 40
 operator-
 VectorIterator< T >, 41
 operator->
 VectorIterator< T >, 41
 operator--
 VectorIterator< T >, 41
 operator-=
 VectorIterator< T >, 41
 operator=
 Studentas, 27
 Vector< T >, 35
 operator==
 Vector< T >, 36
 VectorIterator< T >, 41
 operator[]
 Vector< T >, 36
 VectorIterator< T >, 42
 operator*
 VectorIterator< T >, 40
 ostream, 22

 pavarde_
 Zmogus, 45
 pointer
 VectorIterator< T >, 39
 pop_back
 Vector< T >, 36
 push_back
 Vector< T >, 36

 rbegin
 Vector< T >, 37
 readStudent
 Studentas, 27
 reference
 VectorIterator< T >, 40
 rend
 Vector< T >, 37
 reserve
 Vector< T >, 37
 reservePaz
 Studentas, 27
 reset_reallocation_count
 Vector< T >, 37
 resize
 Vector< T >, 37
 reverse_iterator
 Vector< T >, 30
 runtime_error, 23

 setEgz
 Studentas, 27
 setMed
 Studentas, 27
 setPavarde
 Zmogus, 44
 setPaz
 Studentas, 27
 setRez
 Studentas, 27
 setVardas
 Zmogus, 45
 shrink_to_fit
 Vector< T >, 37
 size
 Vector< T >, 38
 string, 23
 string::const_iterator, 19
 string::const_reverse_iterator, 20
 string::iterator, 22
 string::reverse_iterator, 23

- stringstream, [24](#)
- Studentas, [24](#)
 - ~Studentas, [25](#)
 - addPaz, [26](#)
 - clearPaz, [26](#)
 - getEgz, [26](#)
 - getInfo, [26](#)
 - getMed, [26](#)
 - getPaz, [26](#)
 - getRez, [26](#)
 - isPazEmpty, [26](#)
 - operator<<, [27](#)
 - operator>>, [27](#)
 - operator=, [27](#)
 - readStudent, [27](#)
 - reservePaz, [27](#)
 - setEgz, [27](#)
 - setMed, [27](#)
 - setPaz, [27](#)
 - setRez, [27](#)
 - Studentas, [25](#), [26](#)
- swap
 - Vector< T >, [38](#)
- v
 - VectorTest, [42](#)
- v3.0 relisas: Custom Vector<T> Konteineris, [1](#)
- value_type
 - VectorIterator< T >, [40](#)
- vardas_
 - Zmogus, [45](#)
- Vector
 - Vector< T >, [30](#), [31](#)
- Vector< T >, [28](#)
 - ~Vector, [31](#)
 - assign, [31](#)
 - at, [31](#), [32](#)
 - back, [32](#)
 - begin, [32](#)
 - capacity, [32](#)
 - cbegin, [32](#)
 - cend, [32](#)
 - clear, [32](#)
 - const_iterator, [30](#)
 - const_reverse_iterator, [30](#)
 - crbegin, [32](#)
 - crend, [33](#)
 - data, [33](#)
 - emplace, [33](#)
 - emplace_back, [33](#)
 - empty, [33](#)
 - end, [33](#)
 - erase, [33](#), [34](#)
 - front, [34](#)
 - get_reallocation_count, [34](#)
 - insert, [34](#), [35](#)
 - iterator, [30](#)
 - max_size, [35](#)
 - operator!=, [35](#)
 - operator<, [35](#)
 - operator<=, [35](#)
 - operator>, [36](#)
 - operator>=, [36](#)
 - operator=, [35](#)
 - operator==, [36](#)
 - operator[], [36](#)
 - pop_back, [36](#)
 - push_back, [36](#)
 - rbegin, [37](#)
 - rend, [37](#)
 - reserve, [37](#)
 - reset_reallocation_count, [37](#)
 - resize, [37](#)
 - reverse_iterator, [30](#)
 - shrink_to_fit, [37](#)
 - size, [38](#)
 - swap, [38](#)
 - Vector, [30](#), [31](#)
- vector< T >, [38](#)
- vector< T >::const_iterator, [19](#)
- vector< T >::const_reverse_iterator, [20](#)
- vector< T >::iterator, [22](#)
- vector< T >::reverse_iterator, [23](#)
- VectorIterator
 - VectorIterator< T >, [40](#)
- VectorIterator< T >, [38](#)
 - difference_type, [39](#)
 - iterator_category, [39](#)
 - operator!=, [40](#)
 - operator<, [41](#)
 - operator<=, [41](#)
 - operator>, [42](#)
 - operator>=, [42](#)
 - operator+, [40](#)
 - operator++, [40](#)
 - operator+=, [40](#)
 - operator-, [41](#)
 - operator->, [41](#)
 - operator--, [41](#)
 - operator-=, [41](#)
 - operator==, [41](#)
 - operator[], [42](#)
 - operator*, [40](#)
 - pointer, [39](#)
 - reference, [40](#)
 - value_type, [40](#)
 - VectorIterator, [40](#)
- VectorTest, [42](#)
 - v, [42](#)
- Zmogus, [43](#)
 - ~Zmogus, [44](#)
 - getInfo, [44](#)
 - getPavarde, [44](#)
 - getVardas, [44](#)
 - operator<<, [45](#)
 - pavarde_, [45](#)
 - setPavarde, [44](#)

setVardas, [45](#)
vardas_, [45](#)
Zmogus, [44](#)